

1. 시계열의 구성과 확률보행

학습목표

- ▶ 시계열 소개
- ▶ 시계열의 세 가지 주요 구성요소 이해
- ▶ 베이스라인 모델 구축 및 평가지표 이해
- ▶ 확률보행 프로세스 식별, 자기상관함수 이해
- ▶ 차분, 정상성, 백색소음 분류
- ▶ AutoCorrelation Function 도식화
- ▶ 확률보행 예측

1. 시계열의 구성요소 이해

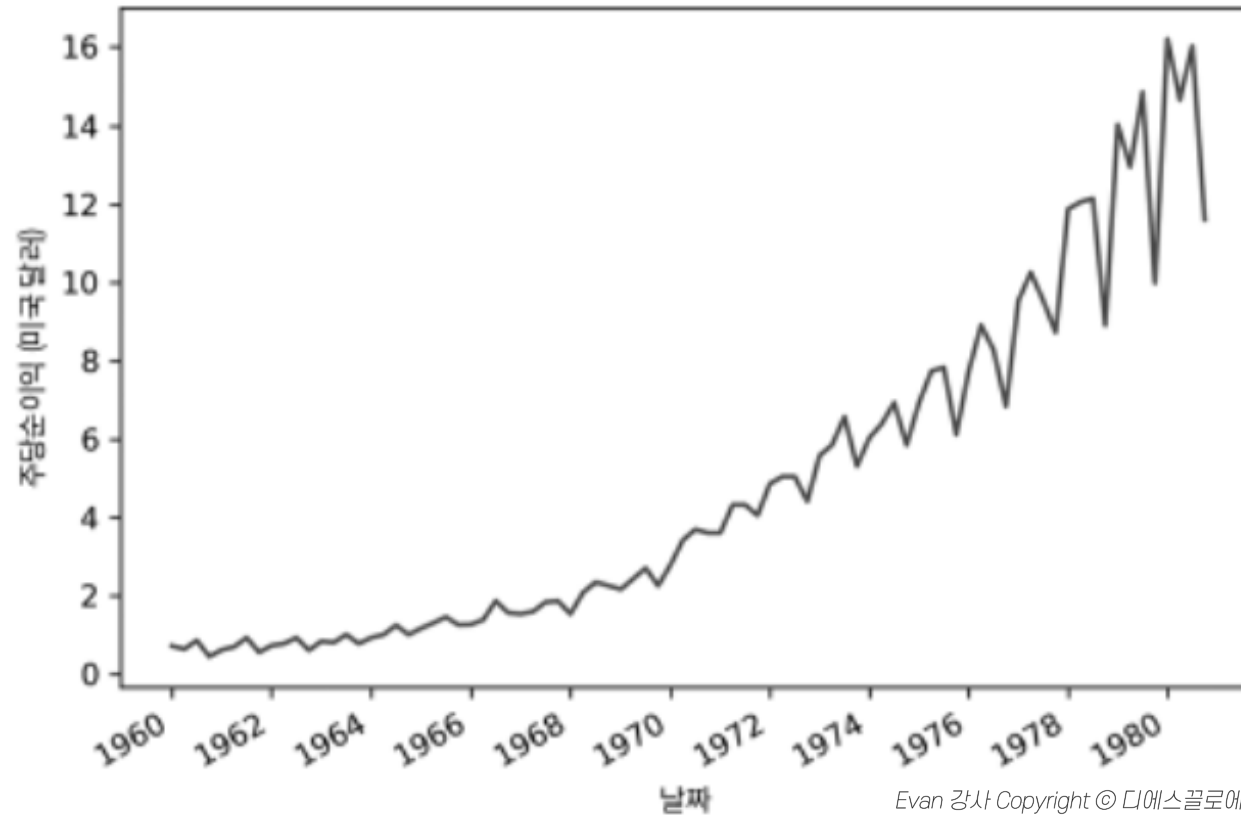
◆ 시계열

- ✓ 시간에 따라 정렬된 데이터 요소의 집합
 - 동일한 시간 단계로 분포하는데, 이는 매시간, 매분, 매월, 매분기 등의 주기로 기록됨
 - 주식의 종가, 가정의 전기 소비량, 실외 온도

1. 시계열의 구성요소 이해

◆ 시계열 그래프 예시

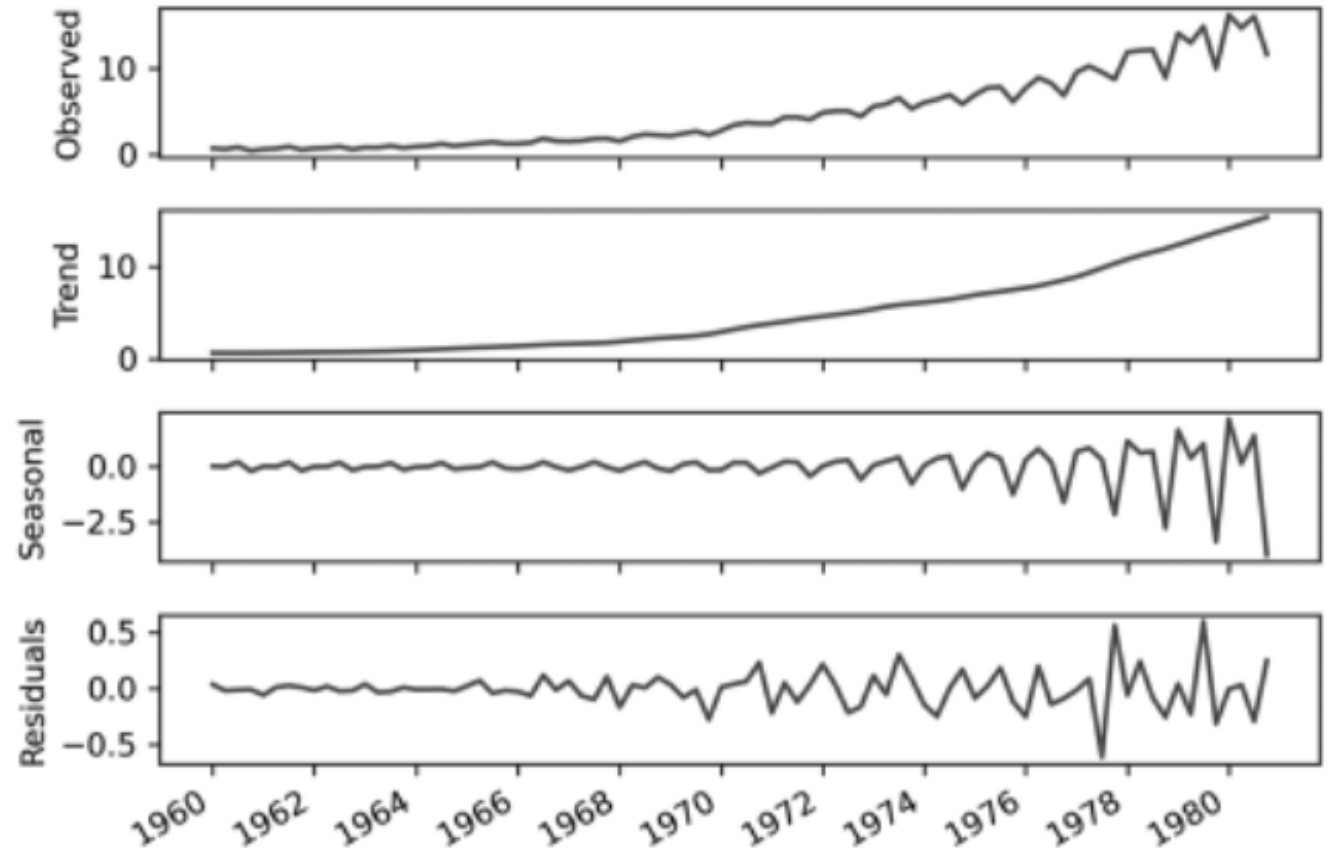
- ✓ 존슨앤드존슨의 분기별 수익(1960년~1980년)



1. 시계열의 구성요소 이해

◆ 세 가지 구성요소

- ✓ 추세Trend
- ✓ 계절성Seasonality
- ✓ 잔차Residual
- ✓ 시계열 분해



1. 시계열의 구성요소 이해

◆ 시계열 분해 $\Rightarrow$ 세 가지 구성요소로 분리

- ✓ 추세^{Trend} : 시계열의 장기적인 변화
- ✓ 계절성^{Seasonality} : 계절적인 변동 (1년 4번의 분기 동안 주당 순이익이 낮게 시작 연말에 감소)
- ✓ 잔차^{Residual} : 추세 및 계절성으로 설명할 수 없는 행태 (백색소음)

1. 시계열의 구성요소 이해

◆ 예측Forecasting 프로세스

- ✓ 목적 설정
- ✓ 목적 달성하기 위해 무엇을 예측해야 하는지 결정
- ✓ 예측할 기간 설정
- ✓ 데이터 수집
- ✓ 예측 모델 개발
- ✓ 상용 환경에 배포
- ✓ 모니터링
- ✓ 새로운 데이터 수집

1. 시계열의 구성요소 이해

◆ 시계열 예측의 주요 특징

- ✓ 수치 예측을 한다는 점에서 회귀와 비슷하지만, 시간의 순서 관점에서 다름
 - 시계열 : 시간의 순서를 지켜야 함
 - 회귀 : 시간의 순서가 존재하지 않는 경우가 많음
- ✓ 시계열은 그 자체 이외에 다른 특징 활용하지 않고 시계열만 활용해서 예측

2. 베이스라인 모델 정의

◆ Baseline Model

- ✓ 간단한 통계 기법 활용 (산술평균), 예시
 - 1960년부터 1979년까지 평균 주당순이익 : \$4.31
 - 1980년 분기당 \$4.31 예측

2. 베이스라인 모델 정의

◆ 예측 성능 평가

- ✓ 평가지표 : 평균절대백분율오차(Mean Absolute Percentage Error, MAPE)
 - 해석하기 쉽고, 데이터의 규모에 영향을 받지 않음

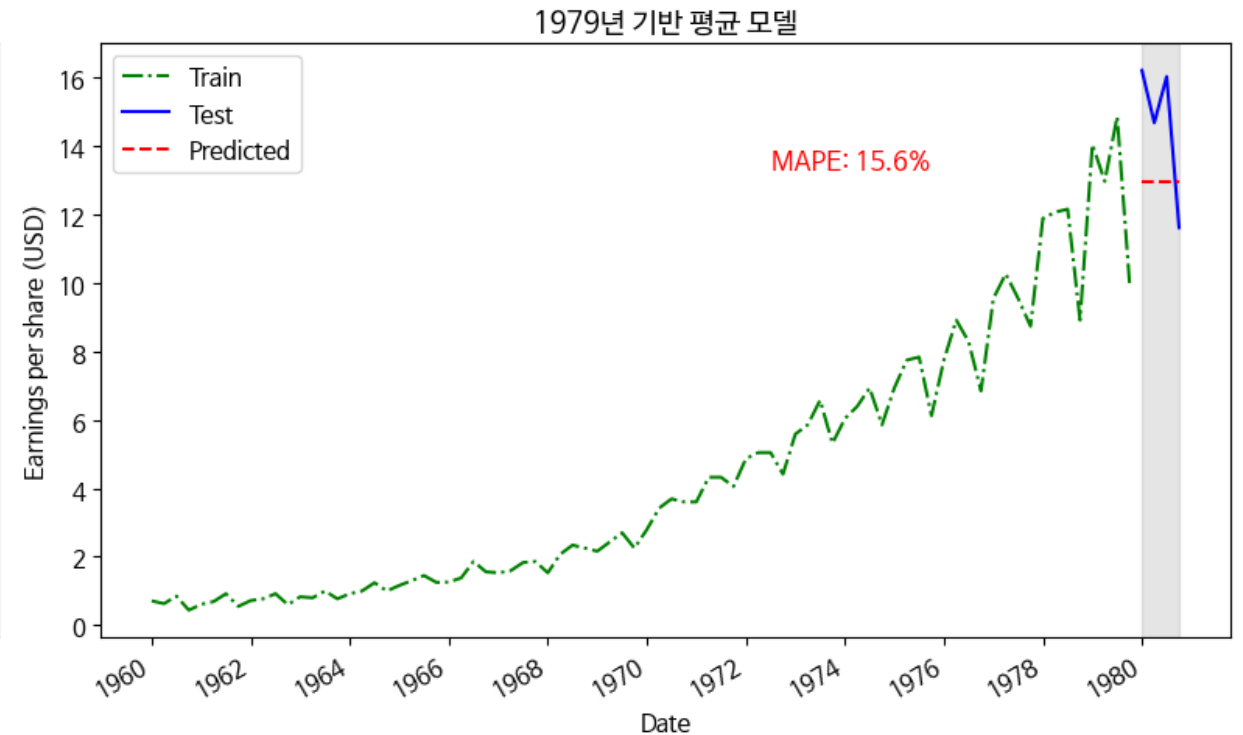
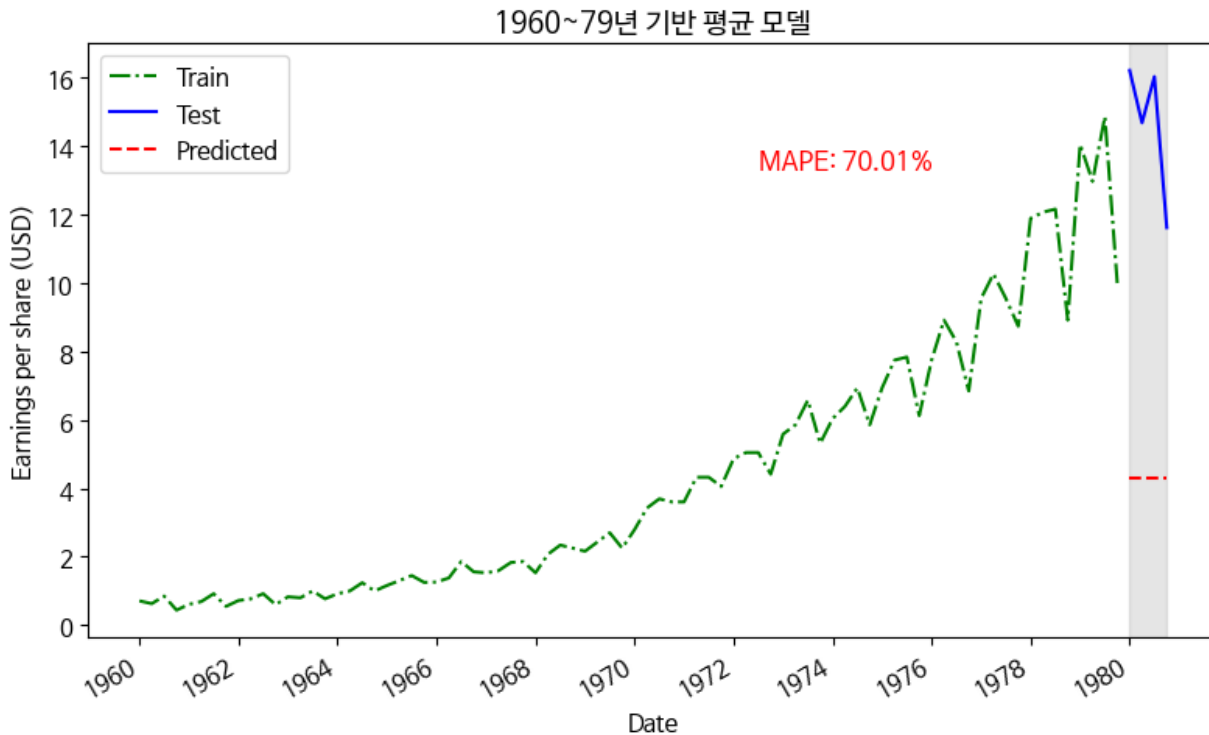
$$\text{MAPE} = \frac{1}{n} \sum_{i=1}^n \left| \frac{A_i - F_i}{A_i} \right| \times 100$$

- 용어
 - A_i : 실제값
 - F_i : 예측값
 - n : 예측 횟수

2. 베이스라인 모델 정의

◆ 두 모델 비교

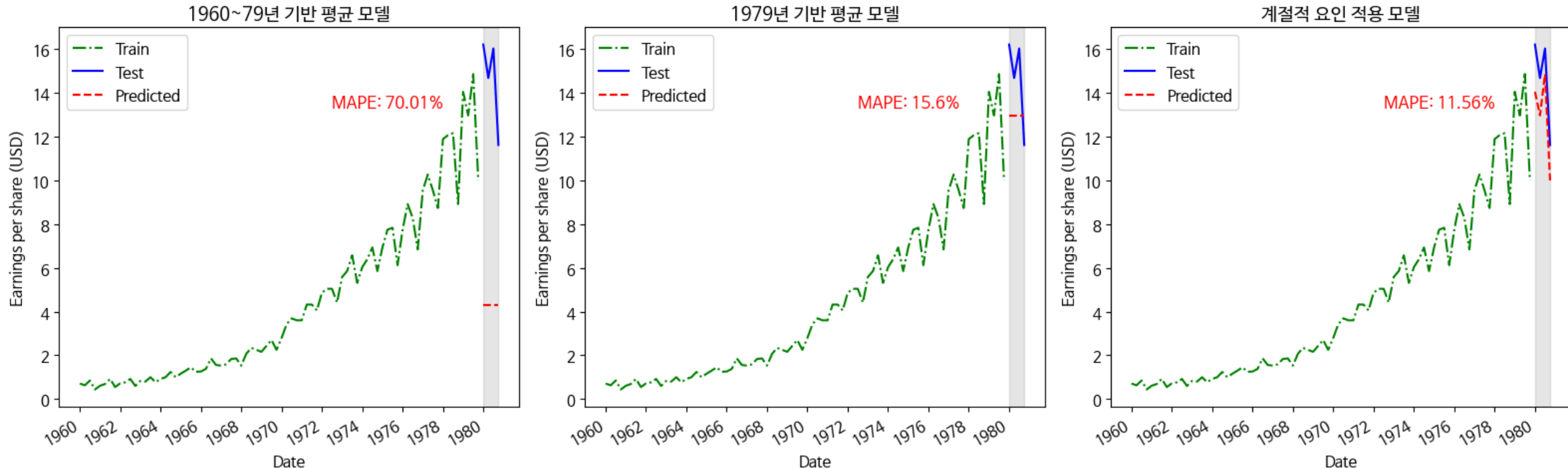
- ✓ 예측값이 너무 멀지 않은 과거의 값과 연관이 있을 가능성이 높음 확인 가능 (자기상관)



2. 베이스라인 모델 정의

◆ 두 모델 비교

- ✓ 계절적 요인 적용 시, 가장 좋은 성능을 보여줌



3. 확률보행 프로세스

◆ 확률보행 시계열 모델 프로세스

- ✓ 정상성Stationary 확인
- ✓ 만약 비정상성이라면
 - 차분Differencing을 통해 정상성으로 변환
- ✓ 자기상관함수ADF: Auto-Correlation Function를 도식화하여 확률보행 시계열 모델로 근사화Approximation
- ✓ 시계열 예측

3. 확률보행 프로세스

◆ 확률보행 수식

$$y_t = C + y_{t-1} + \varepsilon_t$$

✓ 용어 정리

- y_t : 현재값
- y_{t-1} : 이전 시간 단계의 값
- C : 상수
- ε_t : 난수

3. 확률보행 프로세스

◆ 확률보행 수식 시뮬레이션

$$y_t = C + y_{t-1} + \varepsilon_t$$

- ✓ 만약, 상수 C 가 0이면, 초깃값 y_t 을 0으로 지정하면 아래와 같은 수식으로 다시 정리가 될 수 있음

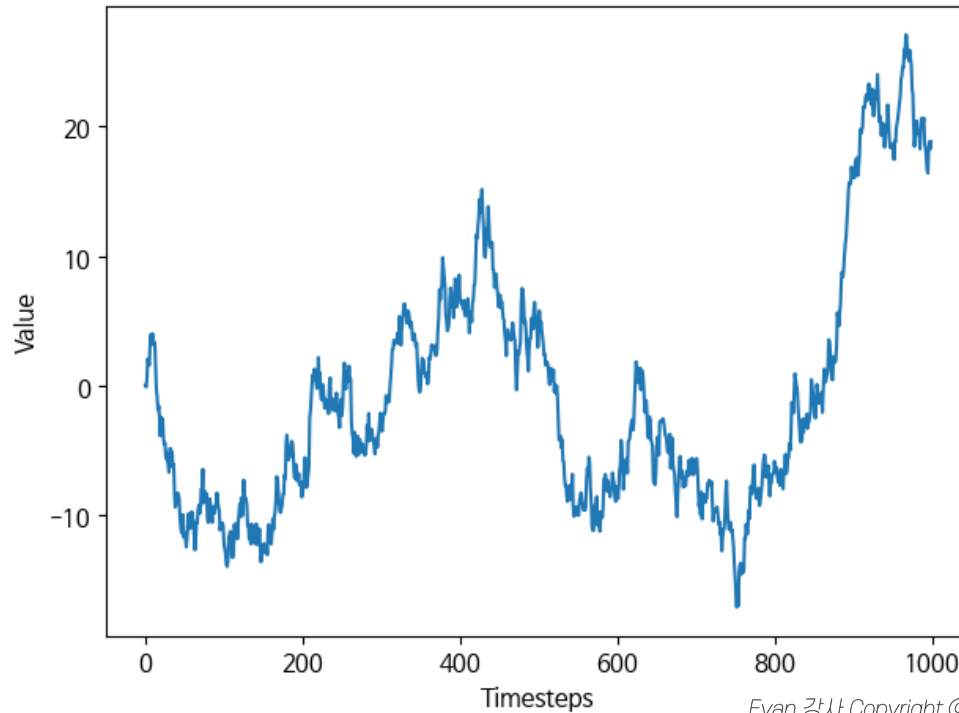
$$y_t = \sum_{t=1}^T \varepsilon_t$$

- ✓ 수식 해석
 - 임의의 시점 t 에서 시뮬레이션된 시계열의 값이 일련의 난수들의 누적 합임을 표현

3. 확률보행 프로세스

◆ 확률보행 수식 시뮬레이션

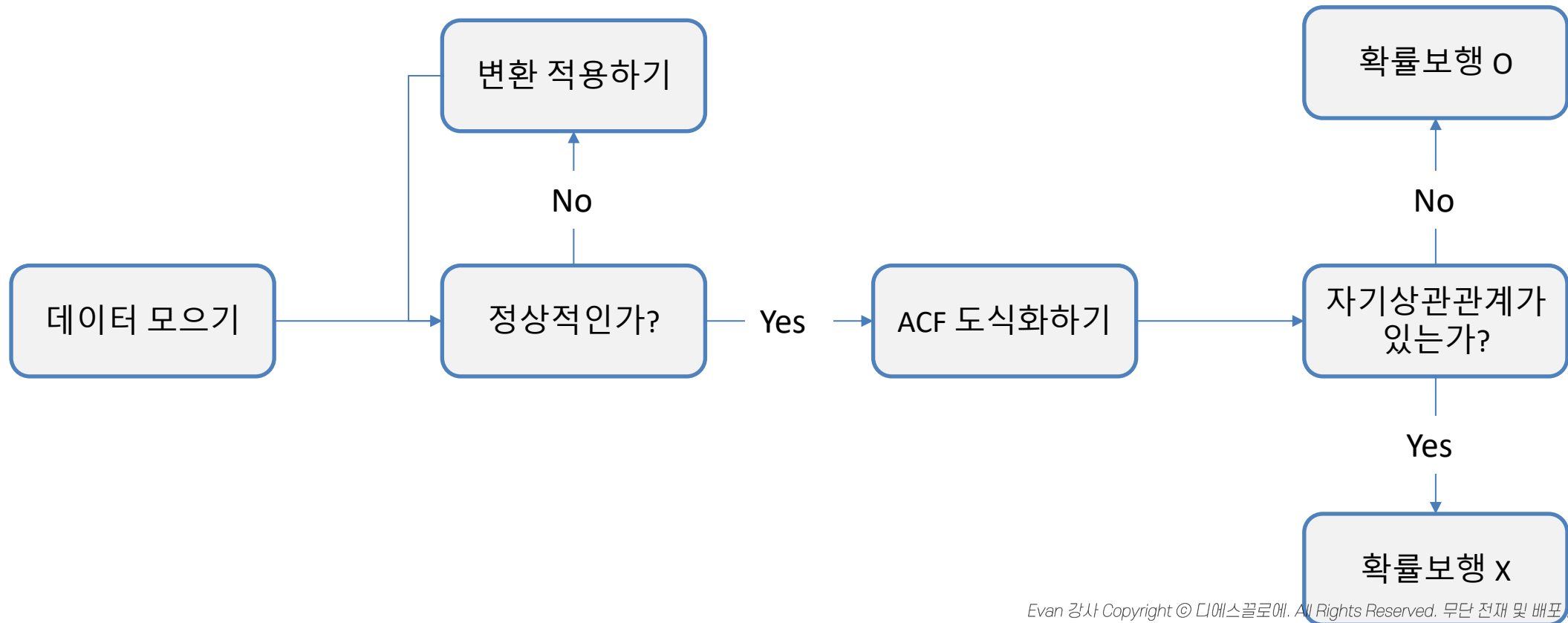
- ✓ 1,000개의 샘플
- ✓ 백색소음(난수)의 누적합계 계산



3. 확률보행 프로세스

◆ 확률보행 식별

- ✓ 확률보행의 정의 : 첫 번째 차분이 정상적이고 상관관계는 없는 시계열로 정의



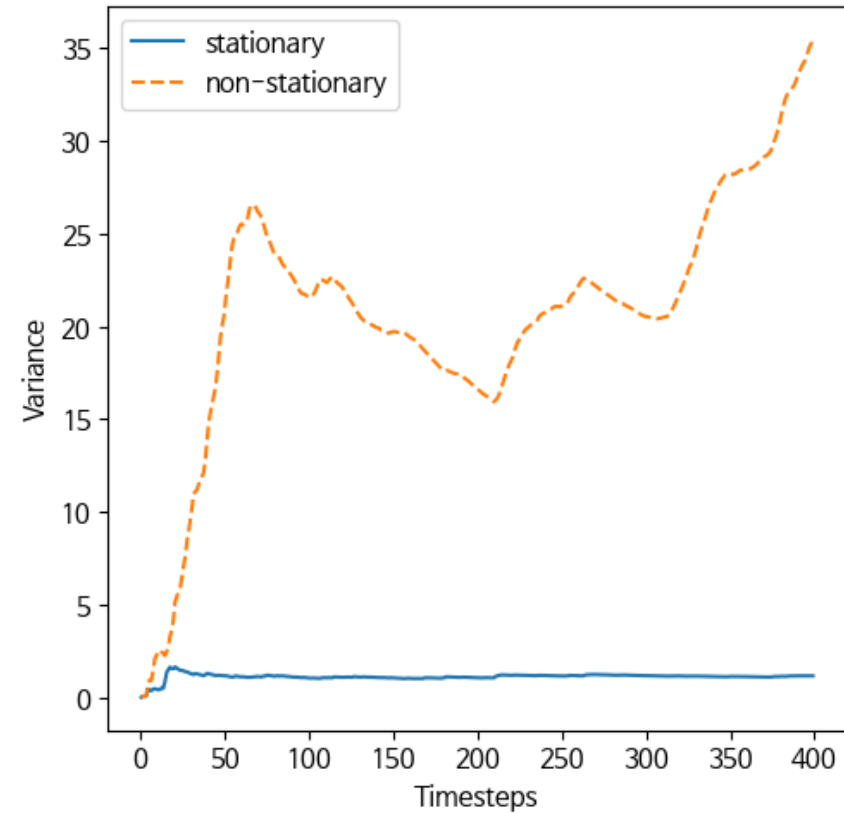
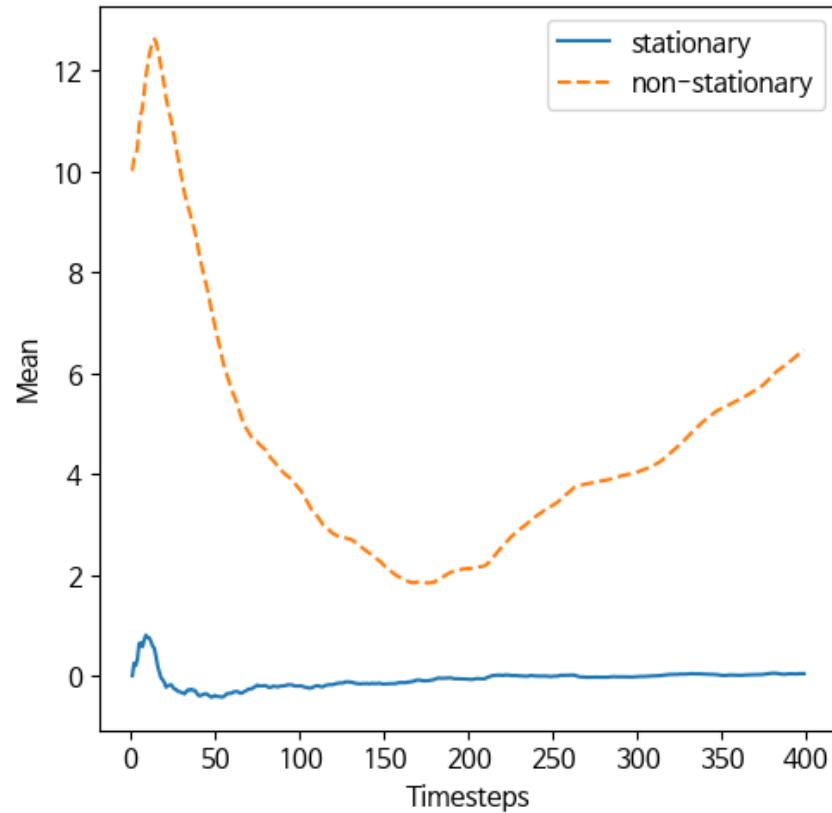
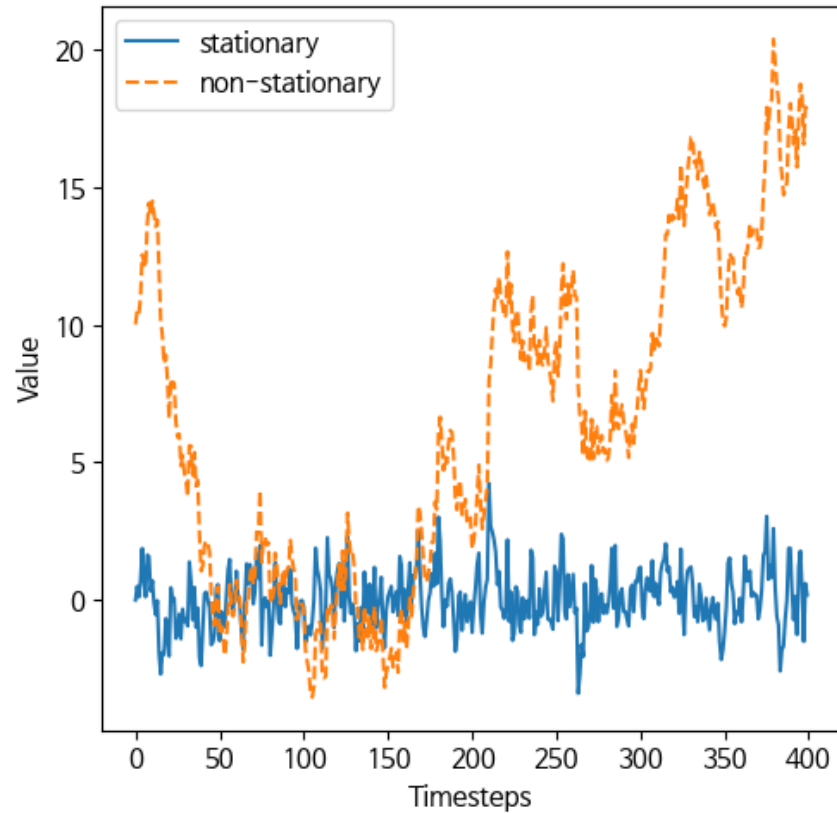
3. 확률보행 프로세스

◆ 정상성 테스트

- ✓ ADF : Augmented Dickey-Fuller
- ✓ 시계열 변환 적용 후, 정상성 테스트 수행하여 다른 변환 더 적용해야 하는지 확인 (ADF)
- ✓ 시계열 가설 검정
 - 귀무가설 : 단위근 unit root 이 존재 (시계열이 비정상적)
 - 대립가설 : 단위근이 없으므로 시계열이 정상적이라는 가설

3. 확률보행 프로세스

◆ 정상성 테스트



3. 확률보행 프로세스

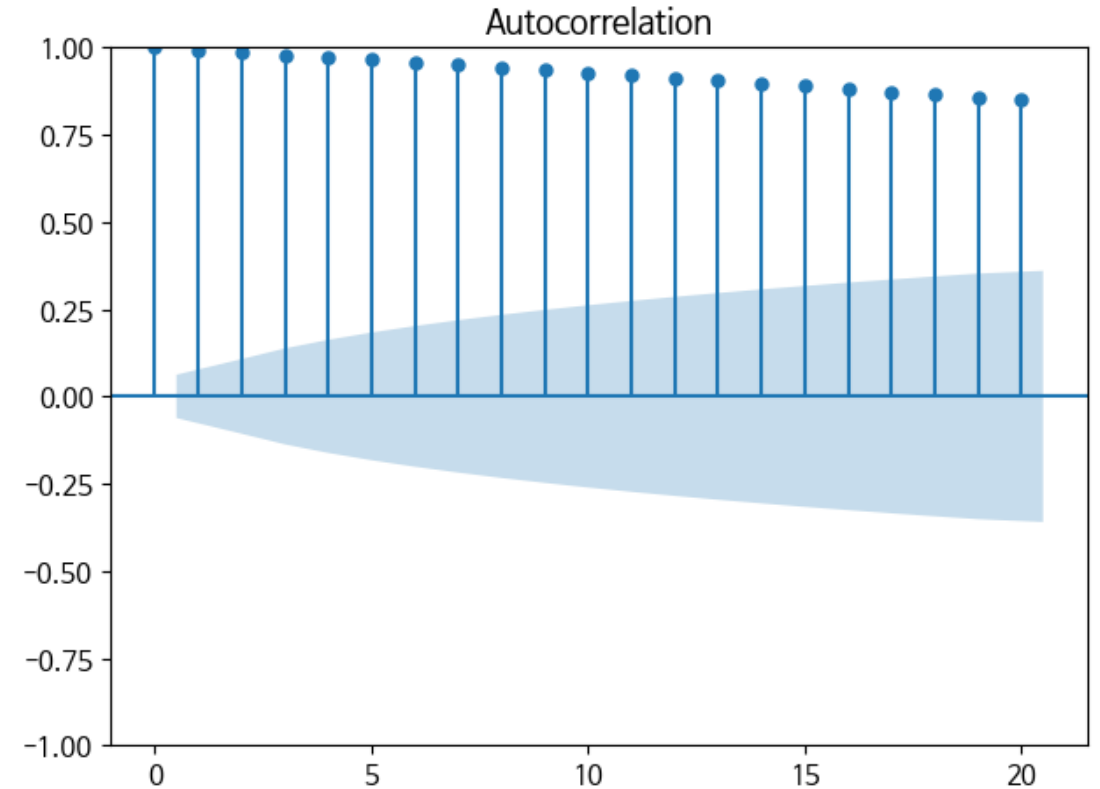
◆ 자기상관관계 Autocorrelation 테스트

- ✓ 상관관계 : 두 변수 사이의 선형 관계 측정
- ✓ 자기상관관계 : 시계열의 선행값과 후행값 사이의 선형 관계 측정
- ✓ 지연 lag 이 증가함에 따라 두 값 사이의 상관관계가 어떻게 변하는지 보여줌
 - 두 값 사이의 시간 단계
 - $\text{lag } 1 : y_t, y_{t-1}$ 사이의 관계
 - $\text{lag } 2 : y_t, y_{t-2}$ 사이의 관계
 - $\text{lag } 0$: 자기상관계수는 언제나 1
- ✓ 추세가 존재 : lag 숫자가 커지면서 계수가 선형적으로 감소
- ✓ 계절성이 존재 : 주기적인 패턴 확인

3. 확률보행 프로세스

◆ 자기상관관계 Autocorrelation 테스트

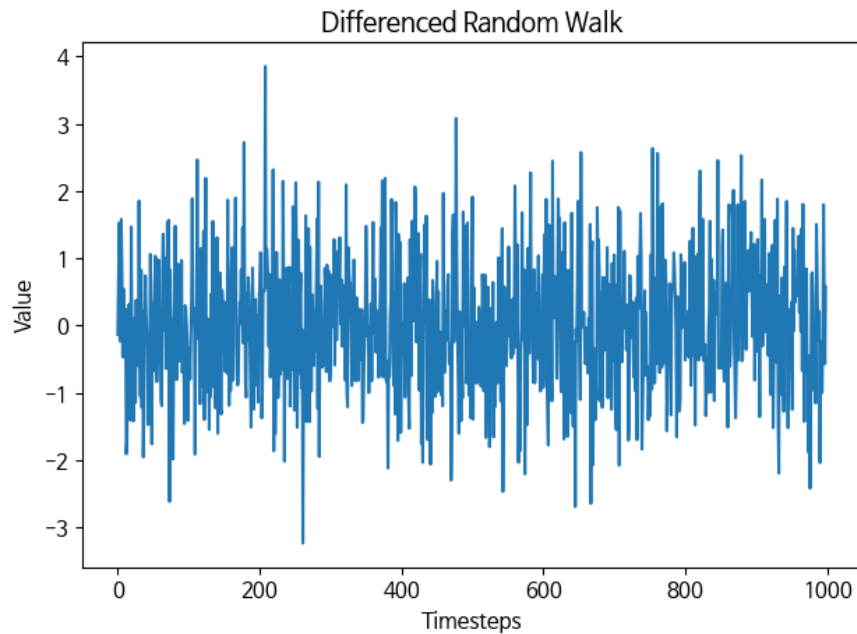
- ✓ 비정상 시계열의 자기상관관계 시각화 예시
 - 지연이 증가함에 따라 서서히 감소
 - 시계열은 정상 시계열이 아님
 - 지연 20에서도 상관계수 존재
 - 해결방안 : 차분



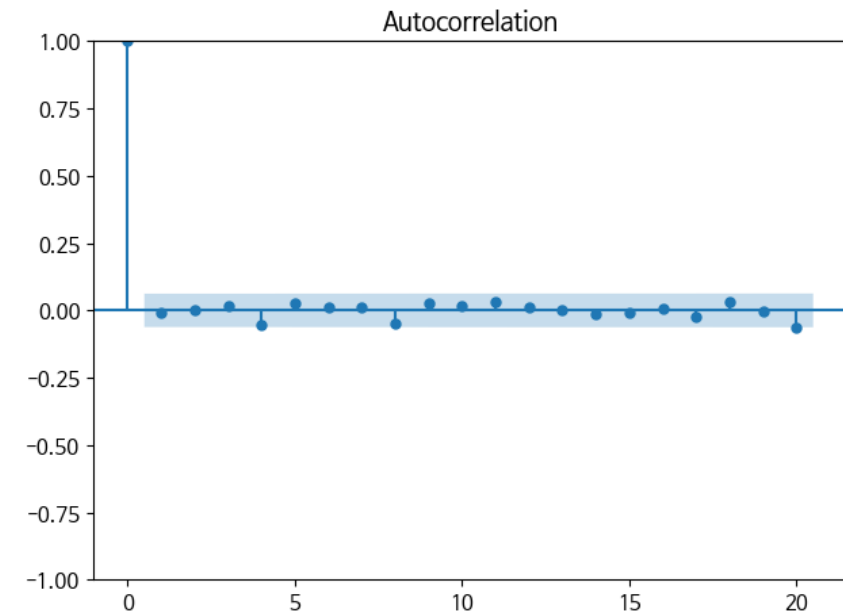
3. 확률보행 프로세스

◆ 자기상관관계 Autocorrelation 테스트

- ✓ 정상 시계열의 자기상관관계 시각화 예시 (정상적인 확률보행 시각화)
 - 차분 진행 후, 시각화



추세 제거하여 분산 안정화 된 그래프

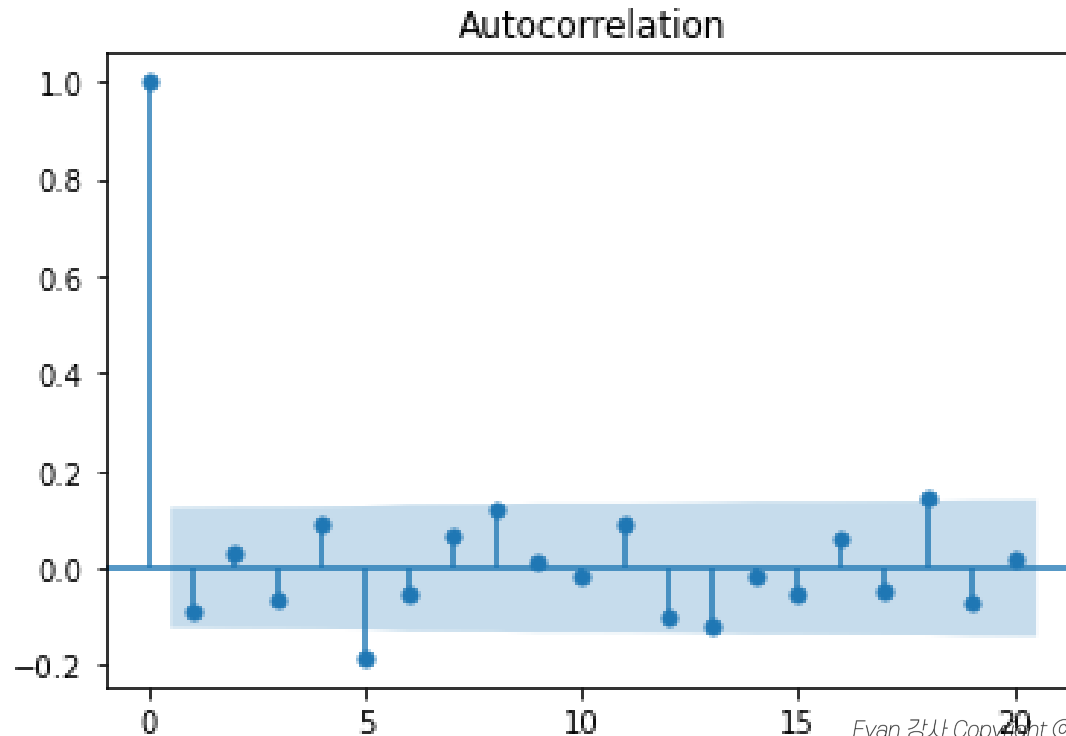


지연 0 이후에 유의한 상관계수 없음

3. 확률보행 프로세스

◆ 자기상관관계 Autocorrelation 테스트

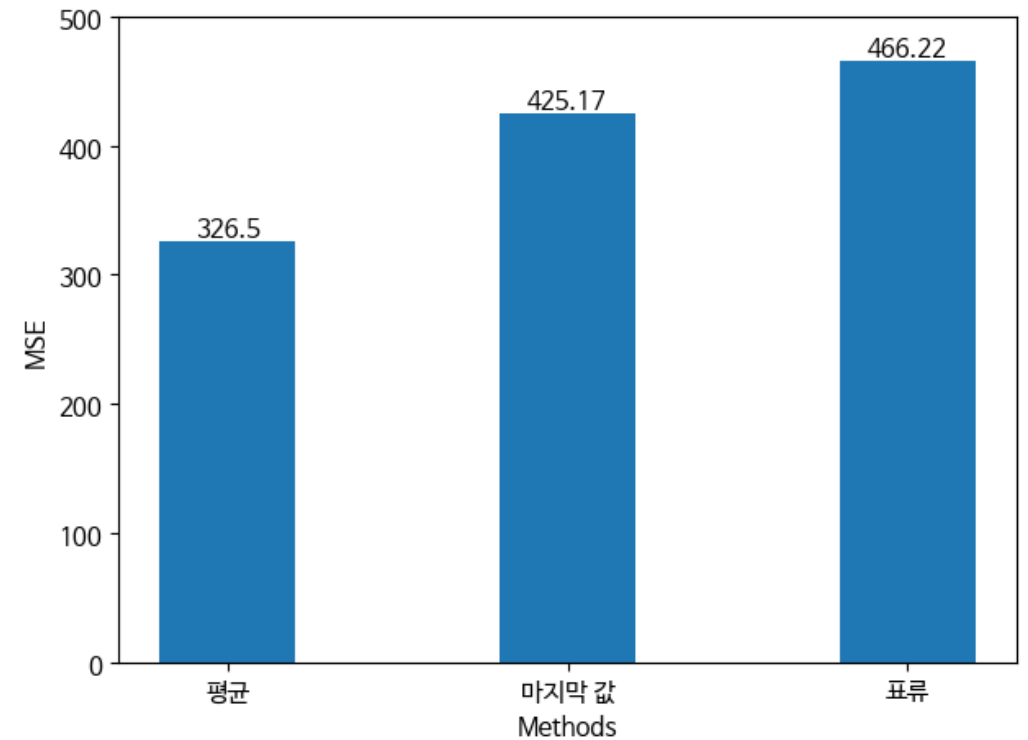
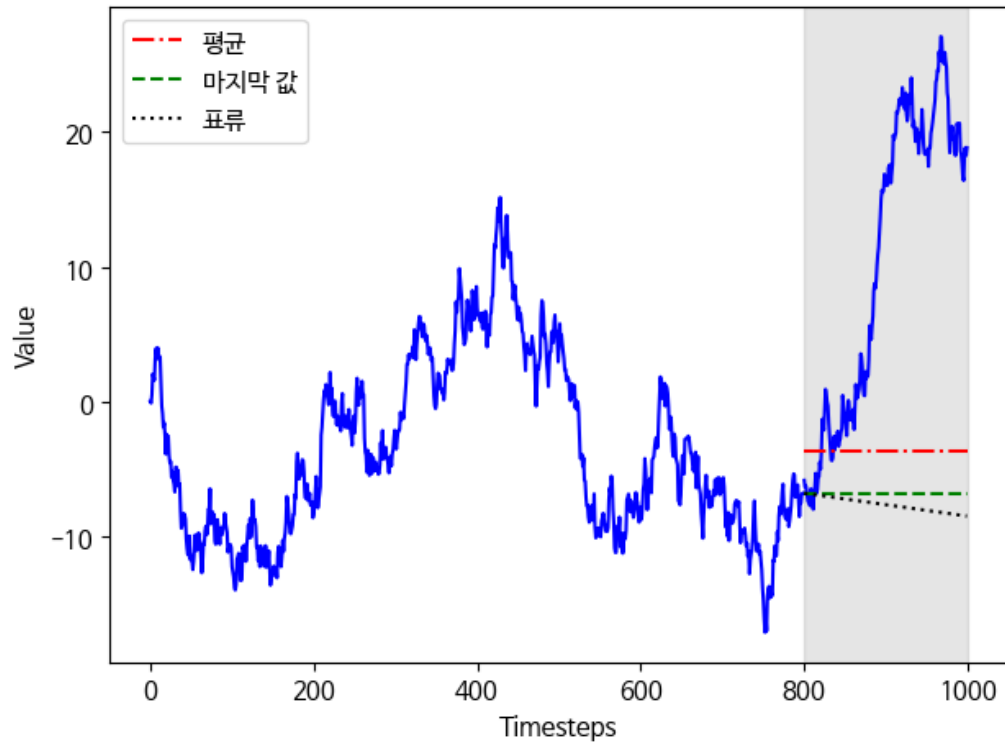
- ✓ ACF 도식에서 유의한 상관계수 없음
- ✓ 5와 18은 유의하지 않기는 하지만, 우연한 상황으로 추정 가능



3. 확률보행 프로세스

◆ 확률보행 예측

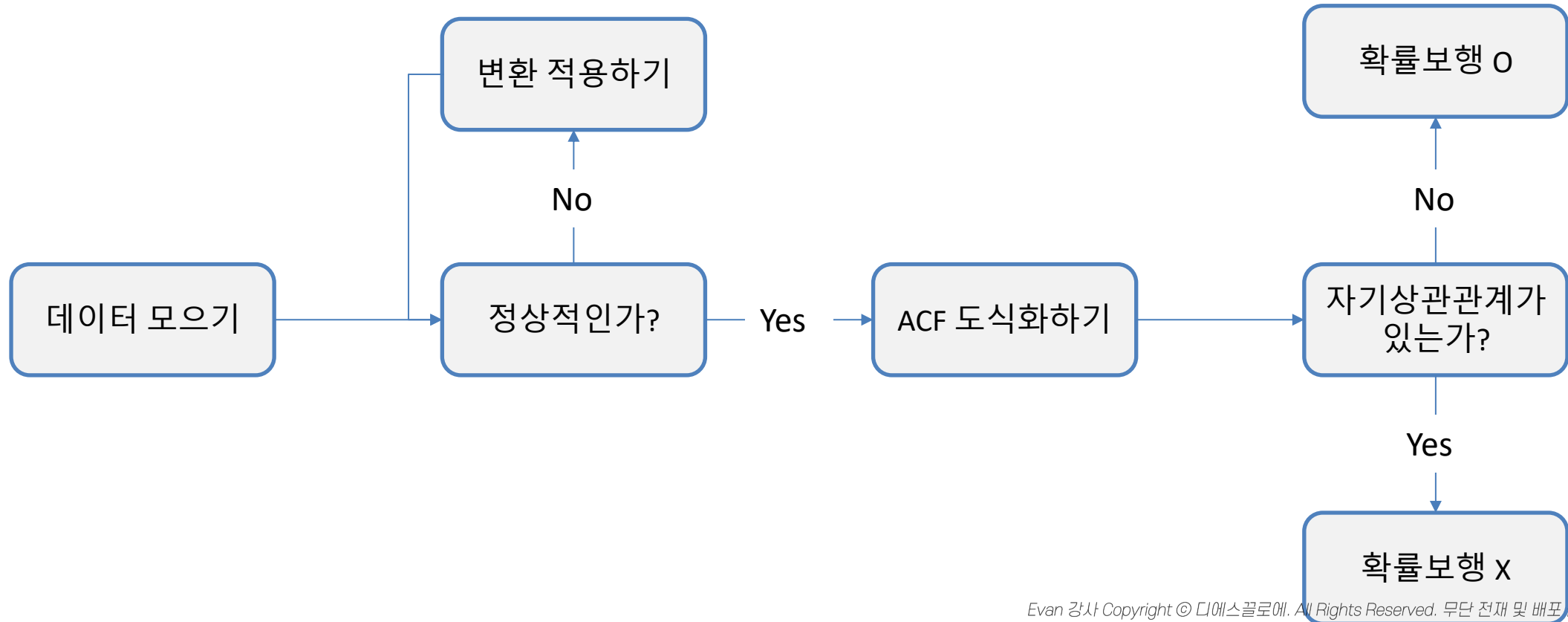
✓ 추세 제거 및 ACF 테스트 통과 ⇒ 미래 예측 가능



3. 확률보행 프로세스

◆ 확률보행 식별

- ✓ 확률보행의 정의 : 첫 번째 차분이 정상적이고 상관관계는 없는 시계열로 정의



2. 통계적 모델을 활용한 시계열 예측

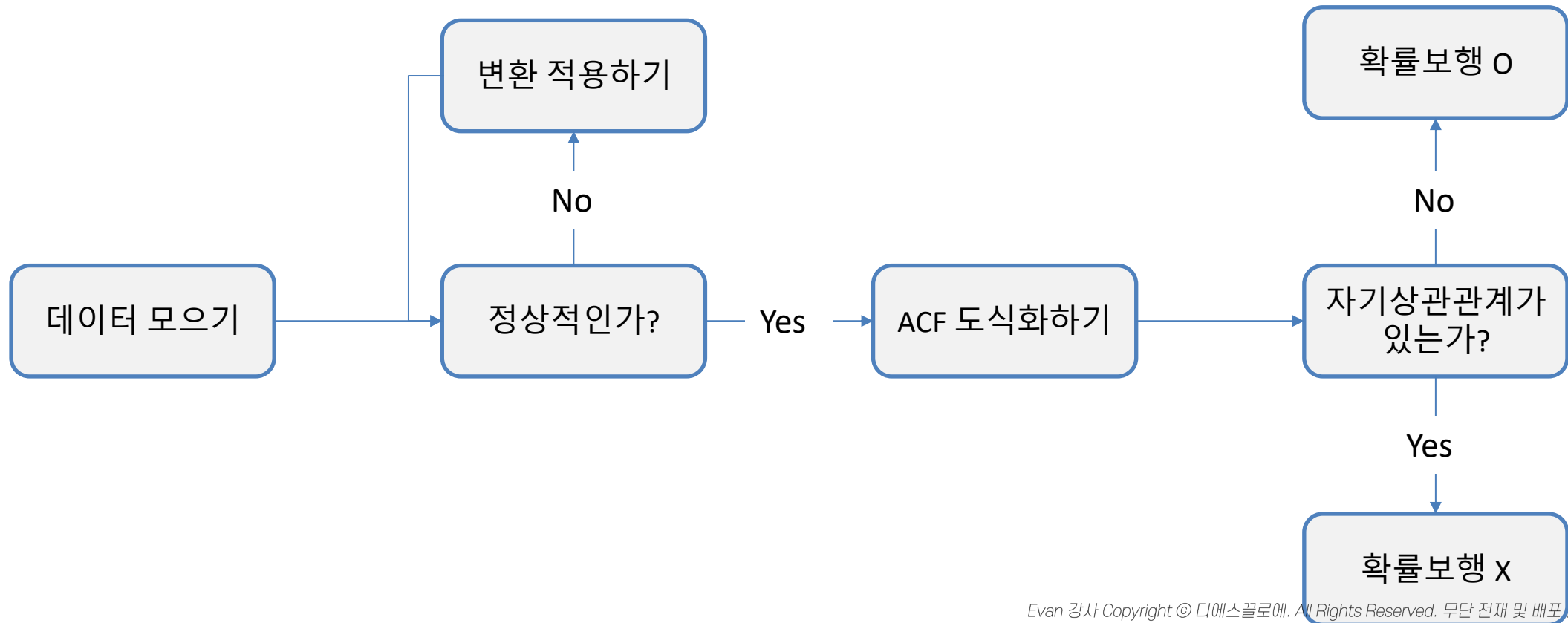
학습목표

- ▶ 이동평균과정 정의
- ▶ ACF를 사용하여 이동평균과정의 차수 식별
- ▶ 이동평균 모델을 사용하여 시계열 예측

1. 이동평균 모델링

◆ 확률보행 식별

- ✓ 확률보행의 정의 : 첫 번째 차분이 정상적이고 상관관계는 없는 시계열로 정의



1. 이동평균 모델링

◆ 이동평균과정 정의

- ✓ 이동평균MA: Moving Average Process은 현재값이 현재와 과거 오차에 선형적으로 비례
 - 오차가 백색소음과 마찬가지로 상호 독립적이며 정규분포한다고 가정
 - 오차항은 정규분포를 따름
- ✓ 이동평균 모델 = $MA(q)$, q 는 차수

$$y_t = \mu + \varepsilon_t + \theta_1 \varepsilon_{t-1} + \theta_2 \varepsilon_{t-2} + \cdots + \theta_q \varepsilon_{t-q}$$

- 용어 정리
 - y_t : 현재값
 - μ : 수열의 평균
 - ε_t : 현재 오차항
 - ε_{t-q} : 과거 오차항
 - θ_q : 현재값에 과거 오차가 미치는 영향의 크기

1. 이동평균 모델링

◆ 이동평균과정 정의

- ✓ 이동평균 모델의 차수 q 정의
 - q 가 클수록 더 많은 과거 오차 항이 현재값에 영향을 미침
- ✓ MA(1)

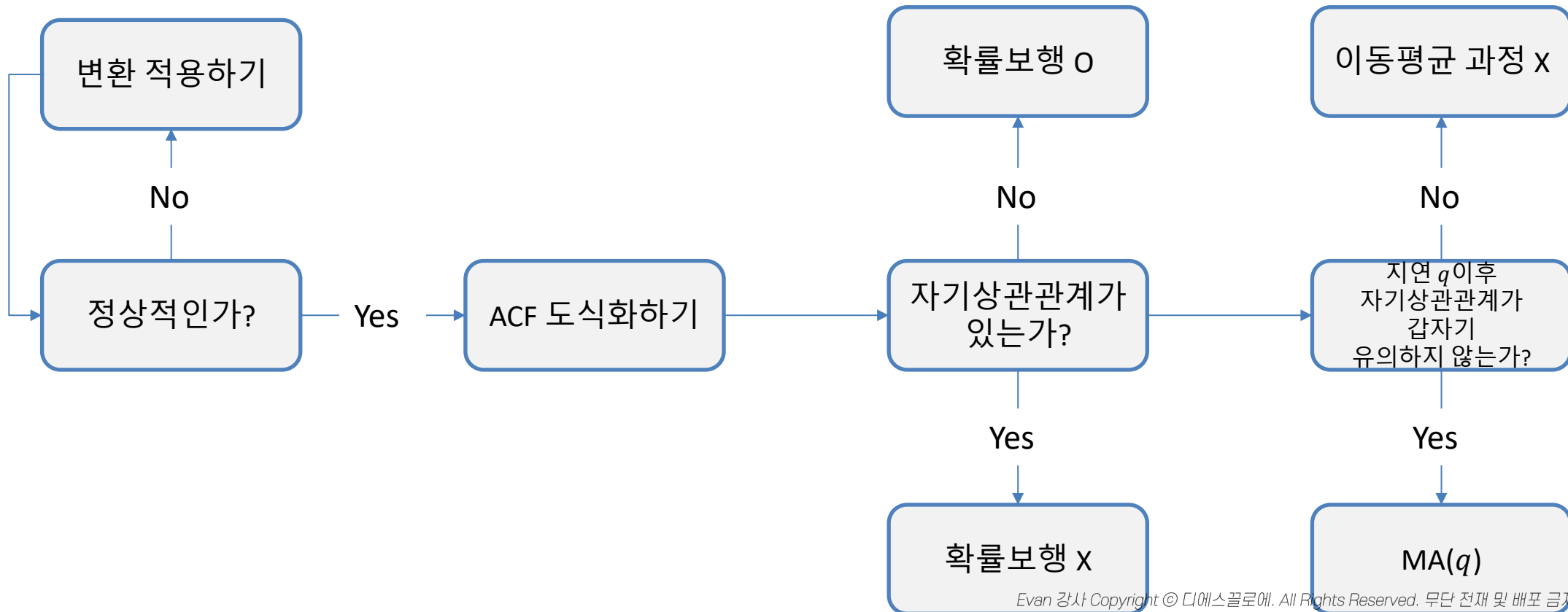
$$y_t = \mu + \varepsilon_t + \theta_1 \varepsilon_{t-1}$$

- ✓ MA(2)

$$y_t = \mu + \varepsilon_t + \theta_1 \varepsilon_{t-1} + \theta_2 \varepsilon_{t-2}$$

1. 이동평균 모델링

◆ 이동평균과정 차수 식별

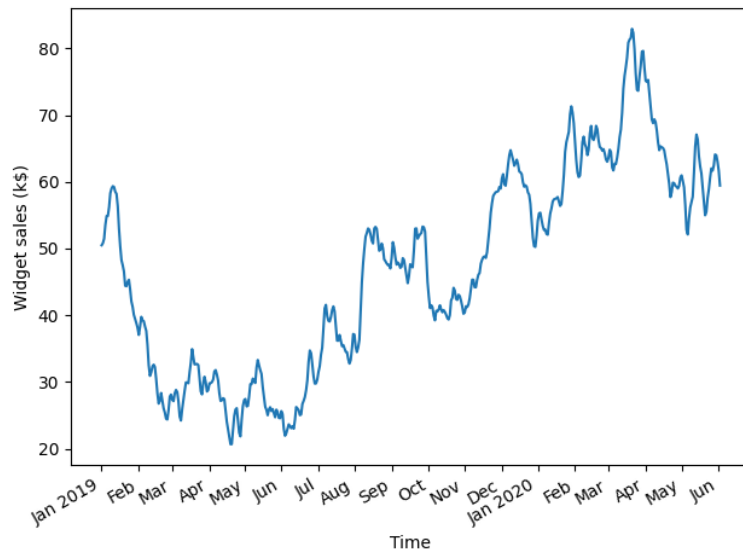


1. 이동평균 모델링

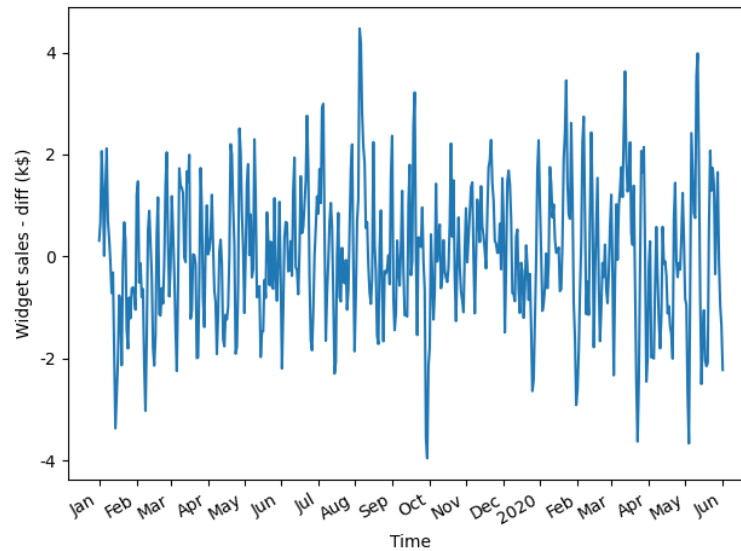
◆ 이동평균과정 차수 식별

✓ 지연 2까지 유의한 계수 확인 $\Rightarrow$ MA(2) 모델

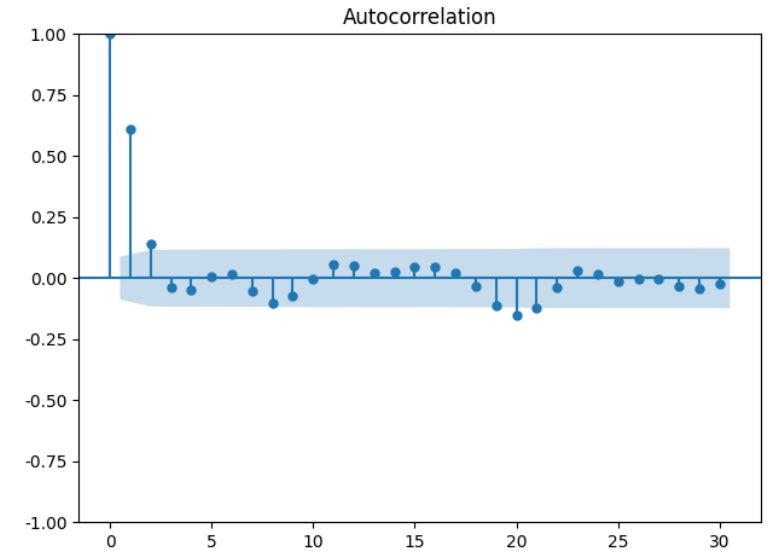
xyz 위젯 회사의 위젯 판매량 데이터



1차 차분한 데이터 시각화 $\Rightarrow$ 분산 대략 안정



자기상관관계 시각화



1. 이동평균 모델링

◆ 이동평균과정 예측하기

- ✓ 전체 데이터 500일, 훈련데이터와 테스트데이터 비율 9:1
- ✓ 이동평균 모델은 한 번에 50단계 앞 예측 할 수 없음
 - 이동평균 모델의 특수성
 - 과거 오차 향에 선형적으로 의존
 - 데이터 집합에서 관찰되지 않기 때문에 재귀적으로 추정
- ✓ $MA(q)$ 모델은 앞으로의 q 단계까지만 예측

1. 이동평균 모델링

◆ 이동평균과정 예측하기

✓ 롤링 예측^{rolling forecast}

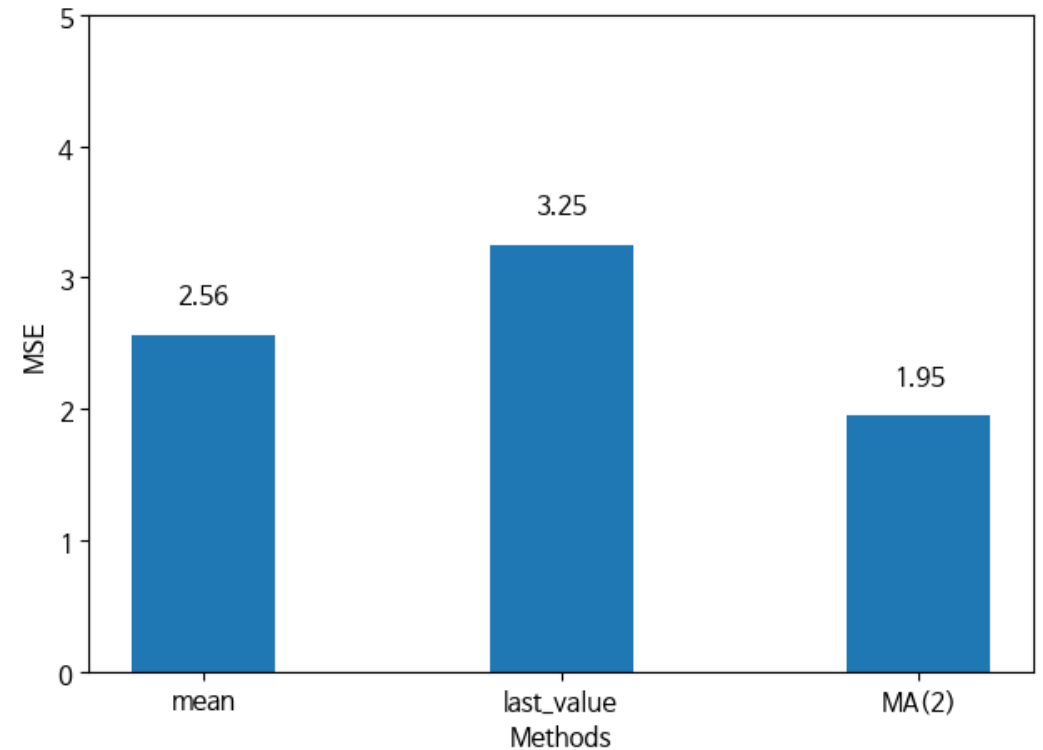
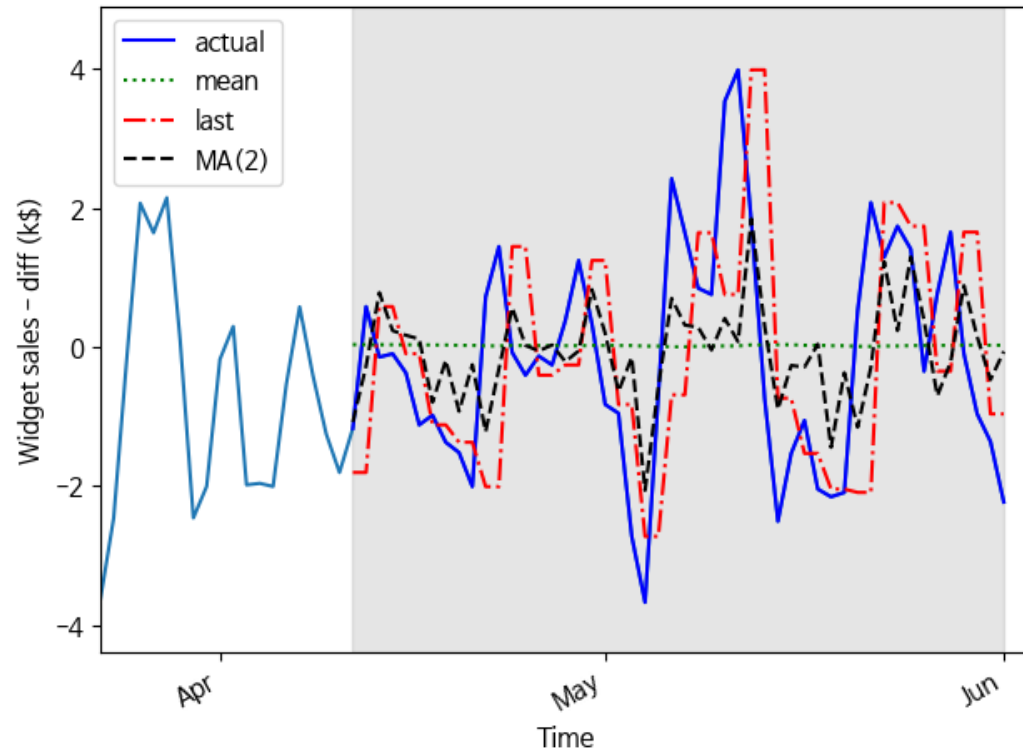
- MA(2) 모델 가정
- 첫번째 반복 : 첫번째~449번째 시간 단계 훈련, 450번째, 451번째 시간 단계 예측
- 두번째 반복 : 첫번째~451번째 시간 단계 훈련, 452번째와 453번째 시간 단계 예측

구분	1	449	450	451	452	453	...	498	499
첫번째 반복	훈련데이터		예측데이터						
두번째 반복	훈련데이터				예측데이터				
...									
마지막								예측데이터	

1. 이동평균 모델링

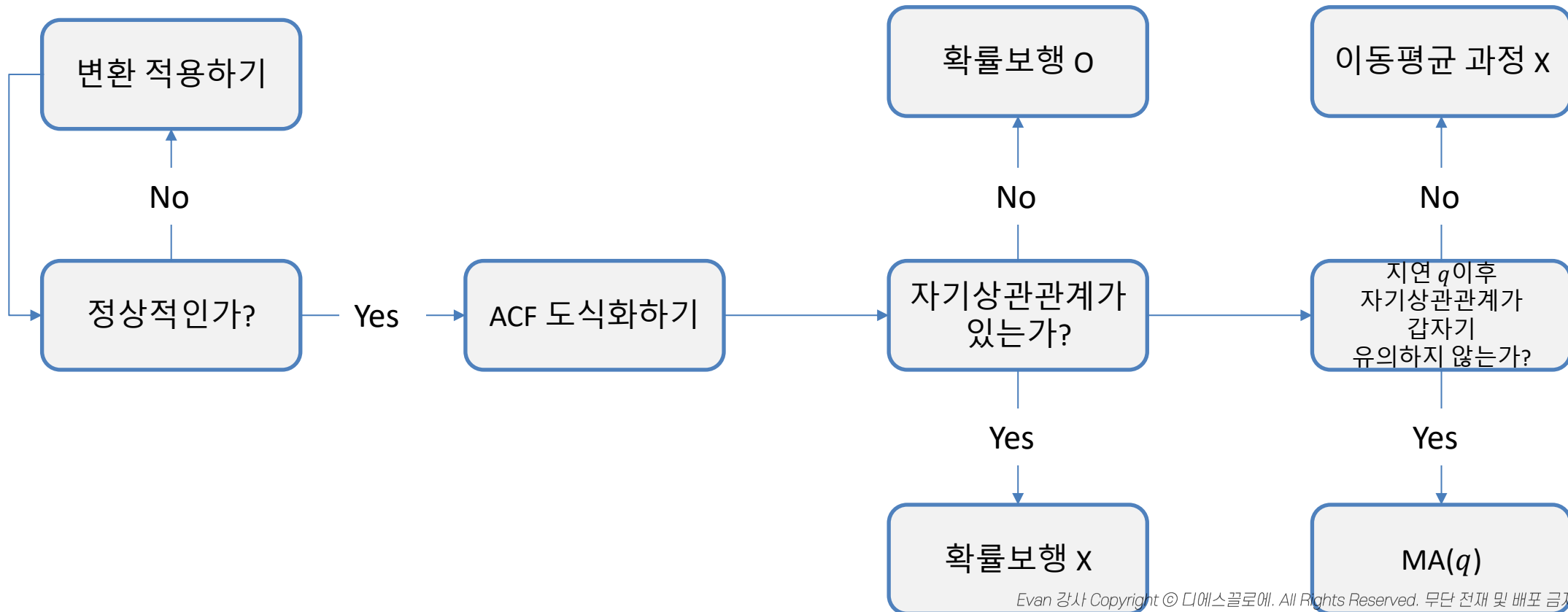
◆ 기존 모형과 비교

- ✓ 기존 모델보다 예측 성능이 향상된 것 확인



2. 자기회귀과정 모델링

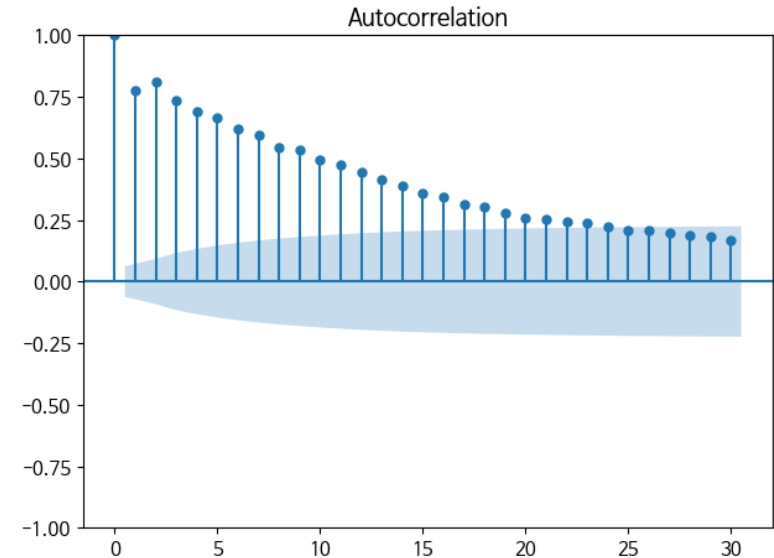
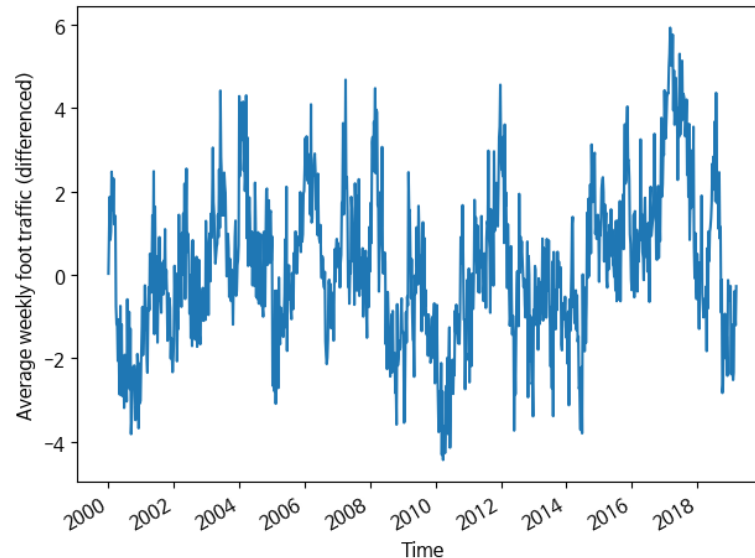
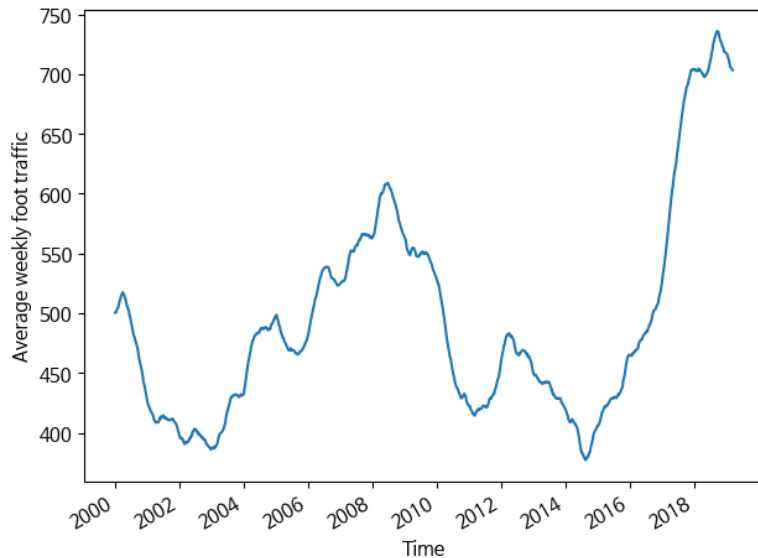
◆ 이동평균과정 차수 식별



2. 자기회귀과정 모델링

◆ 소매점 주간 평균 데이터, 차분, ACF 시각화

- ✓ 정상적인 시계열이 아님 $\Rightarrow$ 차분 $\Rightarrow$ ACF 시각화
- ✓ 서서히 감소하는 추세 $\Rightarrow$ 자기회귀과정임을 나타냄



2. 자기회귀과정 모델링

◆ 자기회귀과정 정의

- ✓ 예측값이 이전 값에만 선형적으로 의존한다고 가정, 변수가 자기 자신에게 회귀
- ✓ 자기회귀과정은 $AR(p)$ 모델 표현, p 는 차수 (현재값에 영향을 미치는 과거값들의 개수 결정)
- ✓ 수식

$$y_t = C + \phi_1 y_{t-1} + \phi_2 y_{t-2} + \cdots + \phi_p y_{t-p} + \varepsilon_t$$

▪ 용어 정리

- y_t : 현재값
- ε_t : 오차항
- C : 상수
- y_{t-p} : 과거값들의 선형 방정식
- ϕ_p : 과거값이 현재값에 미치는 영향의 크기

2. 자기회귀과정 모델링

◆ 자기회귀과정 정의

- ✓ AR(1)은 수식이 확률보행 모델과 유사

$$y_t = C + \phi_1 y_{t-1} + \varepsilon_t$$

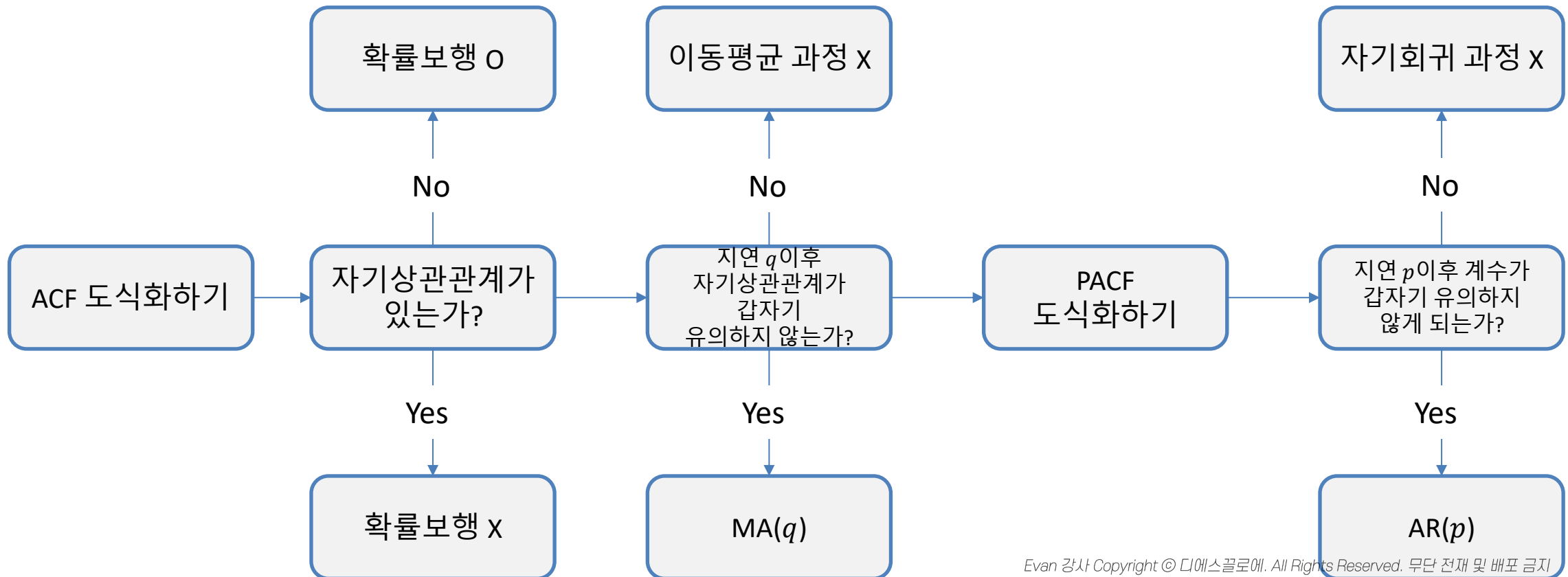
- ✓ AR(2)

$$y_t = C + \phi_1 y_{t-1} + \phi_2 y_{t-2} + \varepsilon_t$$

2. 자기회귀과정 모델링

◆ 정상적 자기회귀과정의 차수

- ✓ 이동평균과정의 차수 식별할 때 필요한 단계 확장



2. 자기회귀과정 모델링

◆ 편자기상관함수 PACF : Partial Autocorrelation Function

- ✓ 자기상관관계 : 시계열상 지연된 값들 간의 선형관계 측정
- ✓ 지연이 증가함에 따라 두 값 사이의 상관관계가 어떻게 변하는지 측정
- ✓ AR(2) 시나리오

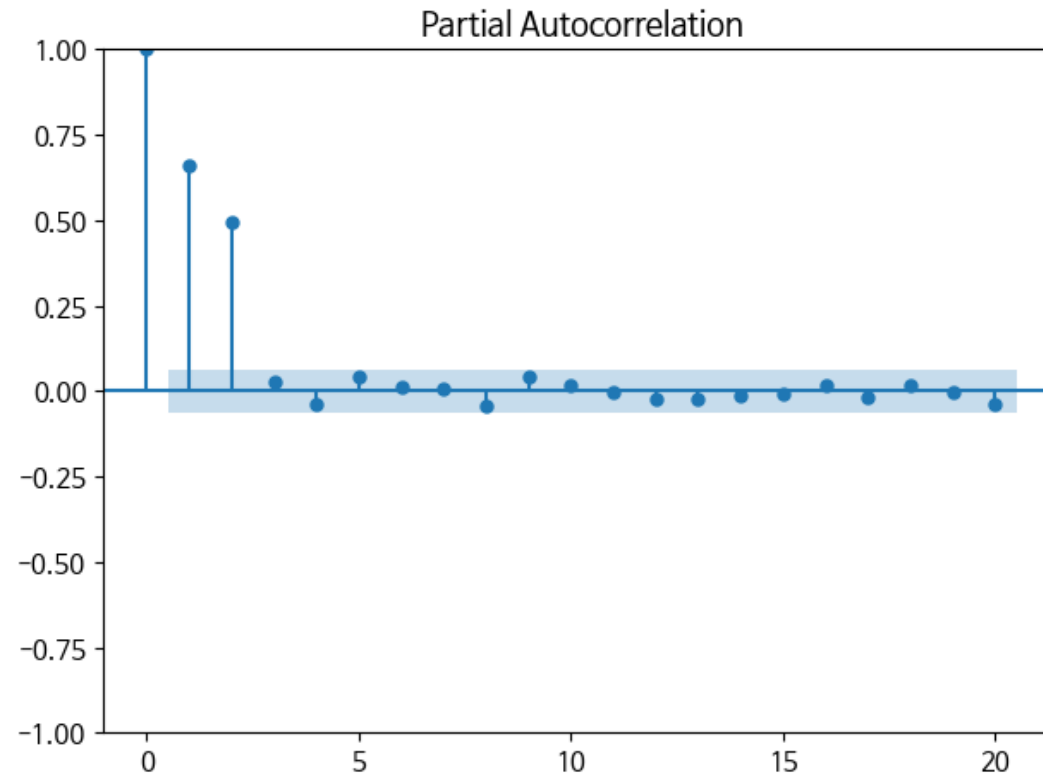
$$y_t = 0.33y_{t-1} + 0.50y_{t-2}$$

- y_t 와 y_{t-2} 가 어떻게 관련되는지, 두 함수의 상관관계 측정
- y_{t-1} 도 y_t 에 영향을 미침
- y_{t-1} 영향을 제거해야 함 $\Rightarrow$ 편자기상관관계

2. 자기회귀과정 모델링

◆ 편자기상관함수 PACF : Partial Autocorrelation Function

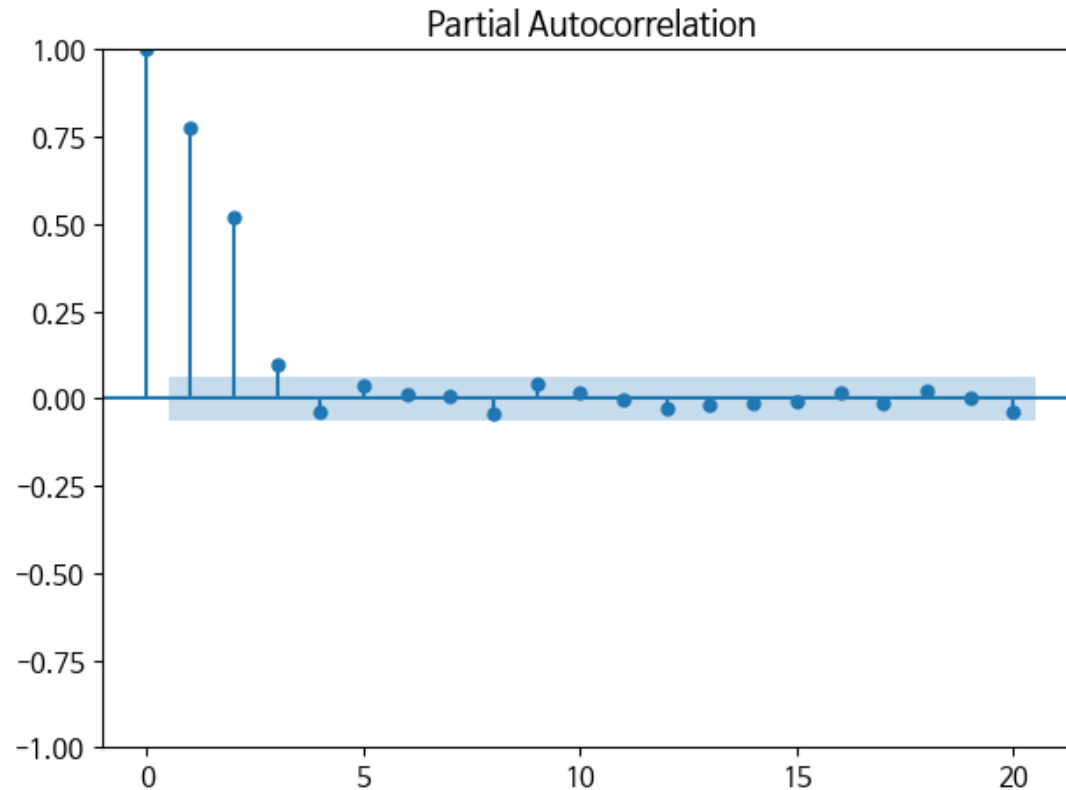
- ✓ AR(2) : 지연 2 이후에 계수들이 유의하지 않음



2. 자기회귀과정 모델링

◆ 편자기상관함수 PACF : Partial Autocorrelation Function

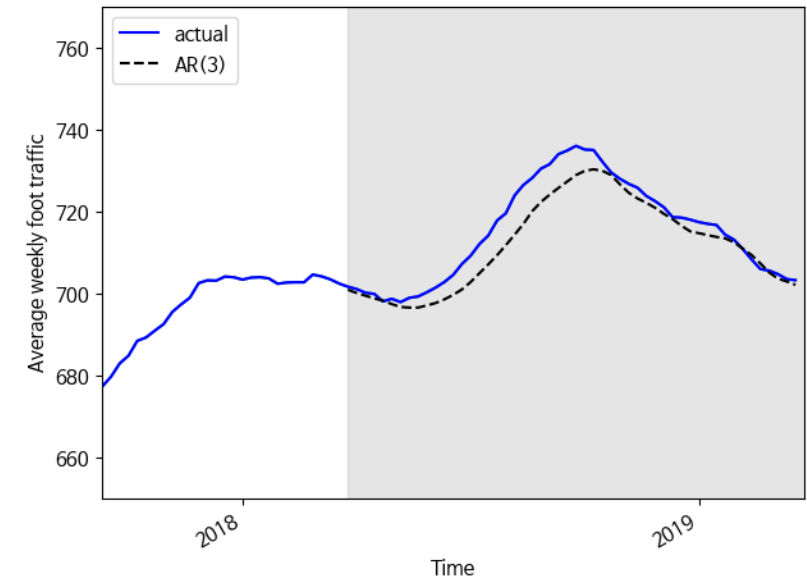
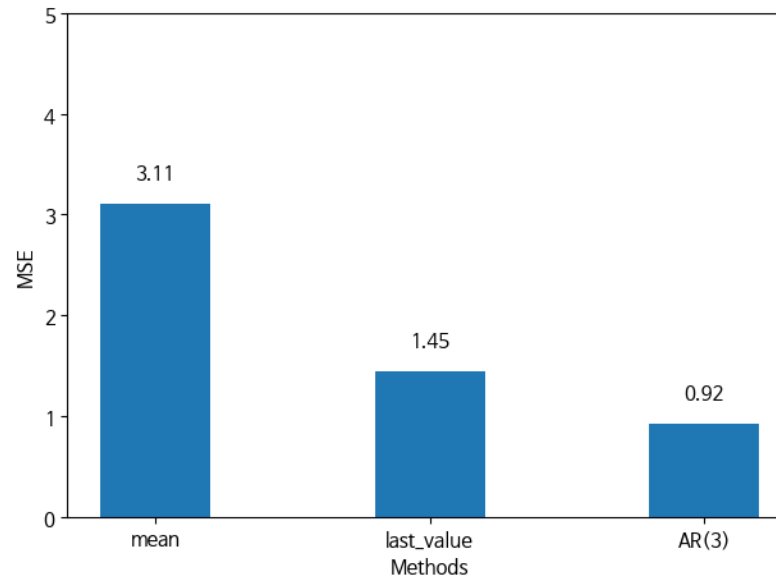
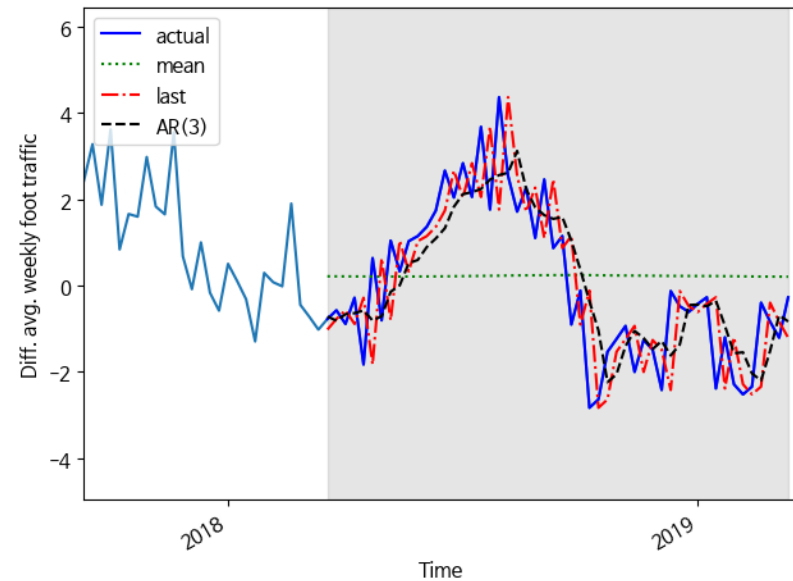
- ✓ AR(3) : 지연 3 이후에 계수들이 유의하지 않음



2. 자기회귀과정 모델링

◆ 자기회귀과정 예측

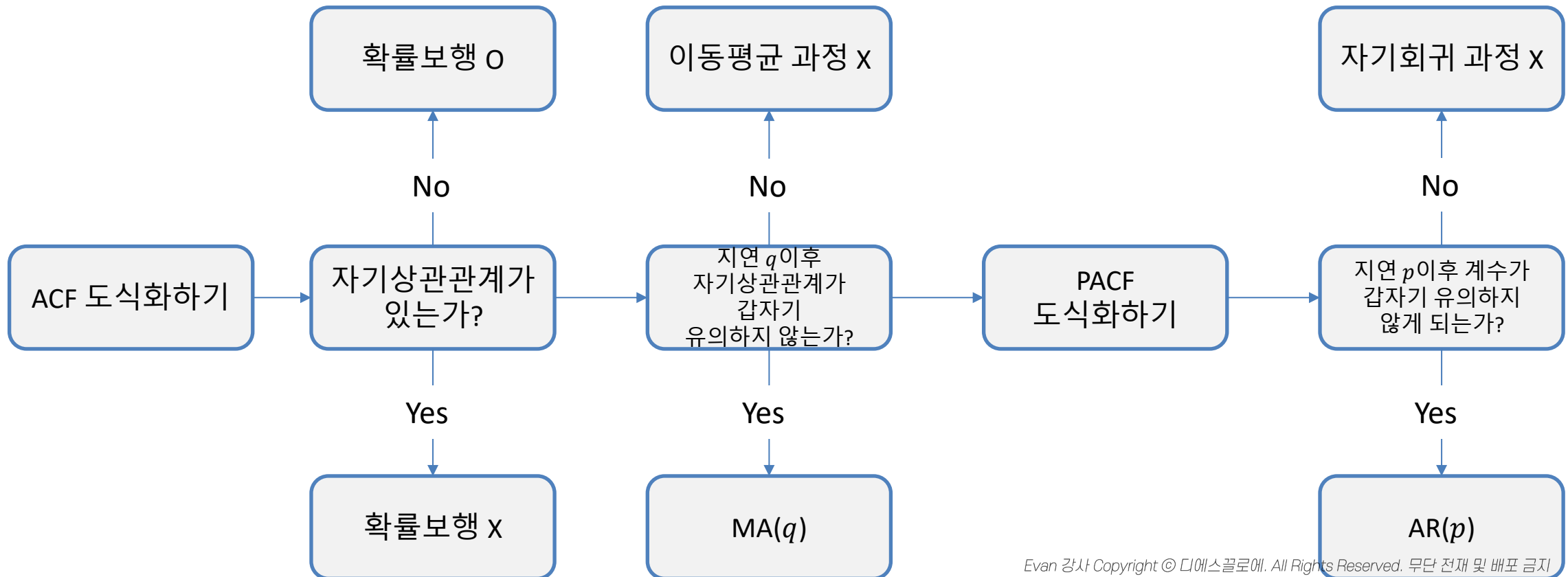
- ✓ 소매점의 주간 평균 유동인구를 차분한 값에 대한 예측 결과



3. 복잡한 시계열 모델링

◆ 자기회귀이동평균과정

- ✓ 확률보행, 이동평균과정, 자기회귀과정 식별



3. 복잡한 시계열 모델링

◆ 자기회귀이동평균과정

✓ 수식 정리 : $ARMA(p, q)$

- p : 자기회귀 부분의 차수, 현재값에 영향을 미치는 과거값의 개수 결정
- q : 이동평균 부분의 차수, 과거 오차 항의 개수 결정

$$y_t = C + \phi_1 y_{t-1} + \phi_2 y_{t-2} + \cdots + \phi_p y_{t-p} + \varepsilon_t + \theta_1 \varepsilon_{t-1} + \theta_2 \varepsilon_{t-2} + \cdots + \theta_q \varepsilon_{t-q}$$

3. 복잡한 시계열 모델링

◆ 자기회귀이동평균과정

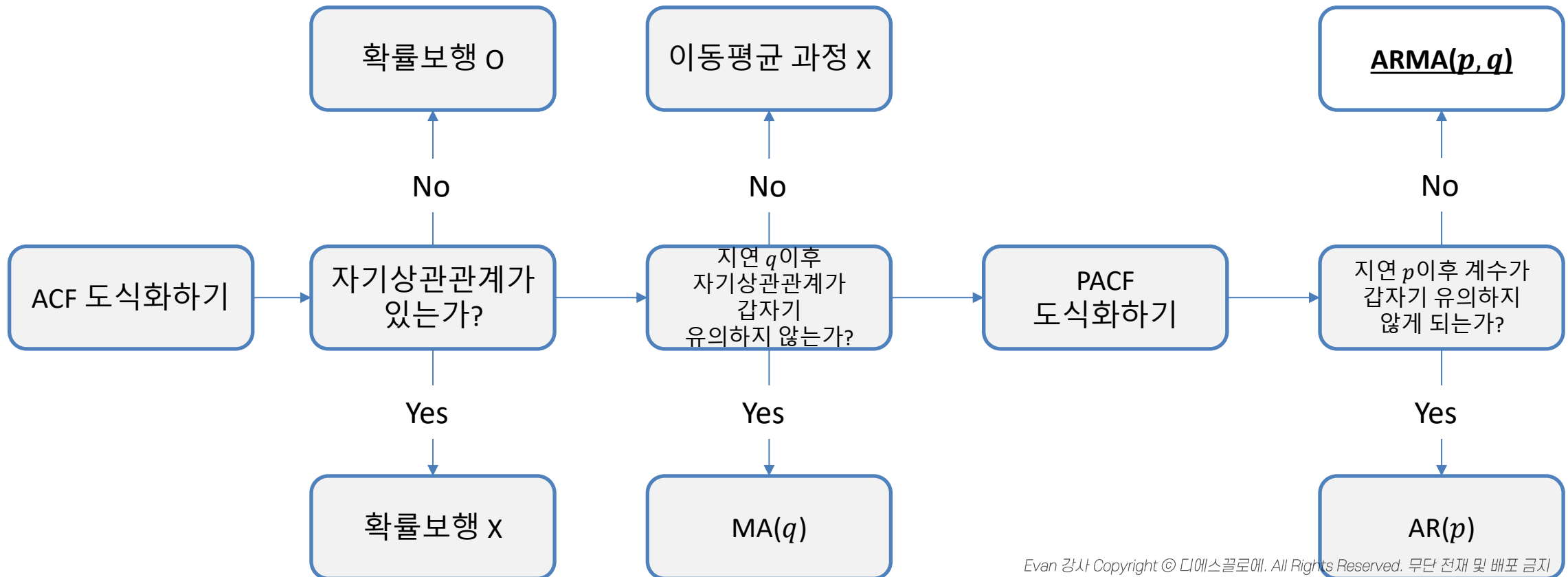
✓ 수식 정리 : $ARMA(p, q)$

- $ARMA(1, 1) := y_t = C + \phi_1 y_{t-1} + \varepsilon_t + \theta_1 \varepsilon_{t-1}$, AR(1), MA(1) 프로세스 결합
- $ARMA(2, 1) := y_t = C + \phi_1 y_{t-1} + \phi_2 y_{t-2} + \mu + \varepsilon_t + \theta_1 \varepsilon_{t-1}$, AR(2), MA(1) 프로세스 결합

3. 복잡한 시계열 모델링

◆ 자기회귀이동평균과정

✓ ARMA 프로세스 식별

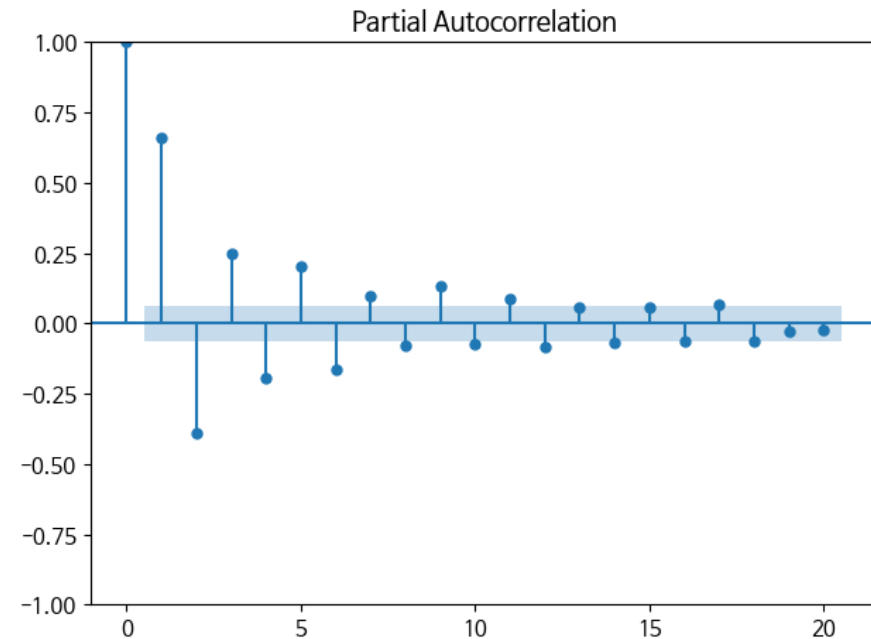
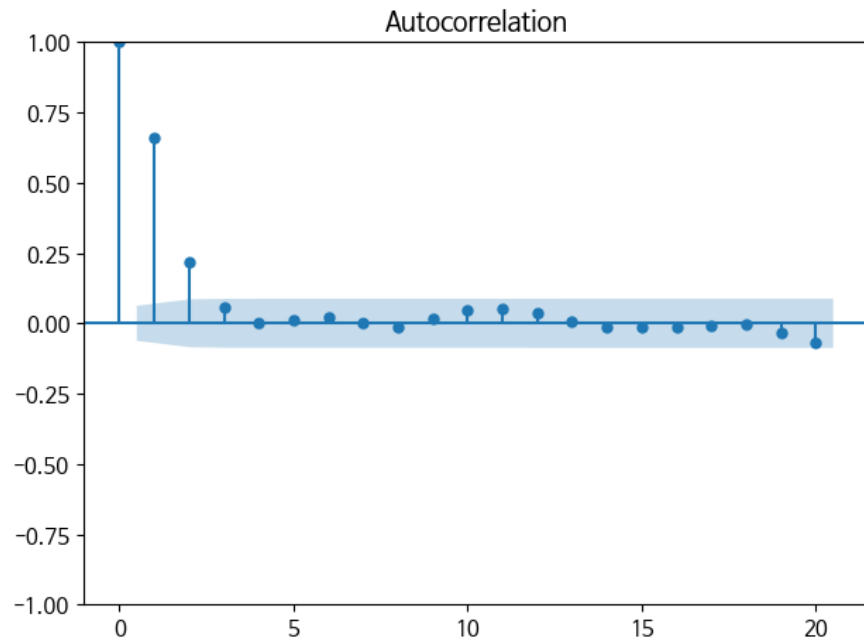


3. 복잡한 시계열 모델링

◆ 자기회귀이동평균과정

- ✓ 정상적 ARMA 프로세스 식별, ARMA(1, 1)

$$y_t = 0.33y_{t-1} + 0.9\varepsilon_{t-1} + \varepsilon_t$$



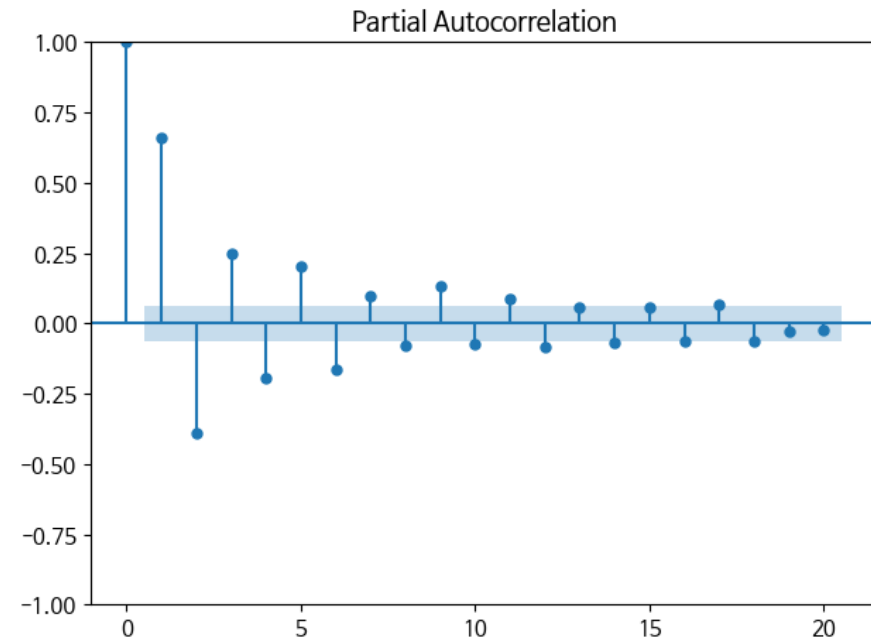
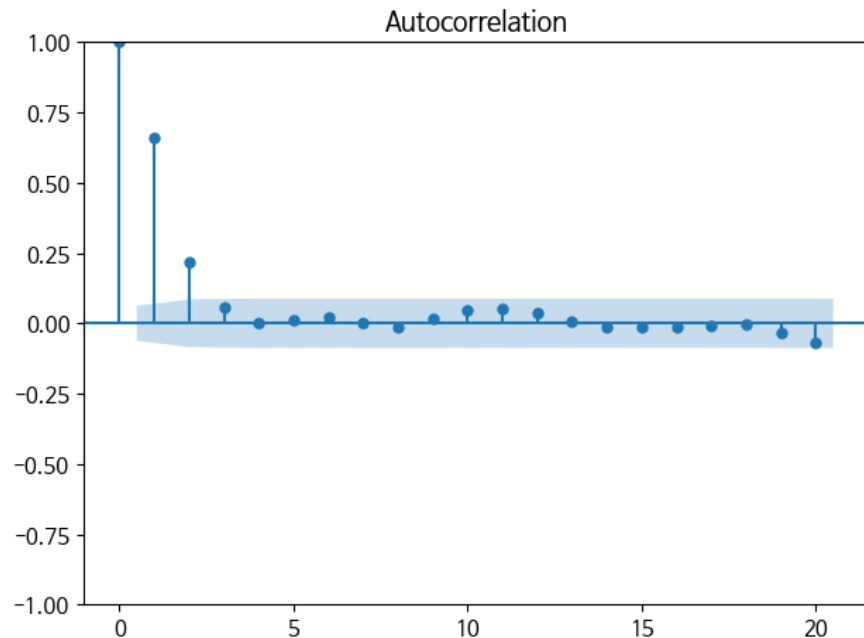
3. 복잡한 시계열 모델링

◆ 자기회귀이동평균과정

- ✓ ACF그래프로는 MA 차수 판단 맞지 않음
- ✓ PACF그래프로는 AR 차수 판단 불가



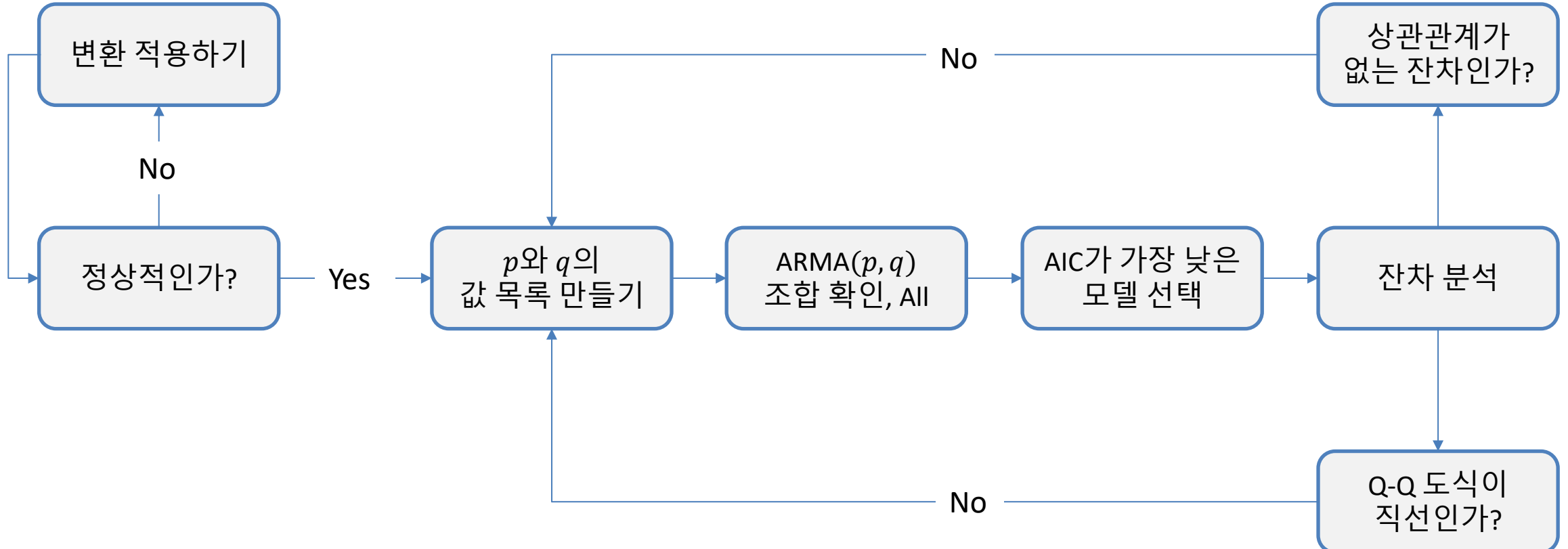
모델링 절차 고안



3. 복잡한 시계열 모델링

◆ 자기회귀이동평균과정

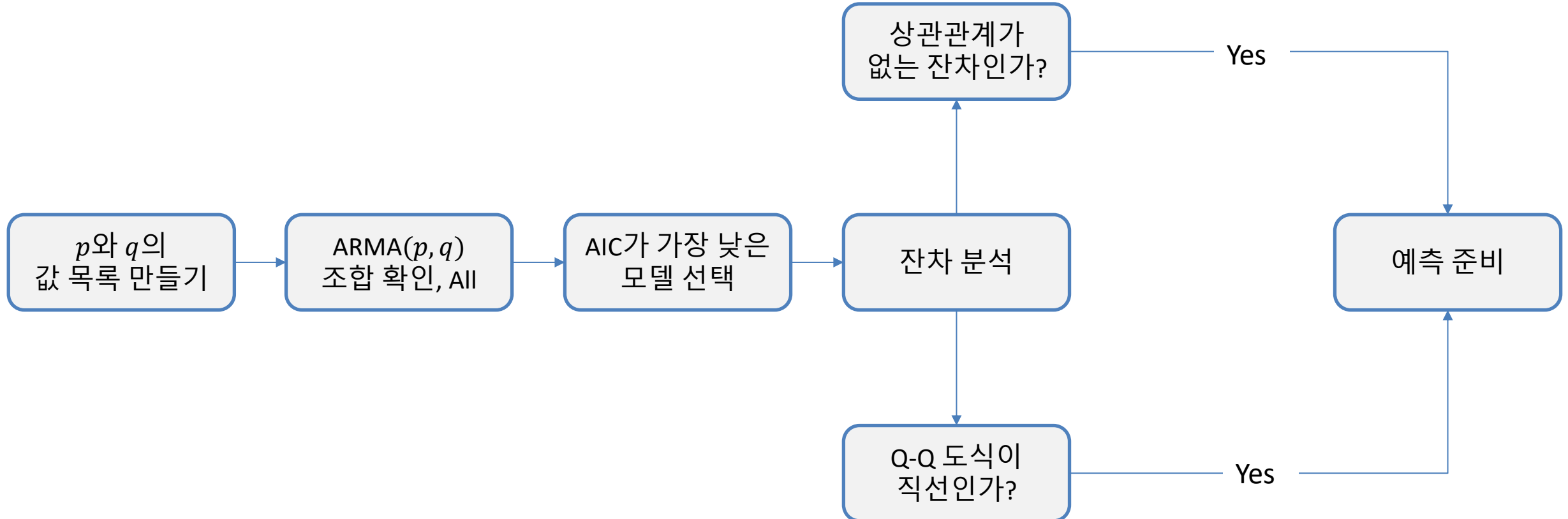
✓ 일반적 모델링 절차 고안



3. 복잡한 시계열 모델링

◆ 자기회귀이동평균과정

- ✓ 일반적 모델링 절차 고안



3. 복잡한 시계열 모델링

◆ 자기회귀이동평균과정

✓ 아카이케 정보 기준

$$AIC = 2k, 2\ln(\hat{L})$$

▪ 용어 정리

- k : 모델의 매개변수 개수
- $\hat{L}$: 가능도 함수의 최댓값

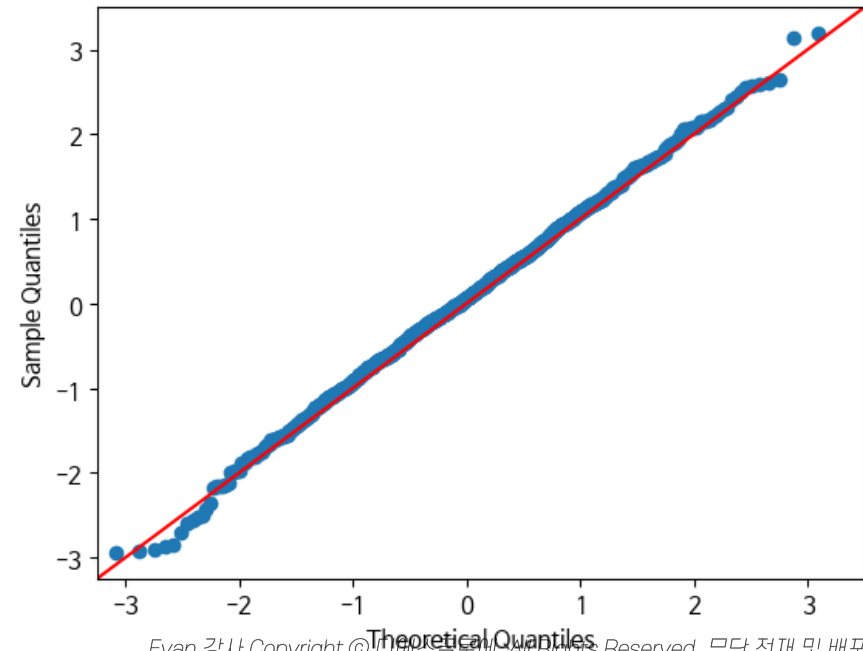
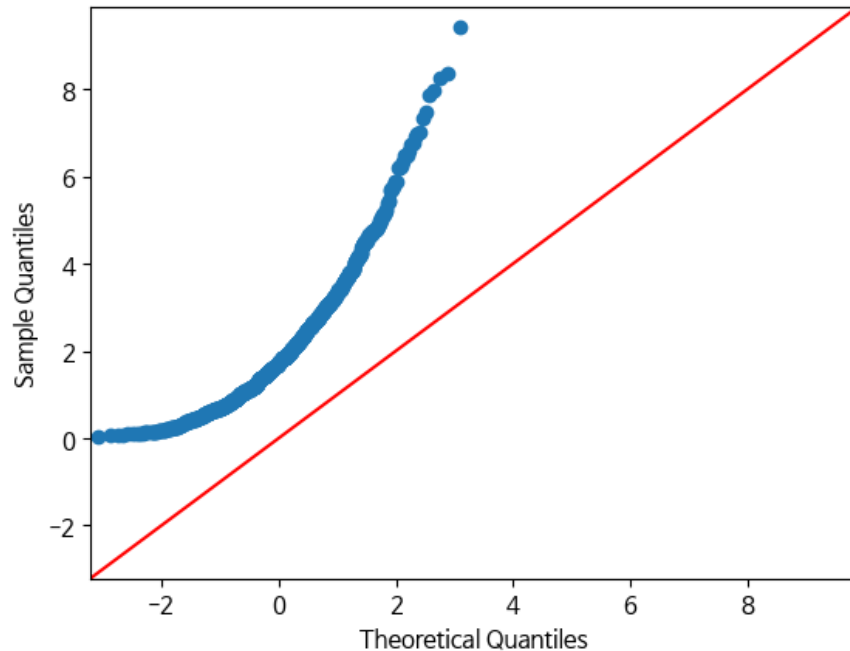
✓ AIC 값이 낮을수록 더 좋은 모델

3. 복잡한 시계열 모델링

◆ 자기회귀이동평균과정

✓ 잔차분석 : Q-Q 도식 확인

- 모델의 잔차가 정규분포인가? 가설 검증하기 위한 시각적 도구
- 정규분포인 경우, 잔차가 백색소음과 유사하여 모델의 적합도가 좋다는 뜻



4. 비정상적 시계열 예측

- ◆ 자기회귀누적이동평균(Autoregressive Integrated Moving Average, ARIMA)
 - ✓ 자기회귀과정 : $AR(p)$, 적분 $I(d)$, 이동평균과정 $MA(q)$ 의 조합
 - ✓ ARMA와의 차이점
 - ARMA : 원본 수열 사용
 - ARIMA : 차분된 수열 사용

4. 비정상적 시계열 예측

◆ 자기회귀누적이동평균(Autoregressive Integrated Moving Average, ARIMA)

- ✓ 자기회귀과정 : $AR(p)$, 적분 $I(d)$, 이동평균과정 $MA(q)$ 의 조합
- ✓ 수식 활용

$$y'_t = C + \phi_1 y'_{t-1} + \phi_2 y'_{t-2} + \cdots + \phi_p y'_{t-p} + \theta_1 \varepsilon'_{t-1} + \theta_2 \varepsilon'_{t-2} + \cdots + \theta_q \varepsilon'_{t-p} + \varepsilon_t$$

■ 용어 정리

- y'_t : 차분된 수열의 현재값
- C : 상수값
- $\phi_1 y'_{t-1}$: 차분된 수열의 과거값
- $\theta_q \varepsilon'_{t-p}$: 과거 오차항
- ε_t : 현재 오차항의 합

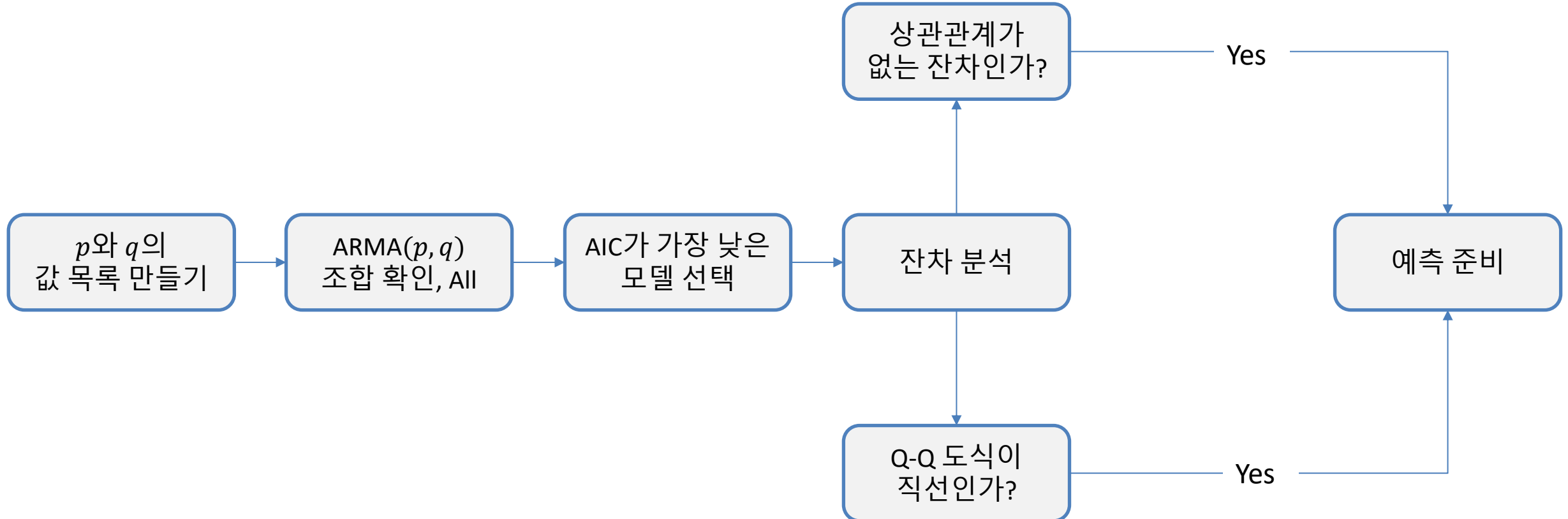
■ 차수 정리

- p : 지연된 값의 개수 결정, q : 지연된 오차 항의 개수

4. 비정상적 시계열 예측

◆ 자기회귀누적이동평균(Autoregressive Integrated Moving Average, ARIMA)

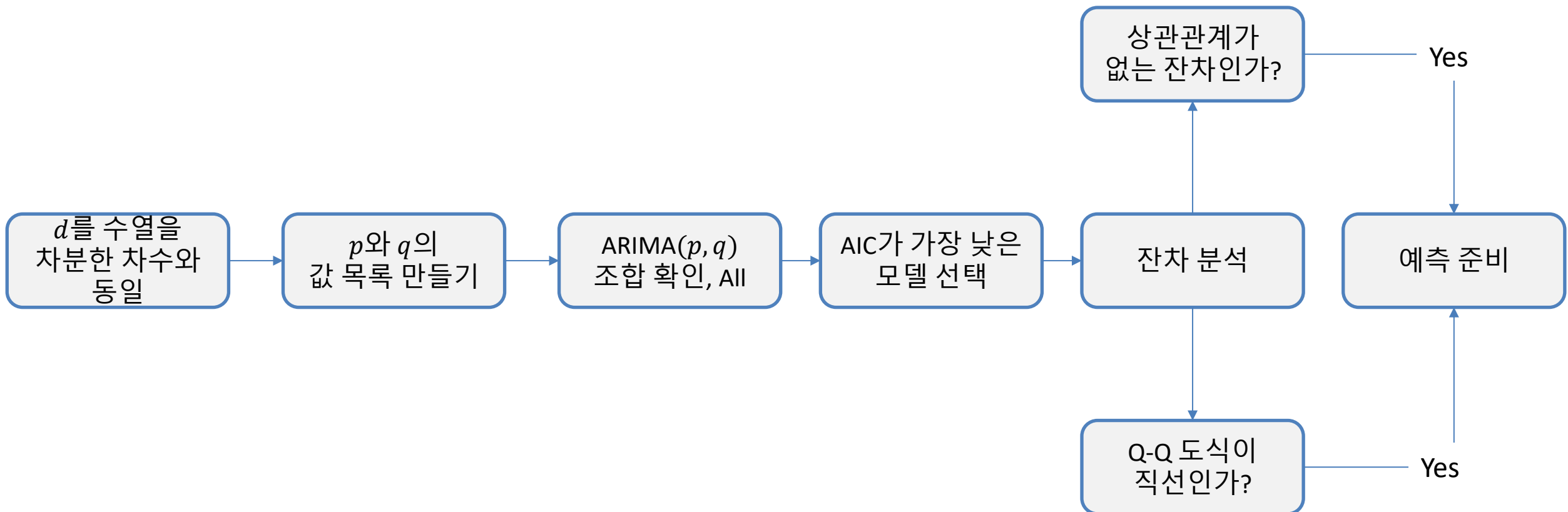
✓ ARMA 모델링 절차



4. 비정상적 시계열 예측

◆ 자기회귀누적이동평균(Autoregressive Integrated Moving Average, ARIMA)

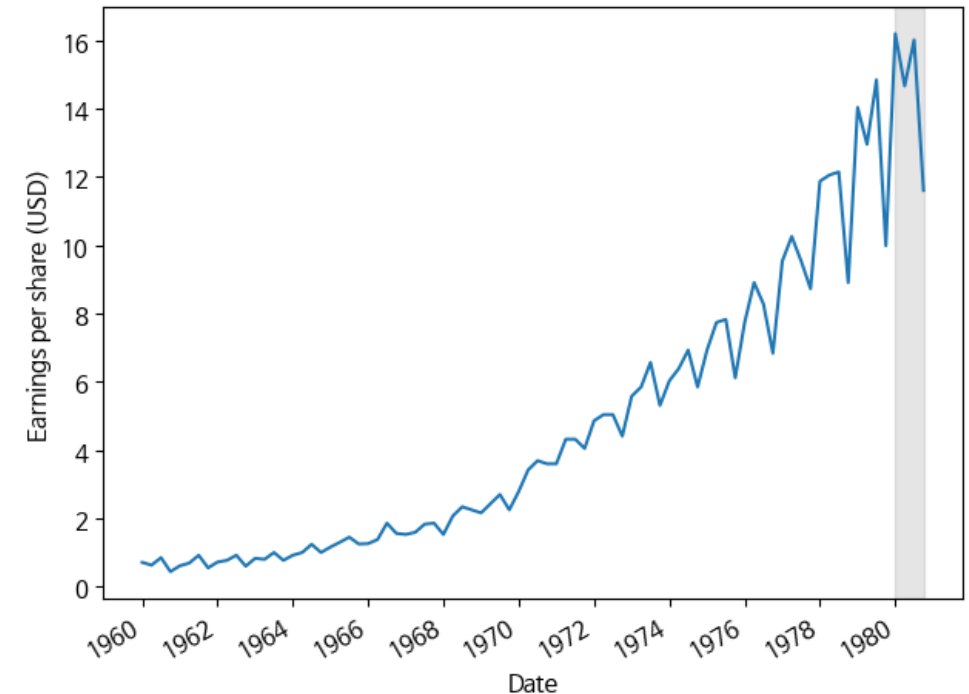
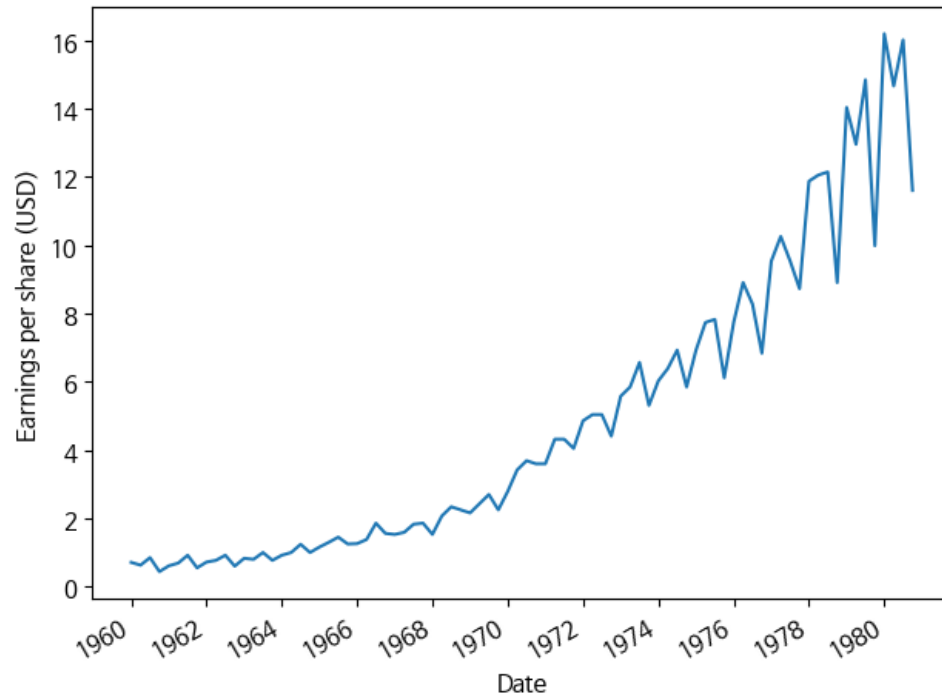
✓ ARMA 모델링 절차



4. 비정상적 시계열 예측

◆ 자기회귀누적이동평균(Autoregressive Integrated Moving Average, ARIMA)

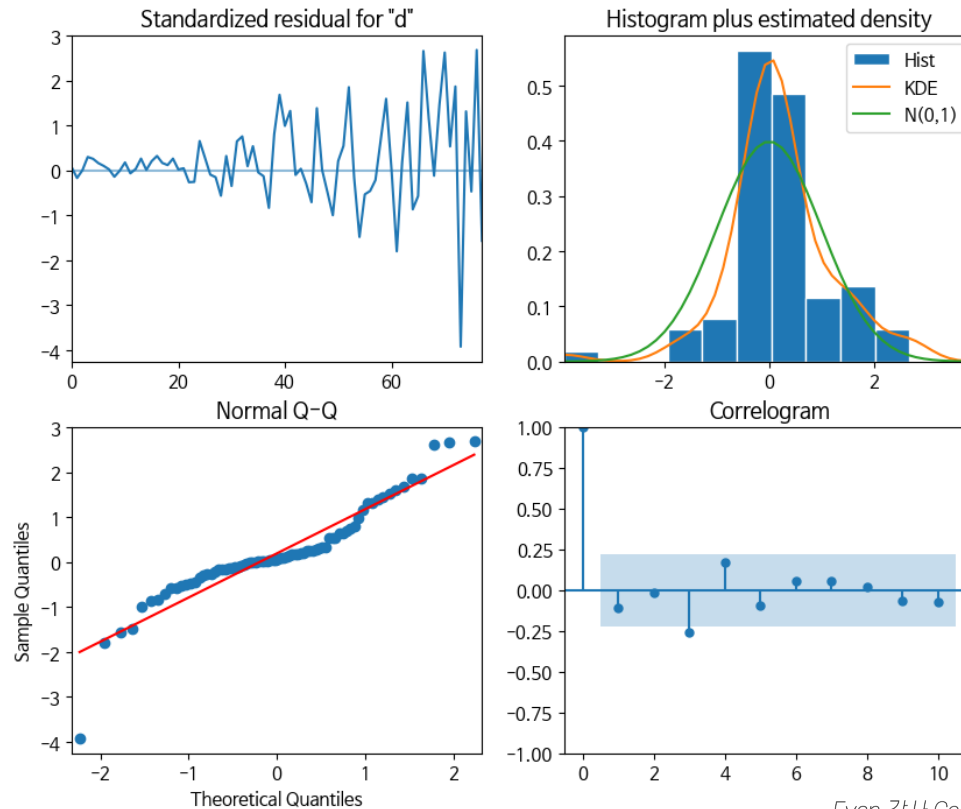
- ✓ 예측 데이터셋 : 존슨앤드존슨의 분기별 주당 순이익과 예측 구간 구하기



4. 비정상적 시계열 예측

◆ 자기회귀누적이동평균(Autoregressive Integrated Moving Average, ARIMA)

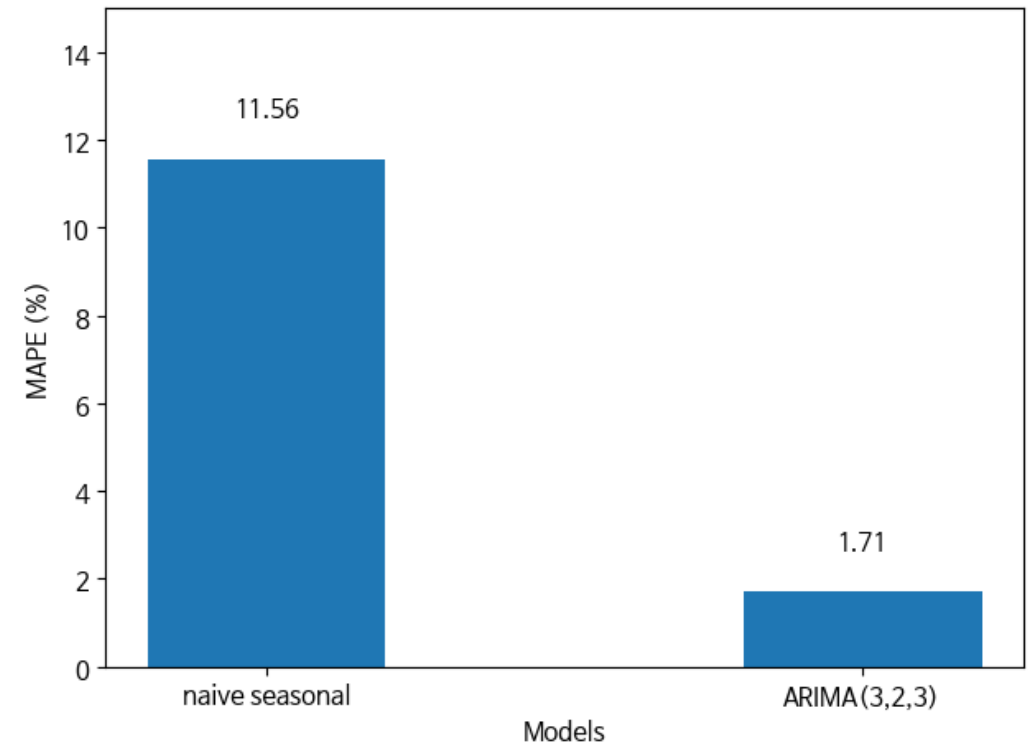
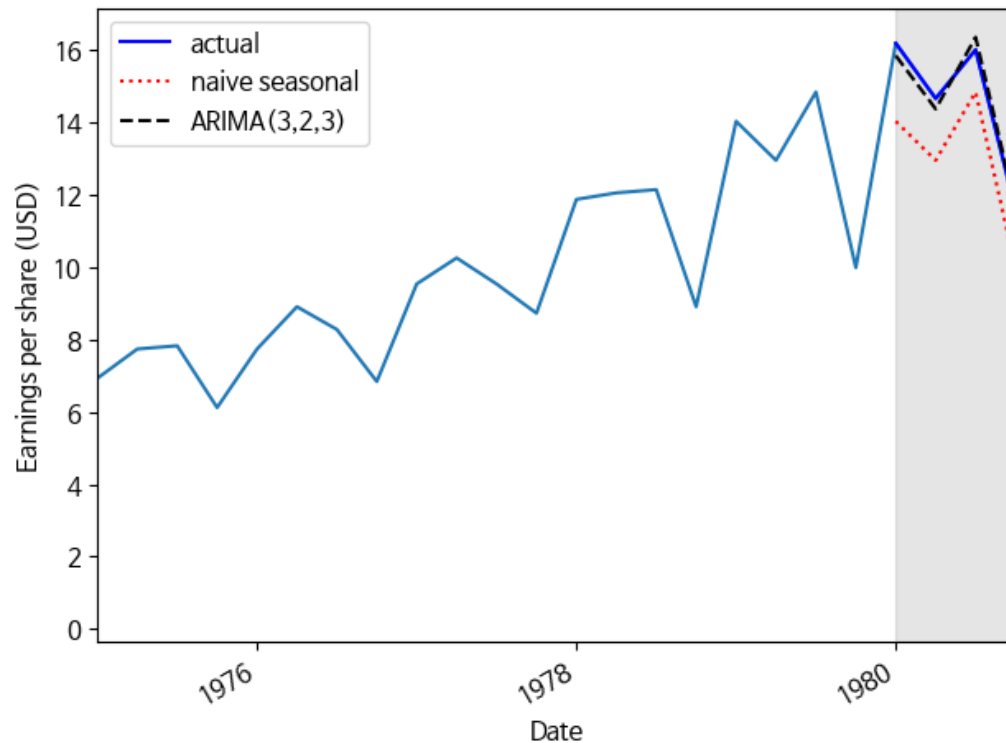
- ✓ ARIMA(3, 2, 3) 모델 피팅 후, 잔차 진단 표시



4. 비정상적 시계열 예측

◆ 자기회귀누적이동평균(Autoregressive Integrated Moving Average, ARIMA)

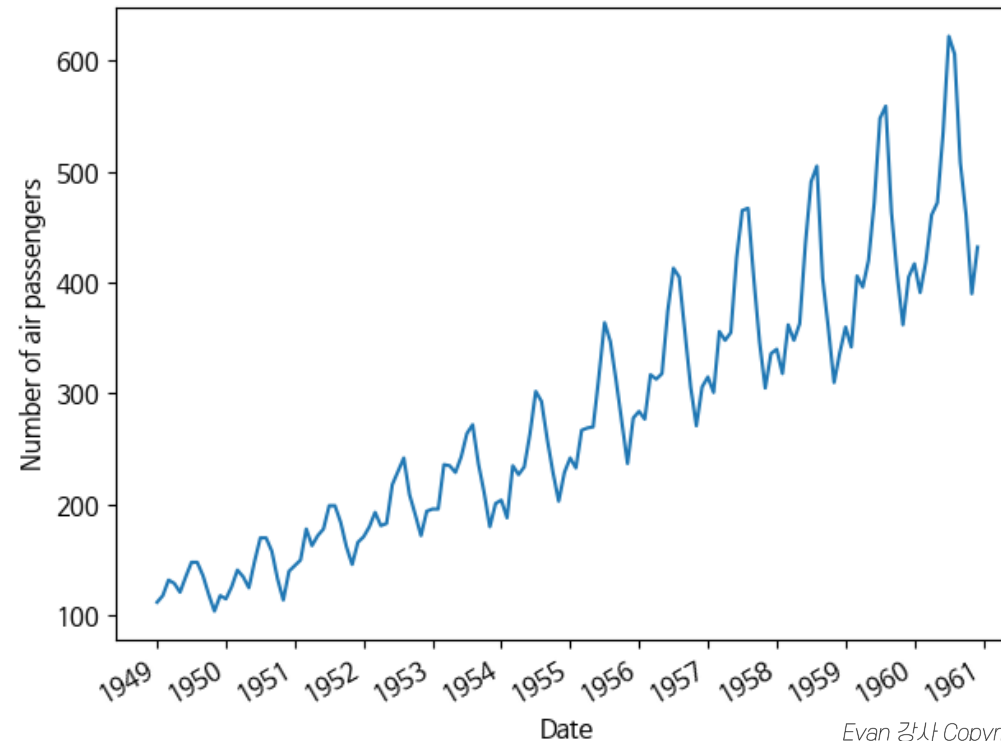
✓ Baseline 모델과 비교



5. 계절성을 고려한 시계열 예측

◆ 계절적 자기회귀누적이동평균(Seasonal ARIMA, SARIMA)

- ✓ 시계열에 계절적 패턴 포함
- ✓ 항공사의 월별 총 항공 승객 수



5. 계절성을 고려한 시계열 예측

◆ 계절적 자기회귀누적이동평균(Seasonal ARIMA, SARIMA)

✓ 모델 살펴보기

$$SARIMA(p, d, q)(P, D, Q)_m$$

▪ 용어 정리

- m : 빈도를 나타냄, 빈도 : 주기당 관측 횟수로 정의

✓ 데이터에 따른 적절한 빈도 m

데이터 수집	빈도 m
연간	1
분기별	4
월간	12
주간	52

5. 계절성을 고려한 시계열 예측

◆ 계절적 자기회귀누적이동평균(Seasonal ARIMA, SARIMA)

- ✓ 일 단위 및 하위 시간 단계 데이터에 대한 적절한 빈도 m 예시

데이터 수집	빈도 m				
연간	분	시간	일	주	년
매일				7	365
시간별			24	168	8766
매분		60	1440	10080	525960
매초	60	3600	86400	604800	31557600

5. 계절성을 고려한 시계열 예측

◆ 계절적 자기회귀누적이동평균(Seasonal ARIMA, SARIMA)

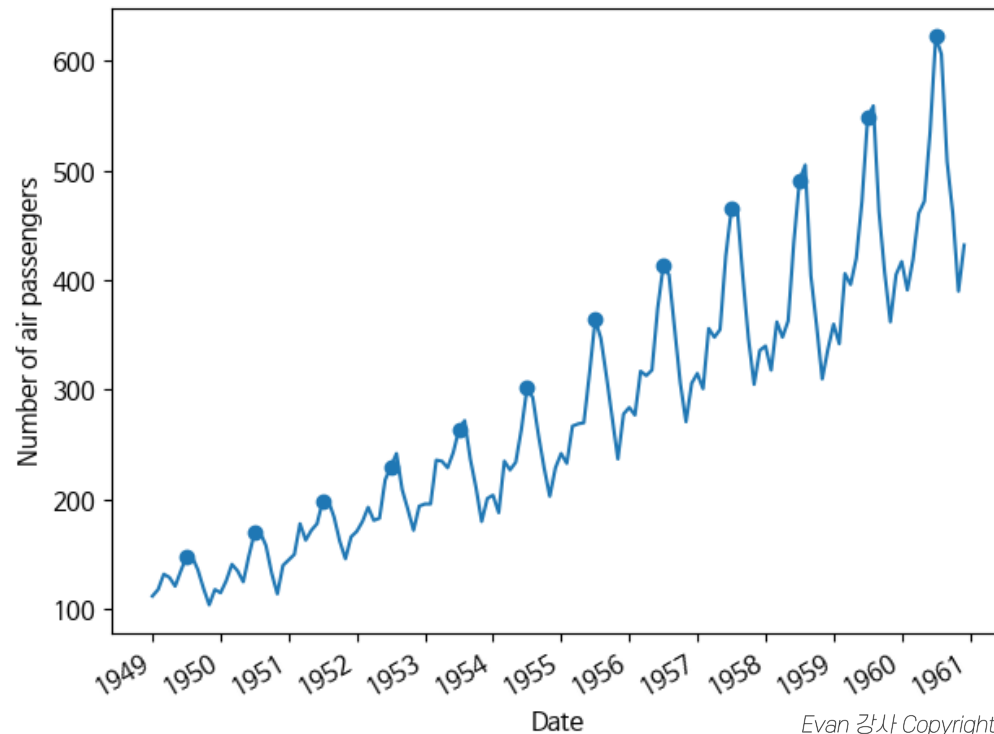
- ✓ P, D, Q 의 의미 : ARIMA 모델에서 계절적으로 상응하는 값
 - P : 계절적 $AR(p)$ 의 차수
 - D : 계절적 적분 차수
 - Q : 계절적 $MA(q)$ 의 차수
- ✓ 만약, P, D, Q 가 모두 0이면 ARIMA 모델과 동일함

5. 계절성을 고려한 시계열 예측

◆ 계절적 자기회귀누적이동평균(Seasonal ARIMA, SARIMA)

✓ 수식 응용

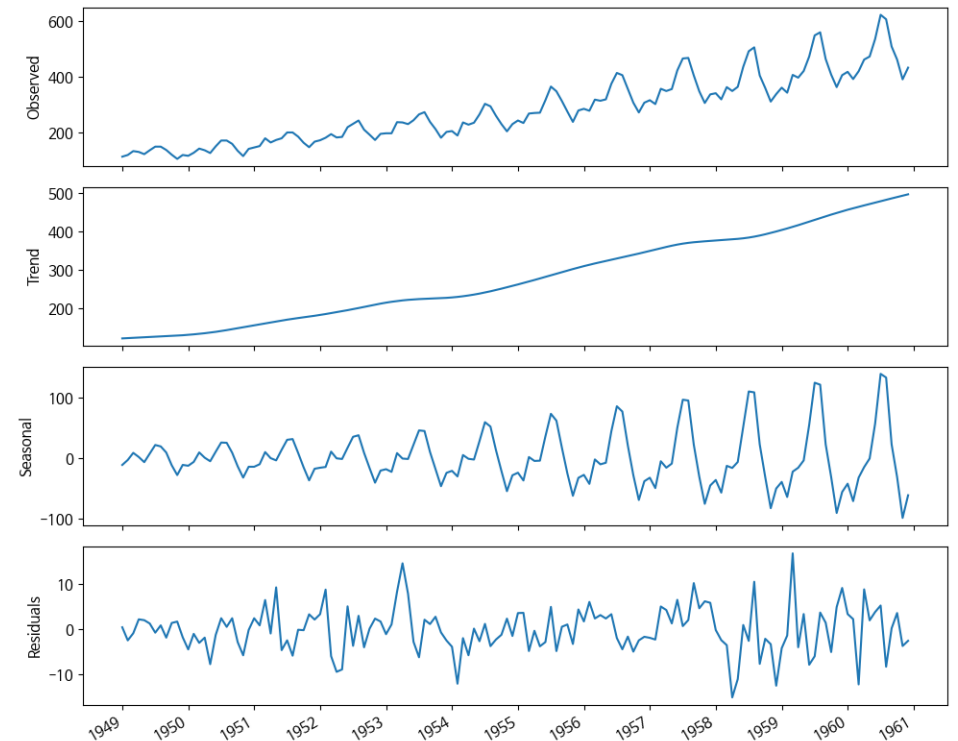
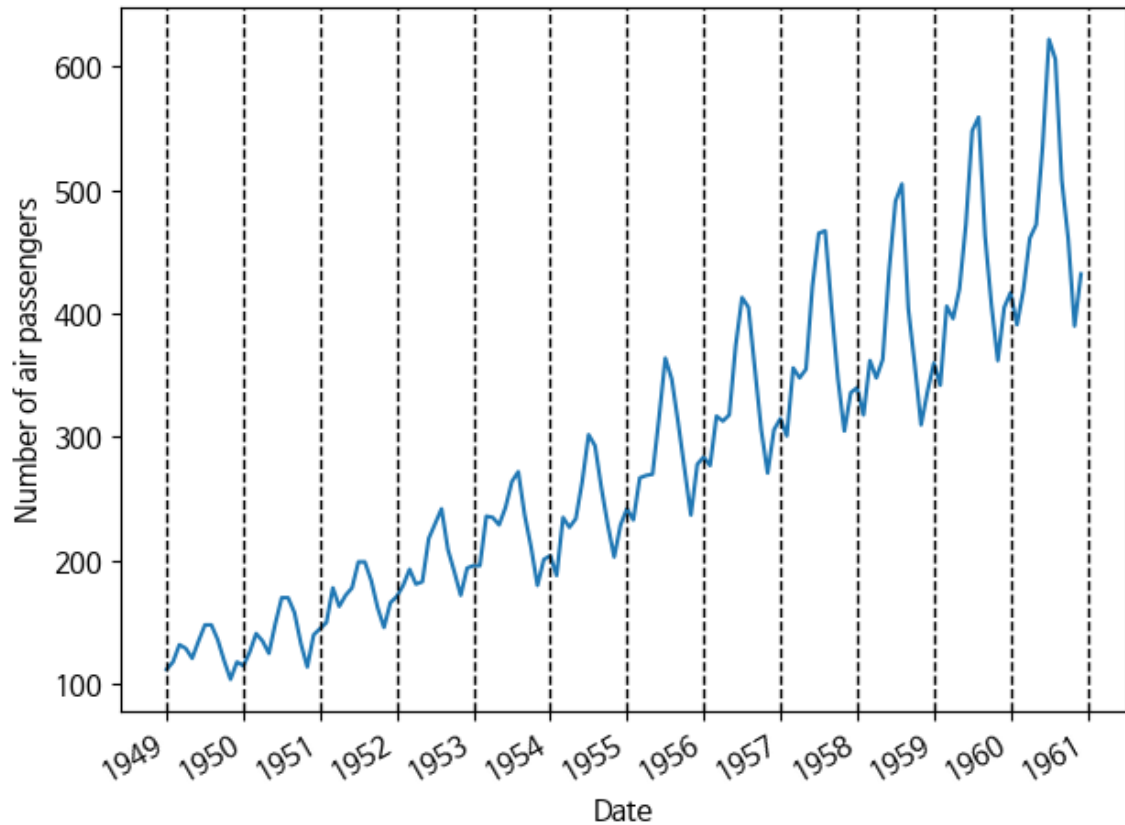
- m 이 12인 경우의 데이터, 매년 7-8월에 연중 항공 승객 수가 많음



5. 계절성을 고려한 시계열 예측

◆ 계절적 자기회귀누적이동평균(Seasonal ARIMA, SARIMA)

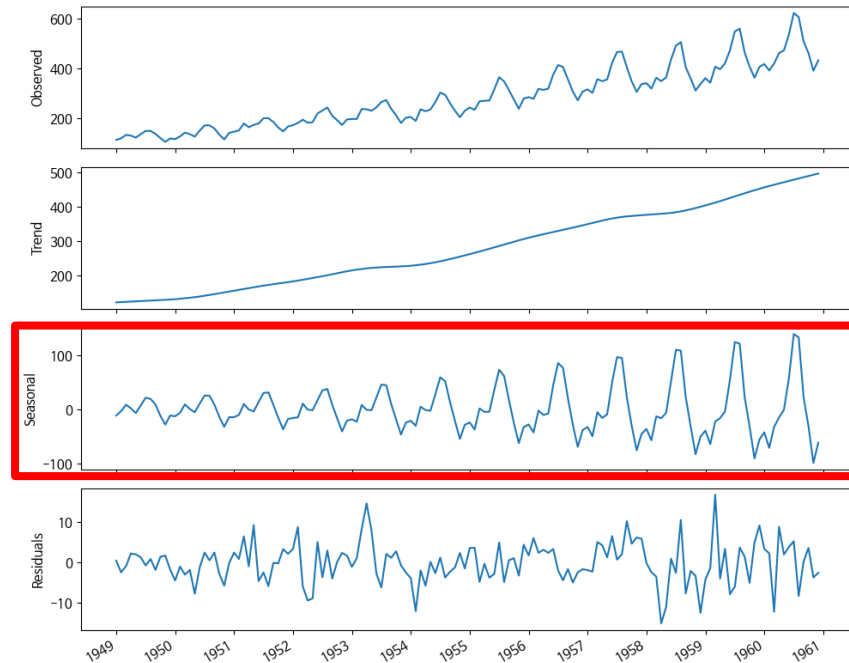
- ✓ 계절별 패턴 식별 : 도식화, 시계열 분해 (추세, 계절, 잔차)



5. 계절성을 고려한 시계열 예측

◆ 계절적 자기회귀누적이동평균(Seasonal ARIMA, SARIMA)

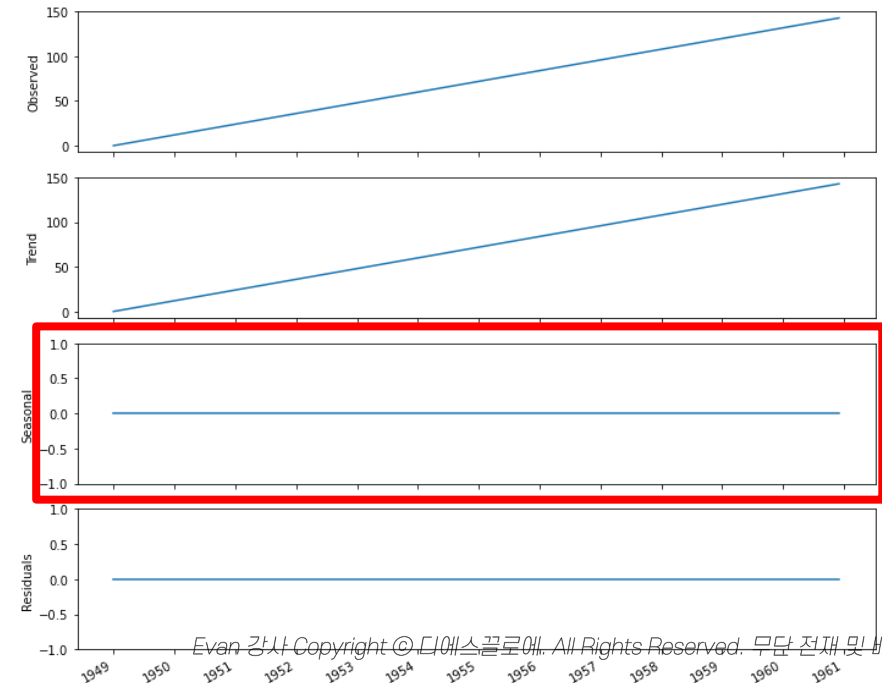
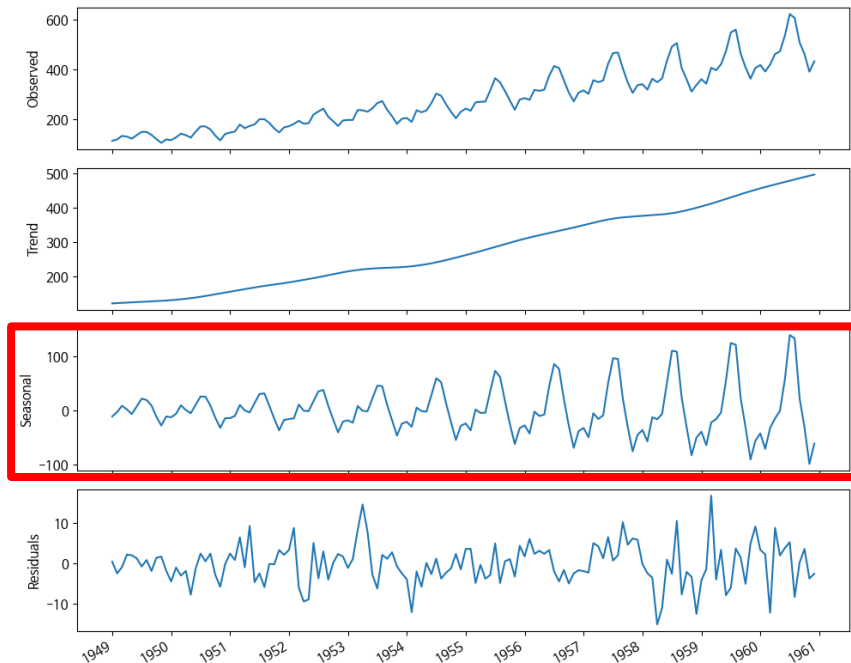
- ✓ 계절별 패턴 식별 : 도식화, 시계열 분해 (추세, 계절, 잔차)
 - 계절적 구성요소 : 관측 데이터에서 최고점과 최저점 생성



5. 계절성을 고려한 시계열 예측

◆ 계절적 자기회귀누적이동평균(Seasonal ARIMA, SARIMA)

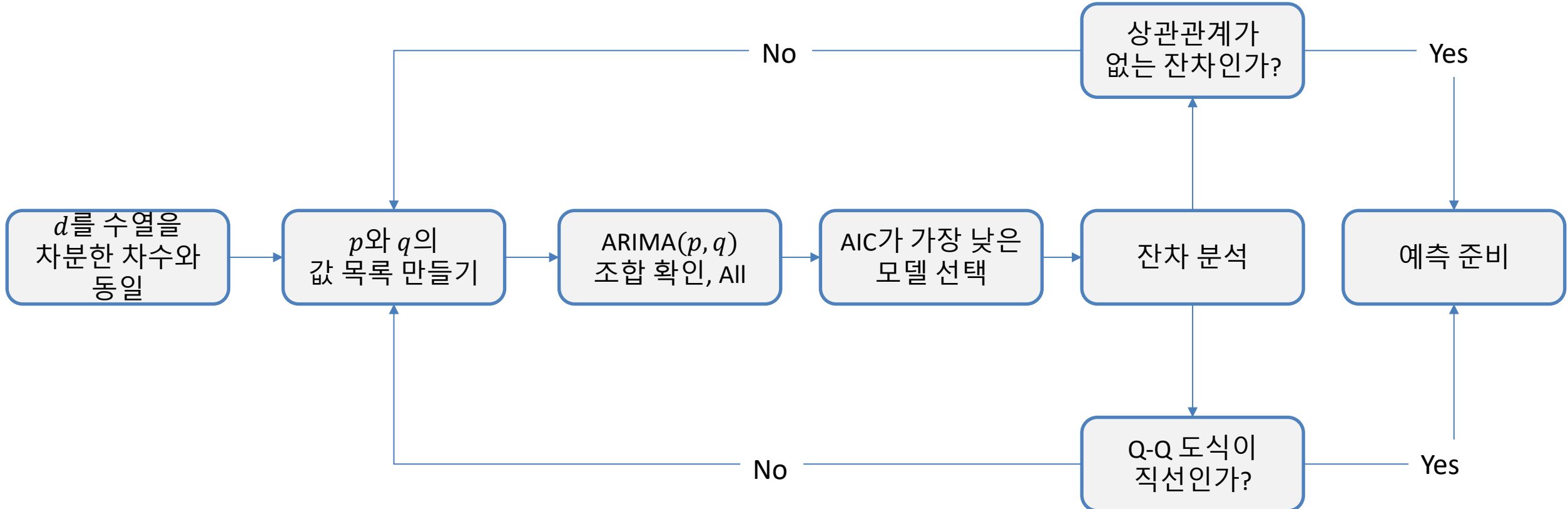
- ✓ 계절별 패턴 식별 : 도식화, 시계열 분해 (추세, 계절, 잔차)
 - 계절성이 없으면 구성요소가 0에서 평평한 수평선으로 표시
 - 시계열에서 계절성을 식별하는 통계적 테스트는 미 존재



5. 계절성을 고려한 시계열 예측

◆ 계절적 자기회귀누적이동평균(Seasonal ARIMA, SARIMA)

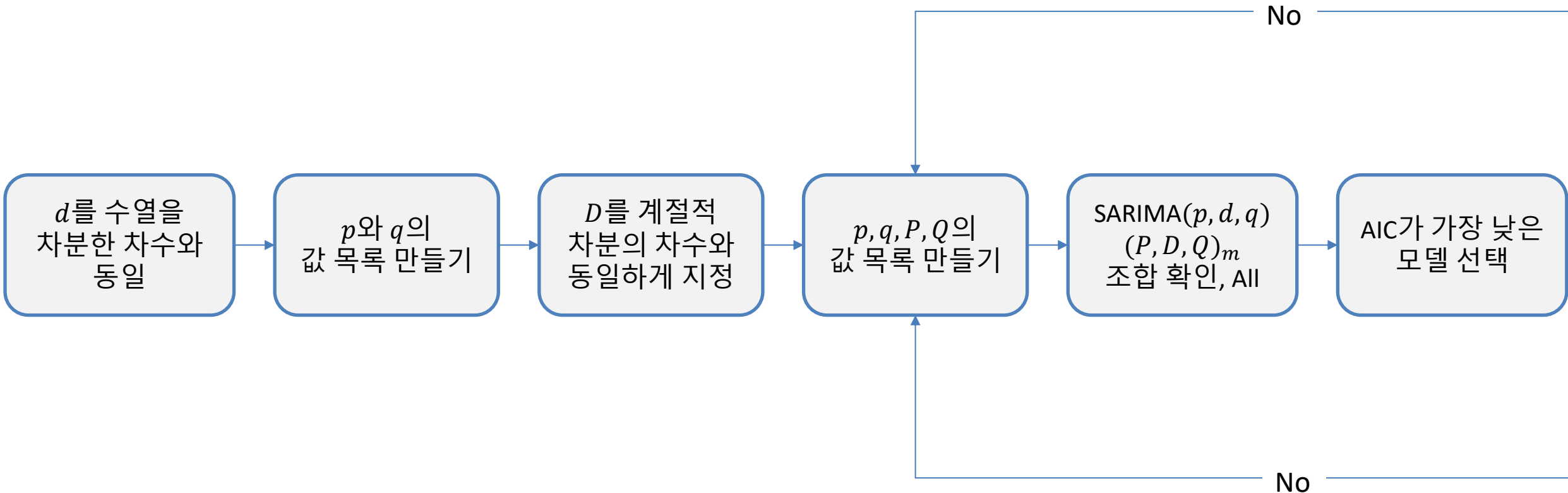
✓ ARIMA 시계열 예측 프로세스



5. 계절성을 고려한 시계열 예측

◆ 계절적 자기회귀누적이동평균(Seasonal ARIMA, SARIMA)

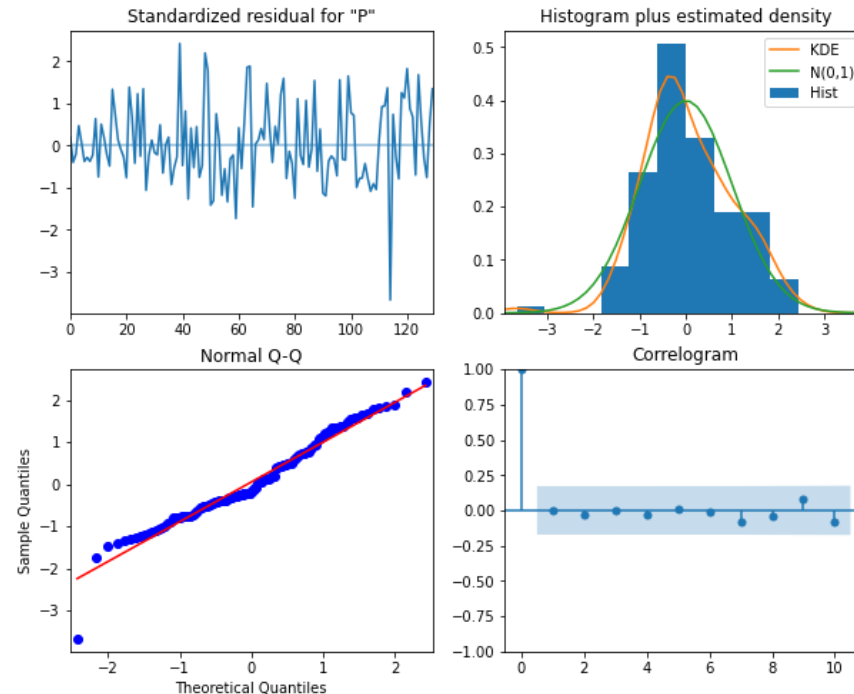
✓ SARIMA 시계열 예측 프로세스



5. 계절성을 고려한 시계열 예측

◆ 계절적 자기회귀누적이동평균(Seasonal ARIMA, SARIMA)

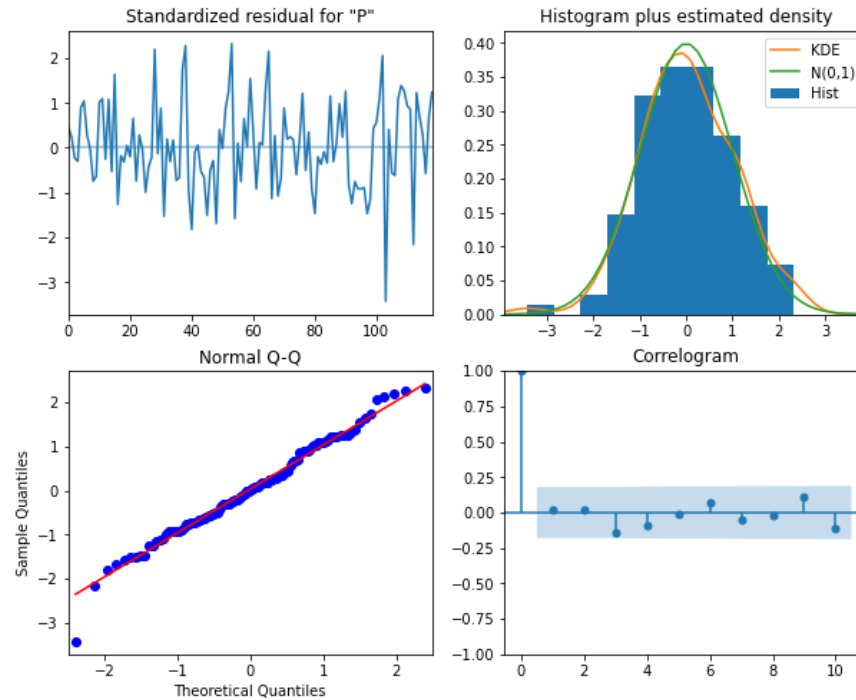
- ✓ ARIMA(11, 2, 3) 모델의 잔차 진단
 - 잔차는 백색소음에 가까우며, 오차가 무작위임



5. 계절성을 고려한 시계열 예측

◆ 계절적 자기회귀누적이동평균(Seasonal ARIMA, SARIMA)

- ✓ SARIMA(2, 1, 1)(1, 1, 2)₁₂ 모델의 잔치 진단
 - 잔차는 백색소음에 가까우며, 오차가 무작위임

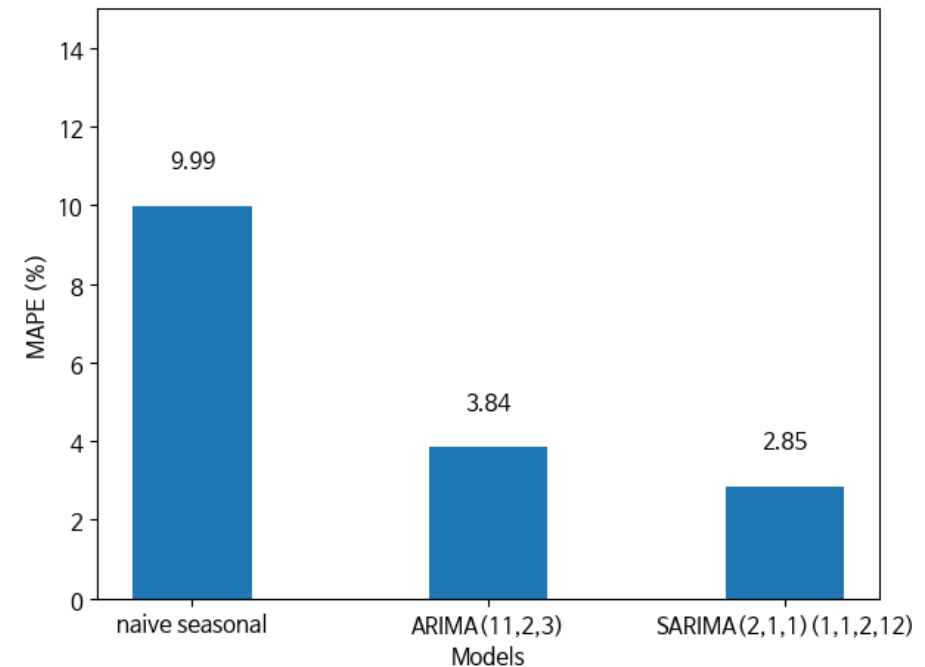
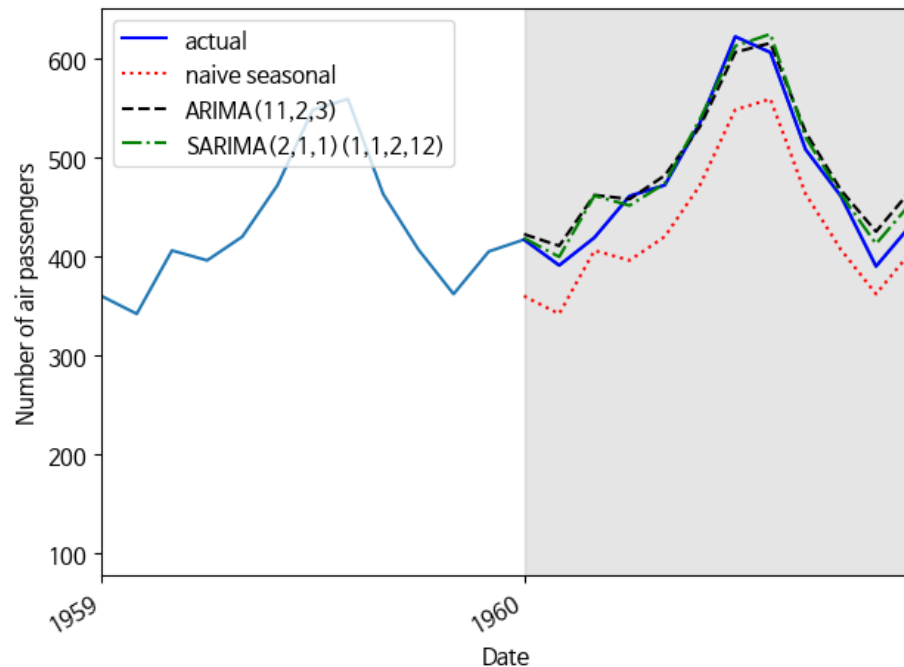


5. 계절성을 고려한 시계열 예측

◆ 계절적 자기회귀누적이동평균(Seasonal ARIMA, SARIMA)

✓ 예측 테스트 및 모델 비교

- 계절성을 고려한 모델의 성능이 기존 ARIMA 모델보다 더 좋음



6. 모델에 외생 변수 추가

◆ SARIMAX 모델

- ✓ 외생변수 Exogenous Variable : 예측 변수 또는 입력 변수 설명
- ✓ 내생변수 Endogenous Variable : 예측하고자 하는 대상 변수 정의
- ✓ 예시 : GDP

$$GDP = C + G + I + NX$$

- 용어 정리

- C : 소비
- G : 정부 지출
- I : 투자
- NX : 순수출

- ✓ 각 요소는 외생 변수의 영향을 받음
 - 예시 : 소비는 실업률의 영향

6. 모델에 외생 변수 추가

◆ SARIMAX 모델

✓ 수식

$$y_t = SARIMA(p, d, q)(P, D, Q)_m + \sum_{i=1}^n \beta_i X_t^i$$

- 기존 SARIMA 모델에 외생 변수를 추가
- 계절적 패턴이 없는 경우 ARIMAX 모델
- 외생변수가 없으면 SARIMA 모델
- 계절성이나 외생변수가 없으면 ARIMA 모델

6. 모델에 외생 변수 추가

◆ SARIMAX 모델

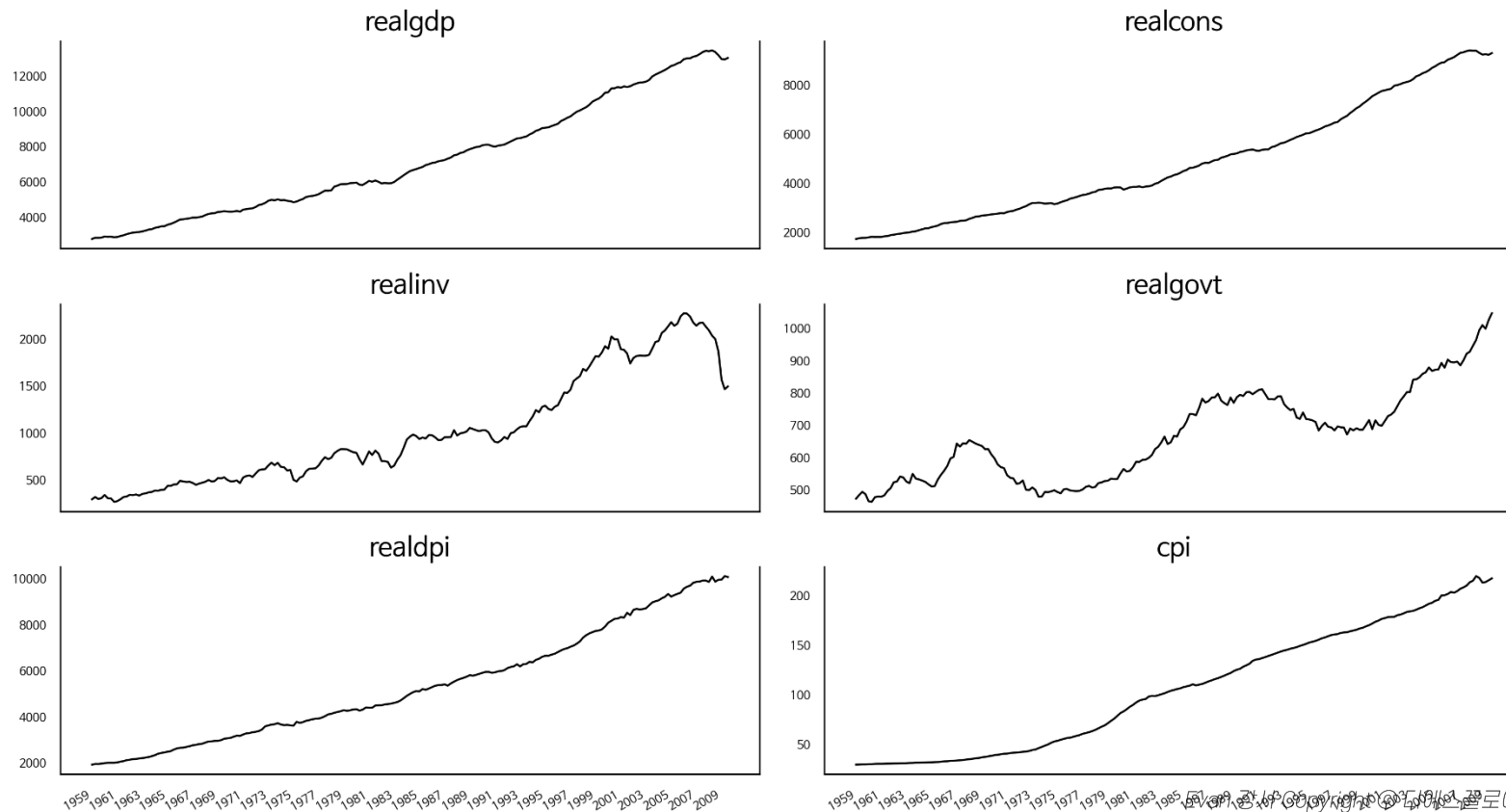
✓ 거시경제 데이터 집합

변수	설명	변수	설명
realgdp	실질 GDP(목표 변수 또는 내생 변수)	pop	분기 말 총인구
realcons	실질 개인 소비 지출	infl	인플레이션율
realinv	실질 국내 총 민간 투자	realint	실질 이자율
realgovt	실질 연방 소비 지출 및 투자		
realdpi	실질 민간 가처분 소득		
cpi	분기 말 소비자물가지수		
M1	M1 명목 통화량		
tbilrate	3개월 만기 국채의 분기별 월평균 금리		
unemp	실업률		

6. 모델에 외생 변수 추가

◆ SARIMAX 모델

✓ 거시경제 데이터 집합 시각화



6. 모델에 외생 변수 추가

◆ SARIMAX 모델

- ✓ p-value 값 보다 중요한 것, AIC
 - 통상적으로 p값을 통해 중요 변수 선택하는데 활용
 - 회귀분석에서 p값은 계수와 0과 유의하게 다른지를 테스트 함
 - AIC가 가장 낮은 모델을 선택하여 이 단계를 수행해야 함W
 - 참고 : [Statistical Tests for variable selection](#)

6. 모델에 외생 변수 추가

◆ SARIMAX 모델

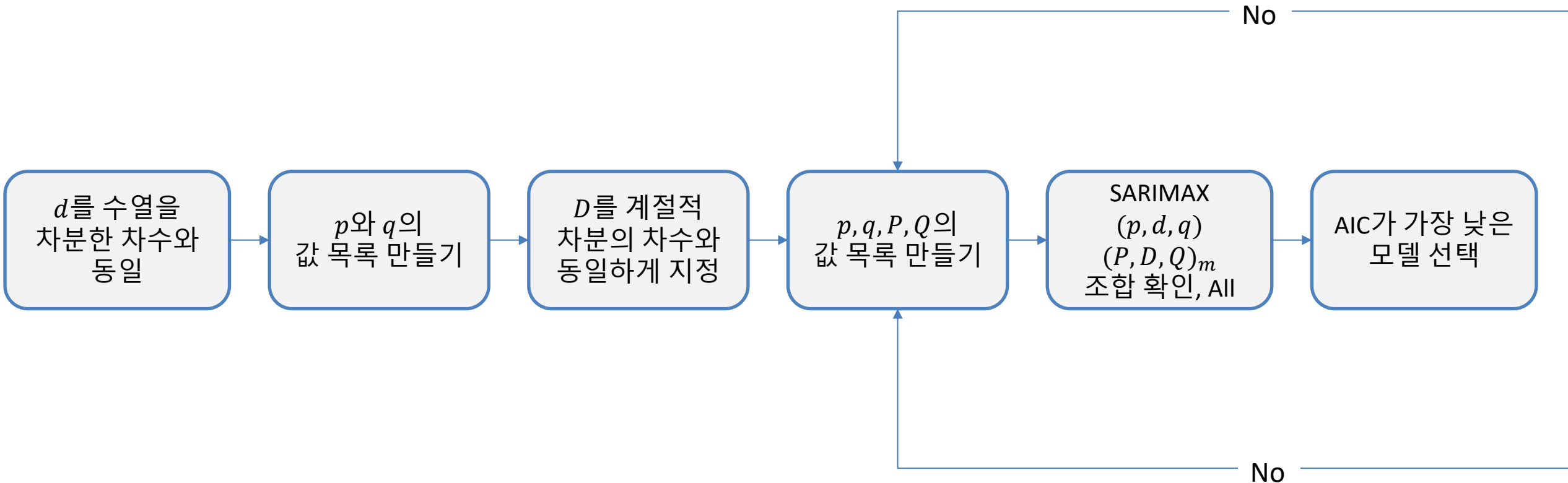
✓ 사용 시 유의사항

- $SARIMA(p, d, q)(P, D, Q)_m$ 모델과 외생 변수의 선형 조합 사용하여 미래의 한 시간 단계 예측 가능
- 미래의 두 시간 단계를 예측하기 위해서는 외생 변수도 예측
- 추천 : 미래의 하나의 시간 단계만 예측하고 외생 변수 관측 후, 미래의 다른 시간 단계 예측

6. 모델에 외생 변수 추가

◆ SARIMAX 모델

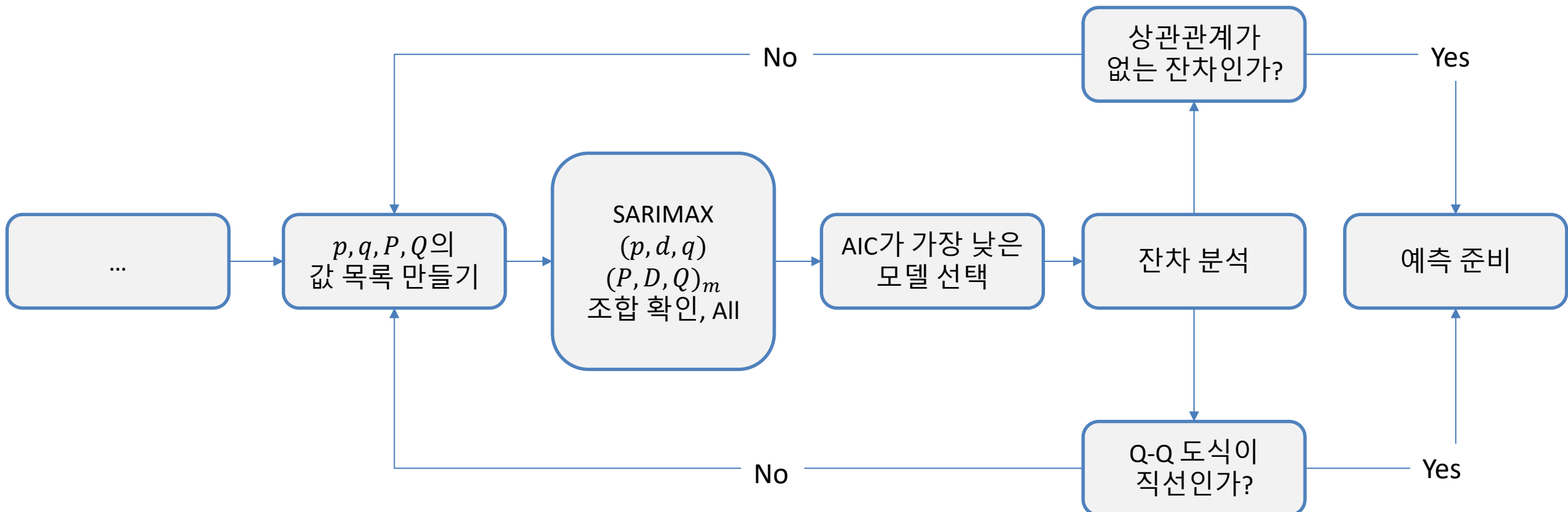
- ✓ SARIMAX 시계열 예측 프로세스



6. 모델에 외생 변수 추가

◆ SARIMAX 모델

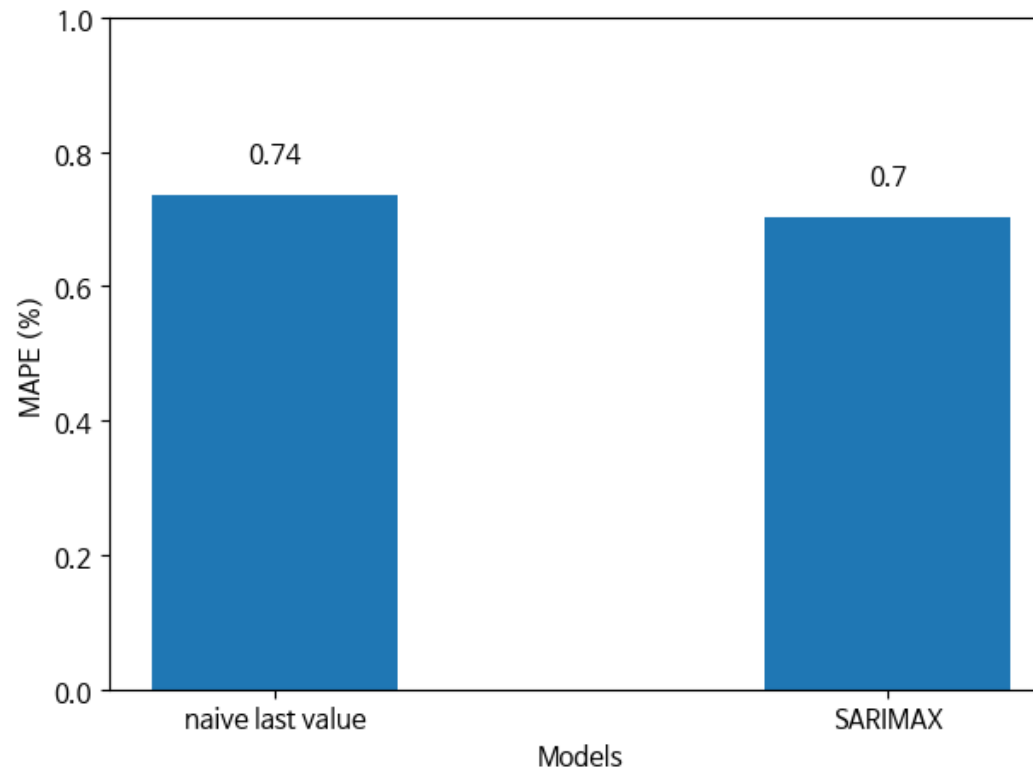
- ✓ SARIMAX 시계열 예측 프로세스



6. 모델에 외생 변수 추가

◆ SARIMAX 모델

- ✓ SARIMAX 시계열 예측 프로세스



7. 다중 시계열 예측하기

◆ VAR Vector Autoregression 모델

- ✓ SARIMAX 모델에서는 관계가 단방향 (외생 변수 → 시계열 대상 데이터)
- ✓ VAR 모델 : 두 시계열이 양방향 관계를 가짐
- ✓ 양방향 관계의 예시 (예시 변수, 실질 소비와 실질 가처분 소득)
 - 가설1 : 가처분 소득이 많을수록 소비가 많다.
 - 가설2 : 소비가 많다는 것은 더 많은 가처분 소득이 있다.

7. 다중 시계열 예측하기

◆ VAR Vector Autoregression 모델

- ✓ VAR 모델은 여러 시계열을 예측할 수 있도록 AR(p) 모델 일반화
- ✓ VAR(1) 모델 수식 (행렬곱셈)

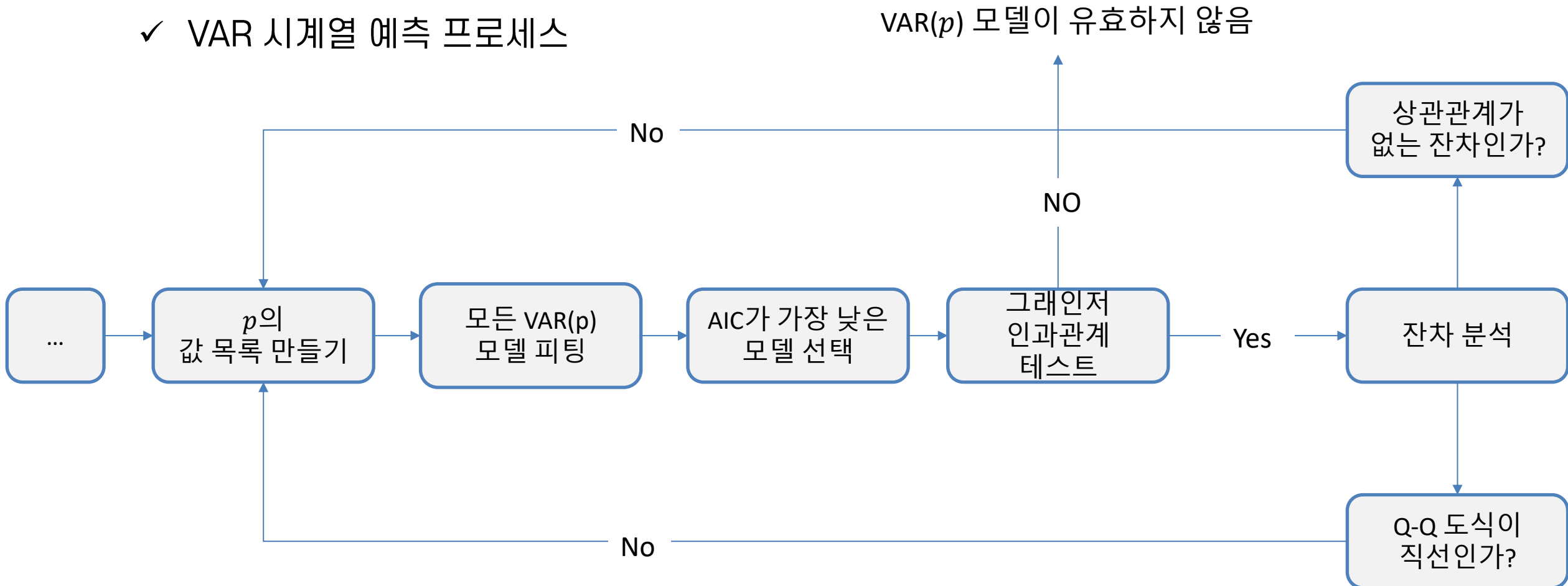
$$\begin{bmatrix} y_{1,t} \\ y_{2,t} \end{bmatrix} = \begin{bmatrix} c_1 \\ c_2 \end{bmatrix} + \begin{bmatrix} \phi_{1,1} & \phi_{1,2} \\ \phi_{2,1} & \phi_{2,2} \end{bmatrix} \begin{bmatrix} y_{1,t-1} \\ y_{2,t-1} \end{bmatrix} + \begin{bmatrix} \epsilon_{1,t} \\ \epsilon_{2,t} \end{bmatrix}$$

- ✓ VAR 모델도 시계열이 정상적이어야 함

7. 다중 시계열 예측하기

◆ VAR 모델

✓ VAR 시계열 예측 프로세스



7. 다중 시계열 예측하기

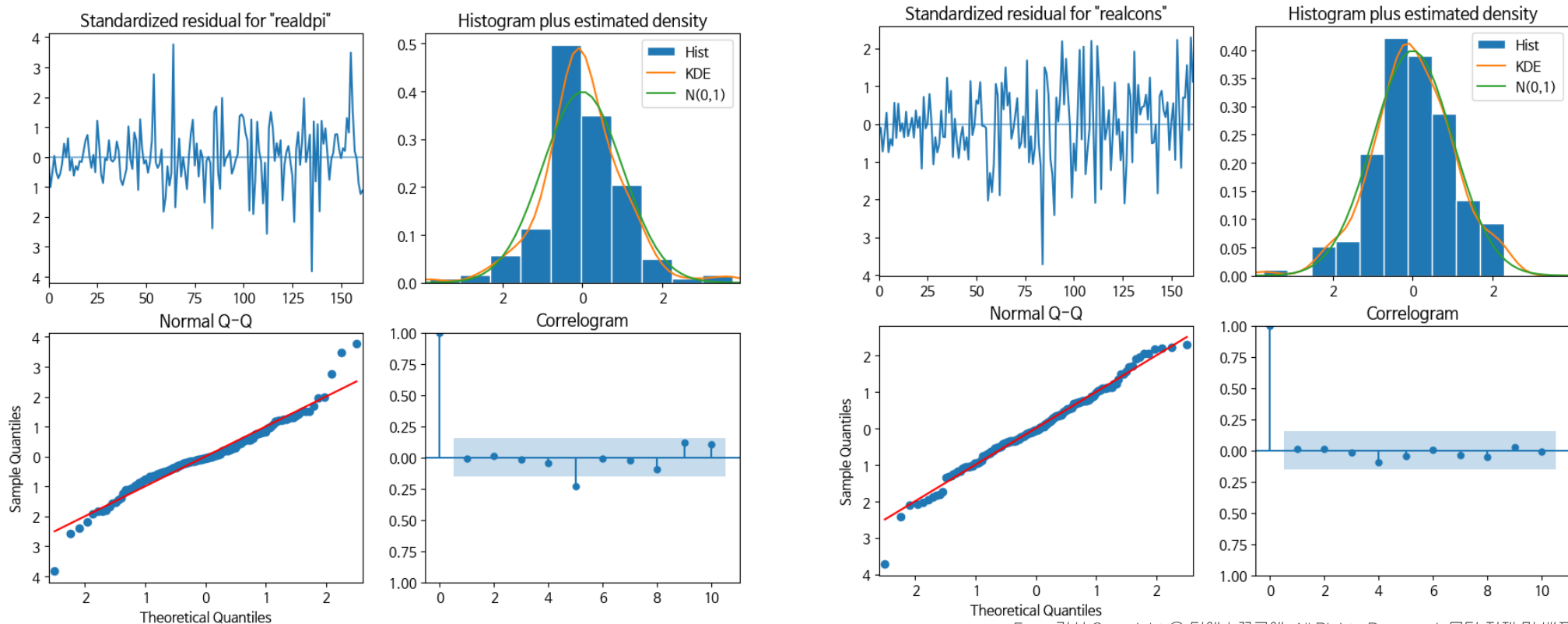
◆ VAR 모델

- ✓ VAR 시계열 예측 프로세스
- ✓ 그레인저 인과관계 테스트 실행 시, 실패하면 $\text{VAR}(p)$ 모델 유효하지 않음, 절차 중지
 - 귀무가설 : 한 변수가 다른 변수를 예측하는데 도움이 되지 않는다.
 - 대립가설 : 한 변수가 다른 변수를 예측하는데 도움이 된다.

7. 다중 시계열 예측하기

◆ VAR 모델

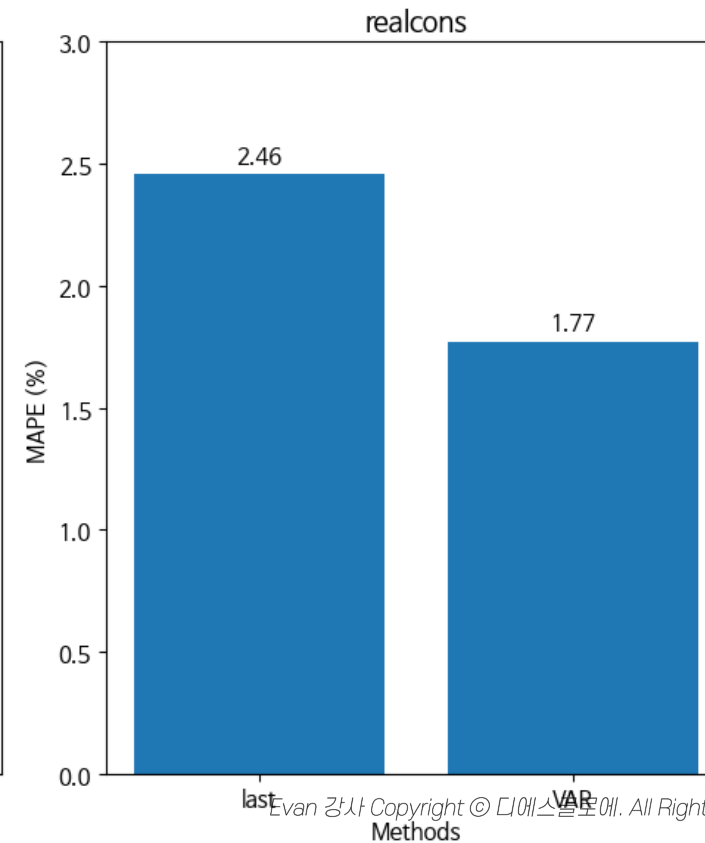
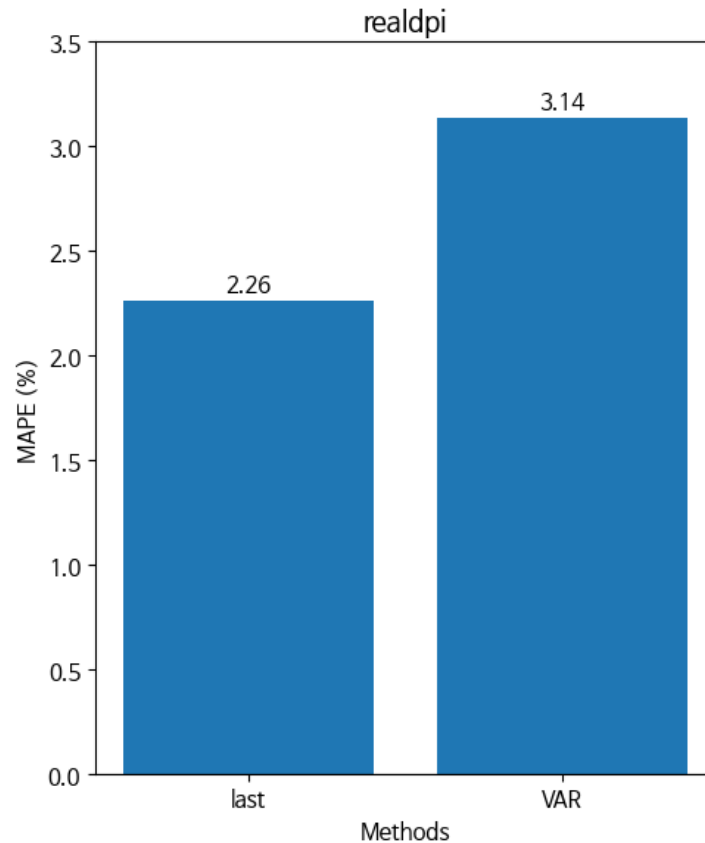
- ✓ realdpi(실질 가처분 소득) 잔차 분석 vs realcons(실질 소비) 잔차 분석 비교



7. 다중 시계열 예측하기

◆ VAR 모델

- ✓ realdpi(실질 가처분 소득), realcons(실질 소비)에 대한 예측 결과의 MAPE



3. 딥러닝을 활용하여 대규모 예측하기

1. 시계열 예측을 위한 딥러닝 소개

◆ 통계적 모델과 딥러닝 모델의 차이점

✓ 데이터 사이즈

통계적 모델	딥러닝 모델
< 10,000	>= 10,000

1. 시계열 예측을 위한 딥러닝 소개

◆ 통계적 모델의 한계

- ✓ 시계열의 경우, 데이터 요소가 10,000개 이상이면 데이터 집합이 큰 것으로 간주
- ✓ SARIMAX 모델에 약간의 변화가 있더라도 피팅하는 데 시간이 오래 걸림
- ✓ 데이터에 여러 가지 계절적 기간이 있는 경우 SARIMAX 모델 사용 어려움
 - 예) 시간별 기온 예측, 일별 계절성 & 연간 계절성 존재 → SARIMAX 모델 사용 어려움

1. 시계열 예측을 위한 딥러닝 소개

◆ 3가지 시계열 딥러닝 모델

- ✓ 단일 단계 모델
- ✓ 다중 단계 모델
- ✓ 다중 출력 모델

1. 시계열 예측을 위한 딥러닝 소개

◆ 단일 단계 모델 Single-Step Model

- ✓ 미래의 시간 단계에 대해 하나의 대상의 값을 출력, 출력은 스칼라
- ✓ 입력은 길어도 관계없지만, 출력은 다음 시간 단계에 대한 단일 예측으로 유지

1. 시계열 예측을 위한 딥러닝 소개

◆ 다중 단계 모델 Multi-Step Model

- ✓ 모델의 출력은 미래의 여러 시간 단계에 대한 예측을 나타내는 일련의 값
- ✓ 모델이 향후 6시간, 24시간, 12개월 예측하는 경우 다중 단계 모델

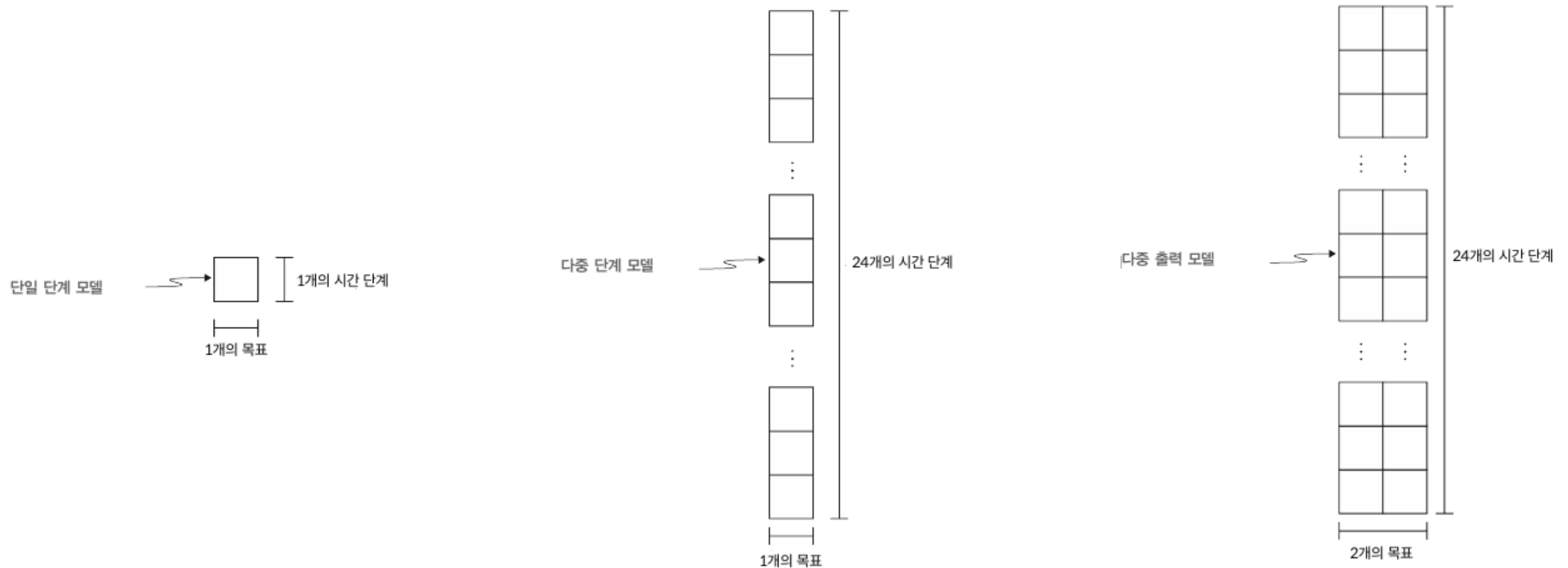
1. 시계열 예측을 위한 딥러닝 소개

◆ 다중 출력 모델 Multi-Output Model

- ✓ 둘 이상의 대상에 대한 예측 생성 (온도와 풍속 예측하는 경우)
- ✓ 미래의 하나 이상의 시간 단계 동안 둘 이상의 대상에 대해 예측 수행

1. 시계열 예측을 위한 딥러닝 소개

◆ 단일 단계, 다중 단계, 다중 출력



1. 시계열 예측을 위한 딥러닝 소개

◆ 실습 데이터 : metro interstate traffic volume

특징	설명
date_time	데이터의 날짜 및 시간으로, CST 표준 시간대. YYYY-MM-DD HH:MM:SS
temp	시간 단계로 기록된 평균 온도로, 켈빈(Kelvin)으로 표시
rain_1h	한 시간 동안 발생한 비의 양. 밀리미터 단위로 표시
snow_1h	한 시간 동안 발생한 눈의 양. 밀리미터로 표시
clouds_all	한 시간 동안 구름이 덮인 비율
traffic_volume	한 시간 동안 I-94에서 서쪽으로 보고된 교통량

1. 시계열 예측을 위한 딥러닝 소개

◆ 데이터셋 가공 및 변환

- ✓ 데이터 특성상 0이 많은 변수는 제거
- ✓ 날짜 초 단위 변환 → Sin, Cosin 함수 적용
- ✓ 데이터 분할 (훈련:검증:테스트) 70:20:10
- ✓ 훈련데이터에만 MinMax Scale 적용
- ✓ 데이터 저장

2. 데이터 윈도우잉 클래스 및 베이스라인 모델

◆ 데이터 윈도우잉Data Windowing

✓ 데이터 윈도우의 예시

입력					레이블				
$t = 0$	$t = 1$	$t = \dots$	$t = 22$	$t = 23$	$t = 24$	$t = 25$	$t = \dots$	$t = 46$	$t = 47$
데이터 윈도우									

2. 데이터 윈도우잉 클래스 및 베이스라인 모델

◆ 데이터 윈도우잉 Data Windowing

✓ 데이터 윈도우의 예시

입력					레이블				
$t = 0$	$t = 1$	$t = \dots$	$t = 22$	$t = 23$	$t = 24$	$t = 25$	$t = \dots$	$t = 46$	$t = 47$
데이터 윈도우 #1									

2. 데이터 윈도우잉 클래스 및 베이스라인 모델

◆ 데이터 윈도우잉Data Windowing

✓ 데이터 윈도우의 예시

입력					레이블				
$t = 1$	$t = 2$	$t = \dots$	$t = 23$	$t = 24$	$t = 25$	$t = 26$	$t = \dots$	$t = 47$	$t = 48$
데이터 윈도우 #2									

2. 데이터 윈도우잉 클래스 및 베이스라인 모델

◆ 데이터 윈도우잉 Data Windowing

✓ 데이터 윈도우의 예시

입력					레이블				
$t = 2$	$t = 3$	$t = \dots$	$t = 24$	$t = 25$	$t = 26$	$t = 27$	$t = \dots$	$t = 48$	$t = 49$
데이터 윈도우 #3									

2. 데이터 윈도우잉 클래스 및 베이스라인 모델

◆ 배치^{batch}

- ✓ 훈련을 위해 모델에 공급되는 데이터 윈도우의 모음 (전체데이터수 / 32)



2. 데이터 윈도우잉 클래스 및 베이스라인 모델

◆ 에포크Epoch

- ✓ 모든 배치에 대해 모델을 한 번 훈련 하는 것
- ✓ 하나의 에포크로는 정확한 모델 만들 수 없으므로 예측 정확도가 더 이상 개선되지 않을 때까지 필요한 만큼의 에포크에 걸쳐 훈련함

2. 데이터 윈도우잉 클래스 및 베이스라인 모델

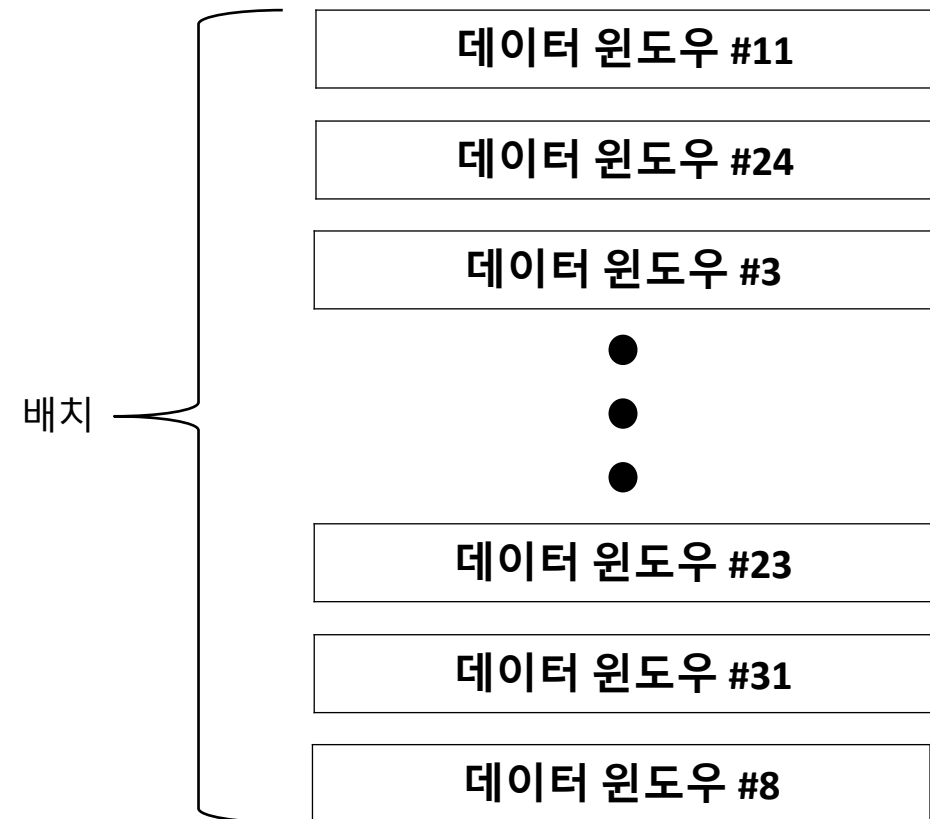
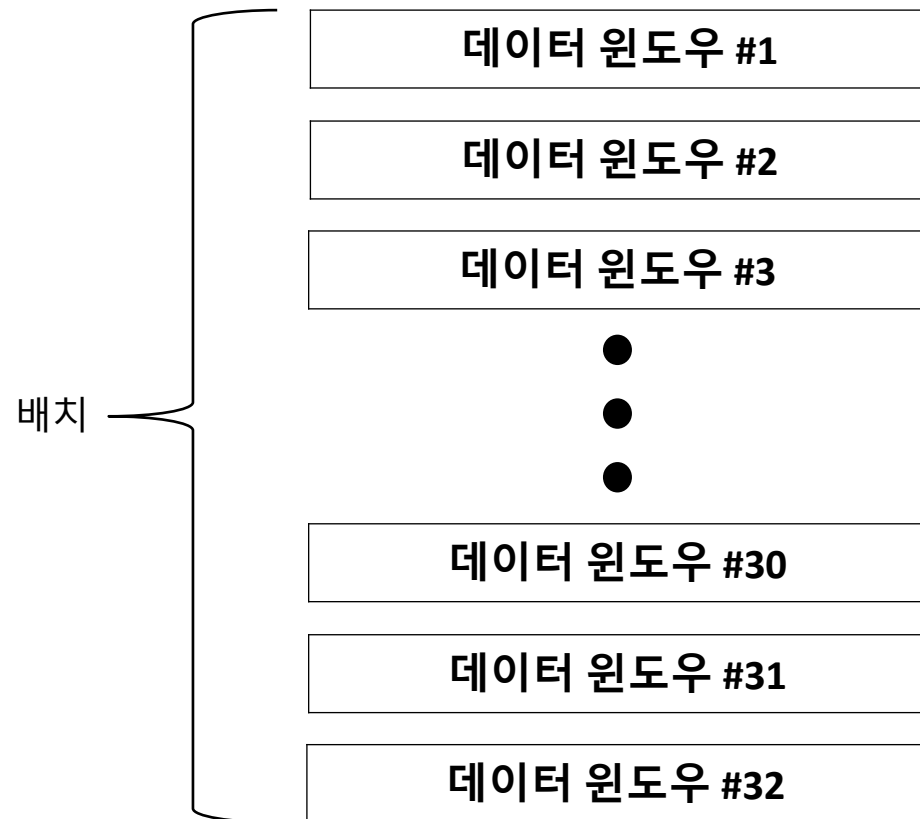
◆ 셔플링Shuffling

- ✓ 모든 배치에 대해 모델을 한 번 훈련 하는 것
- ✓ 하나의 에포크로는 정확한 모델 만들 수 없으므로 예측 정확도가 더 이상 개선되지 않을 때까지 필요한 만큼의 에포크에 걸쳐 훈련함
- ✓ 배치 내에서 데이터 윈도우를 섞어도 시계열의 순서는 그대로 유지
- ✓ 견고한 모델을 만드는 데 도움을 줄 수 있음

2. 데이터 윈도우 클래스 및 베이스라인 모델

◆ 셔플링Shuffling

✓ 셔플링 실행 False/True



2. 데이터 윈도우 클래스 및 베이스라인 모델

◆ 데이터 윈도우 개념

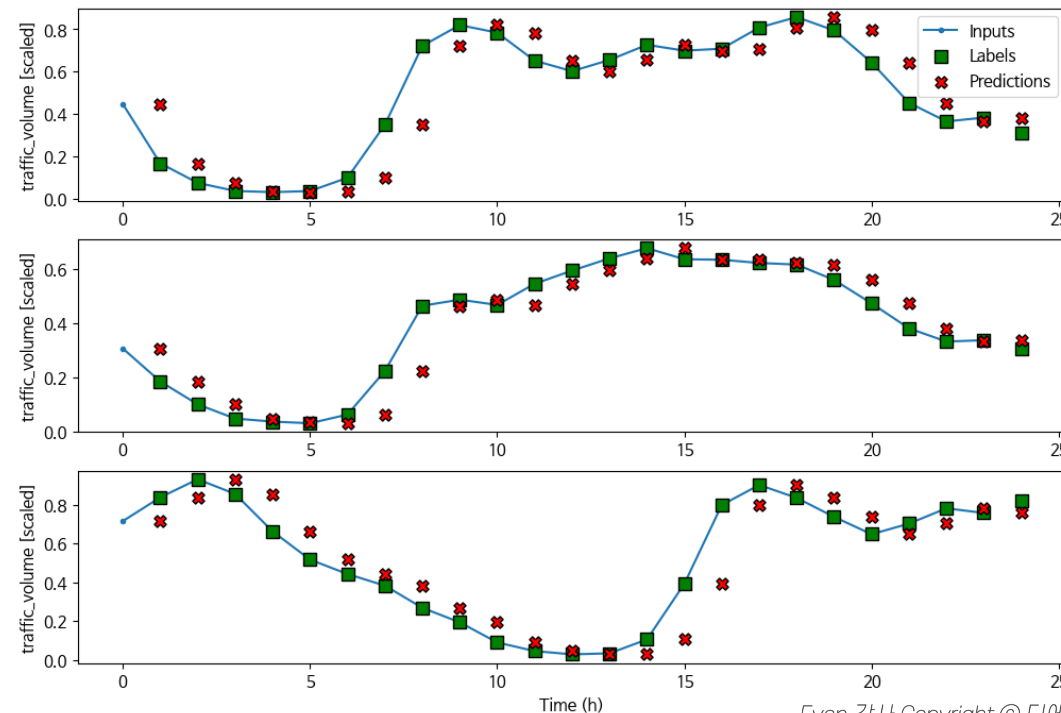
- ✓ 입력 너비, 레이블 너비, 이동, 윈도우 크기
- ✓ 각 집합에 해당하는 데이터 윈도우는 전체 데이터 집합에서 파생

입력 너비 = 24					레이블 너비 = 24				
$t = 0$	$t = 1$	$t = \cdots$	$t = 22$	$t = 23$	$t = 24$	$t = 25$	$t = \cdots$	$t = 46$	$t = 47$
					이동 = 24				
총 윈도우 크기 = 48									

2. 데이터 윈도우잉 클래스 및 베이스라인 모델

◆ 단일 단계 베이스라인 모델

- ✓ 레이블은 사각형, 예측은 십자가 모양
- ✓ 각 시간 단계의 십자가는 단순히 마지막으로 측정된 값 (베이스라인 모델 존재)



2. 데이터 원도잉 클래스 및 베이스라인 모델

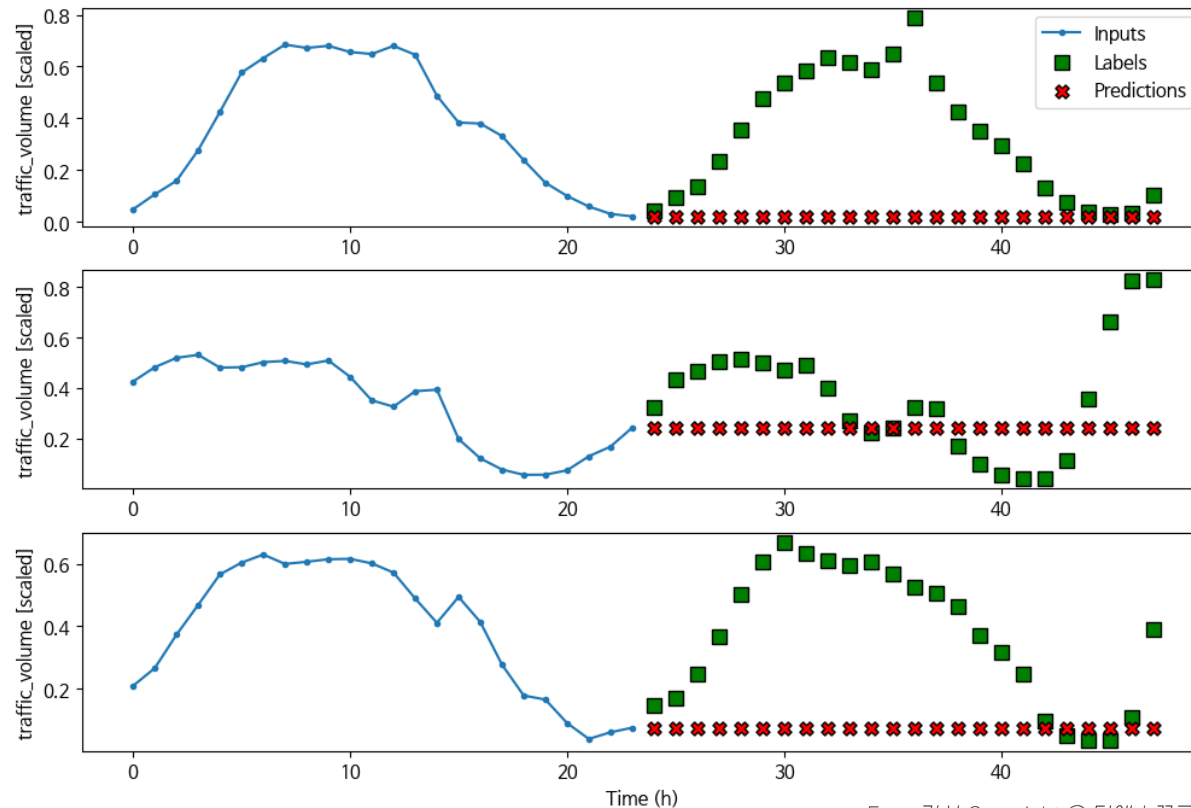
◆ 다중 단계 베이스라인 모델

- ✓ 24시간 입력, 미래의 24개 시간 단계 예측, 입력 너비 24, 레이블 너비 24, 이동 너비 24
- ✓ 두개의 베이스라인
 - 다음 24개 시간 단계에 대해 마지막으로 측정된 값 예측
 - 다음 24개 시간 단계에 대해 마지막으로 24개 시간 단계 예측

2. 데이터 윈도우잉 클래스 및 베이스라인 모델

◆ 다중 단계 베이스라인 모델 – 마지막으로 측정된 값 예측

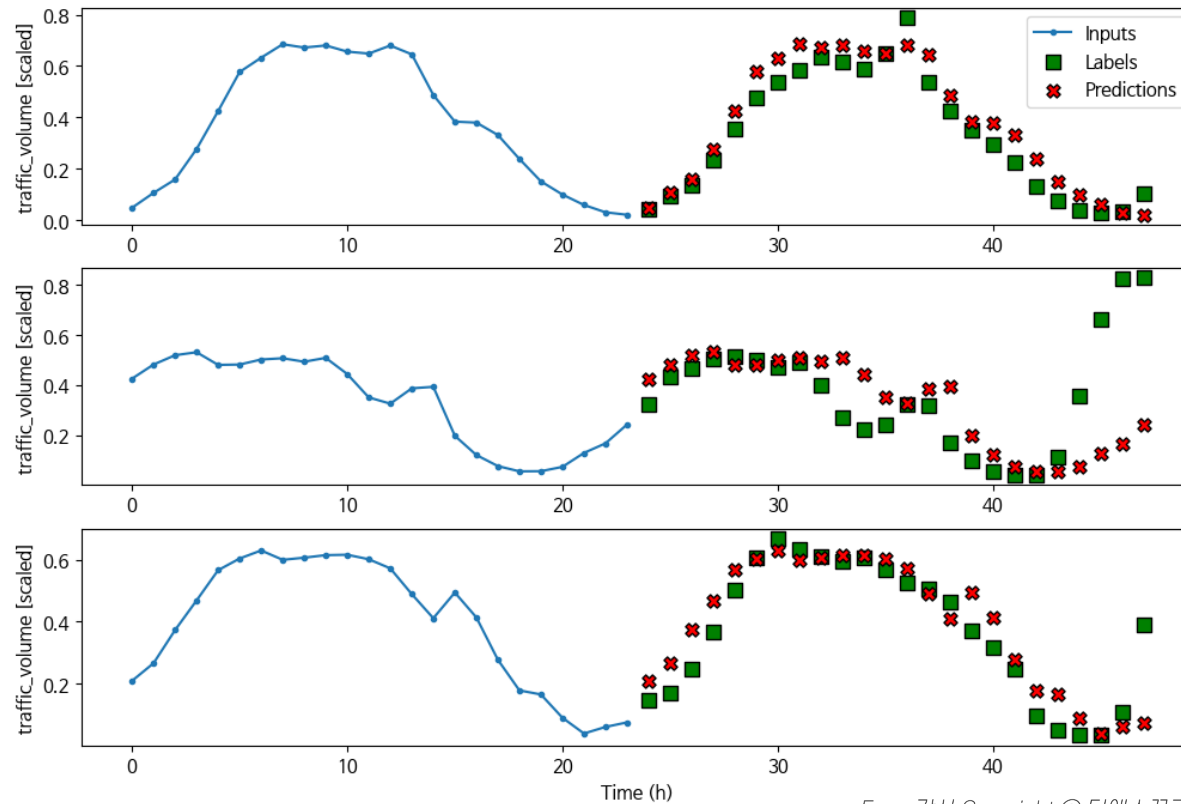
- ✓ 24시간 입력, 미래의 24개 시간 단계 예측, 입력 너비 24, 레이블 너비 24, 이동 너비 24



2. 데이터 윈도우잉 클래스 및 베이스라인 모델

◆ 다중 단계 베이스라인 모델 – 입력 배열 반복하기

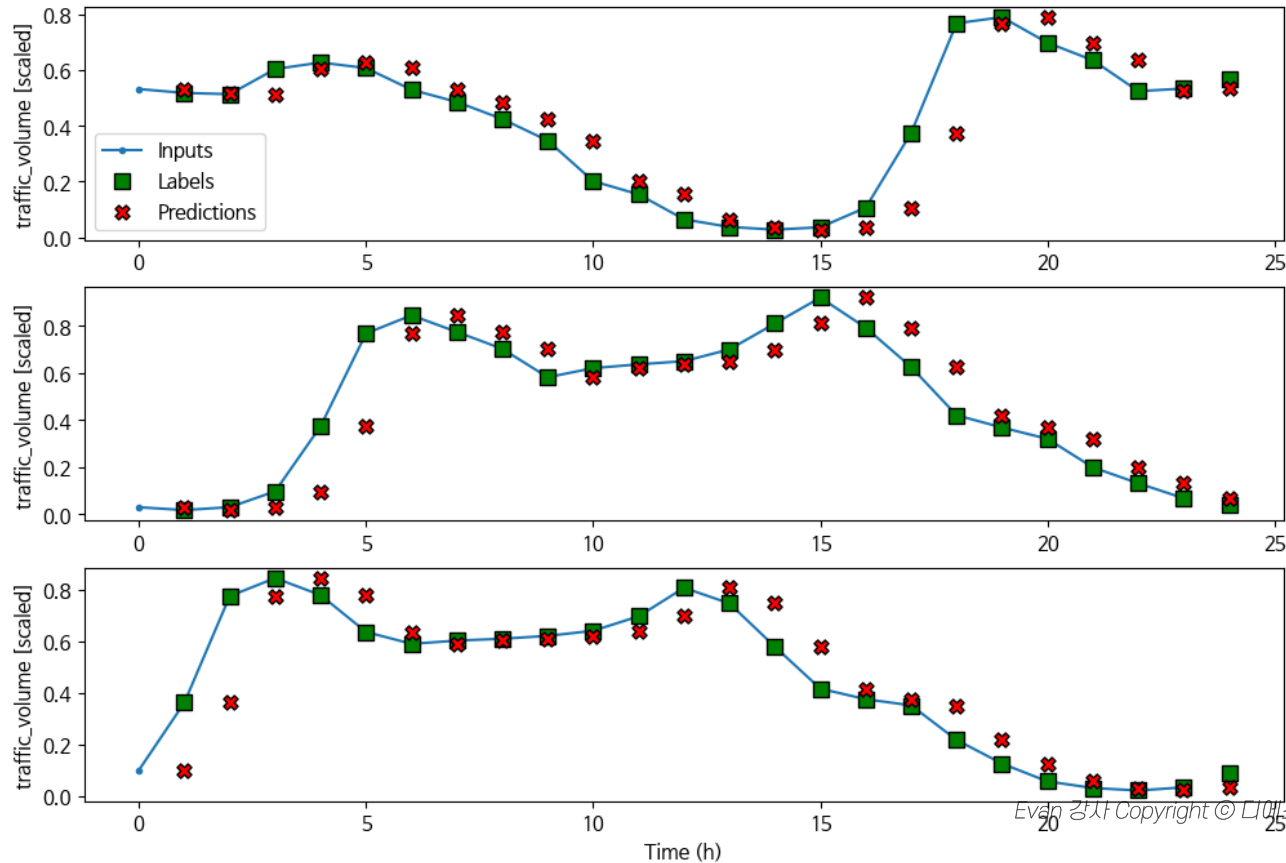
- ✓ 24시간 입력, 미래의 24개 시간 단계 예측, 입력 너비 24, 레이블 너비 24, 이동 너비 24



2. 데이터 윈도우잉 클래스 및 베이스라인 모델

◆ 다중 출력 베이스라인 모델

- ✓ 하나의 입력 데이터 요소를 사용하여 다음 시간 단계의 교통량과 온도 예측

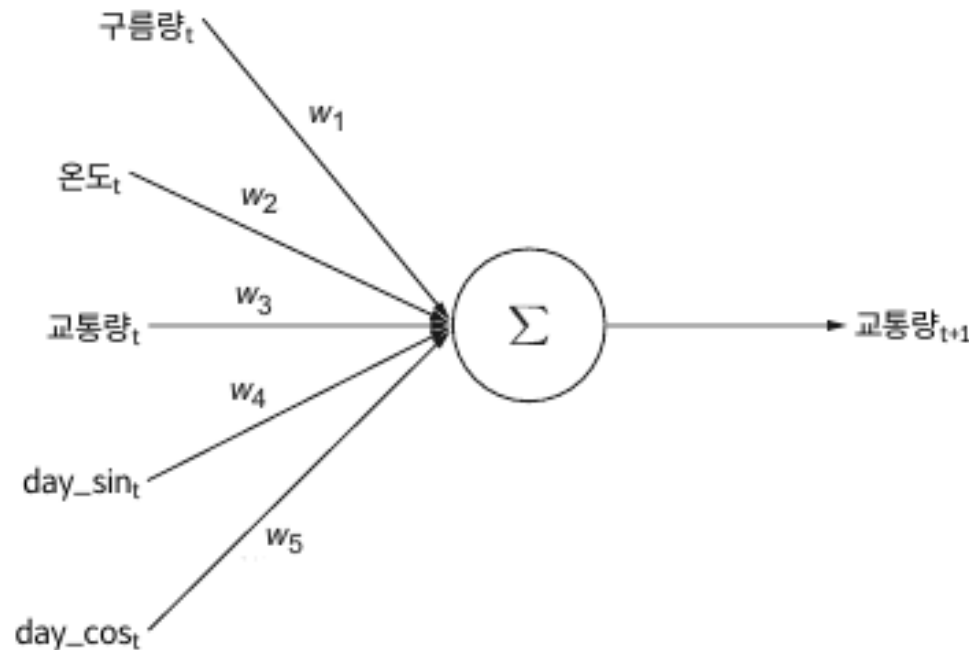


3. 선형모델과 심층신경망 구축

◆ 단일 단계 선형 모델 구현

- ✓ 각 입력 특징에 가중치를 부여하고 이를 결합하여 기존의 선형 회귀와 마찬가지로 예측 출력

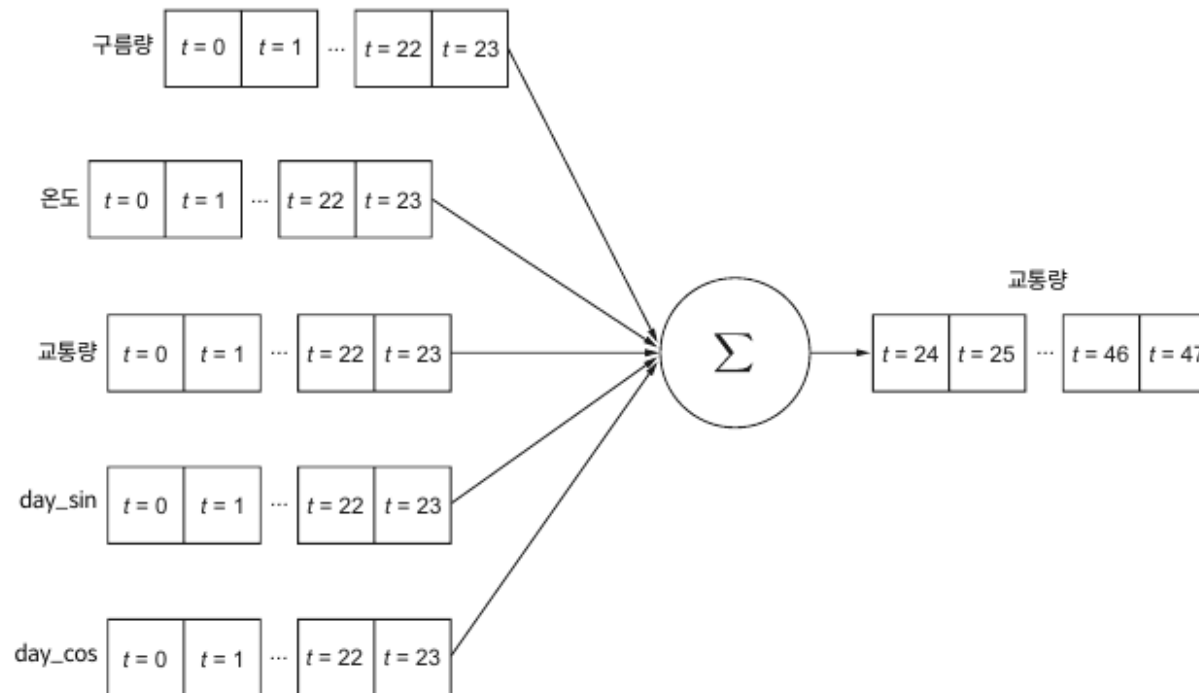
$$\text{traffic volume}_{t+1} = w_1x_{1,t} + w_2x_{2,t} + w_3x_{3,t} + w_4x_{4,t} + w_5x_{5,t}$$



3. 선형모델과 심층신경망 구축

◆ 다중 단계 선형 모델 구현

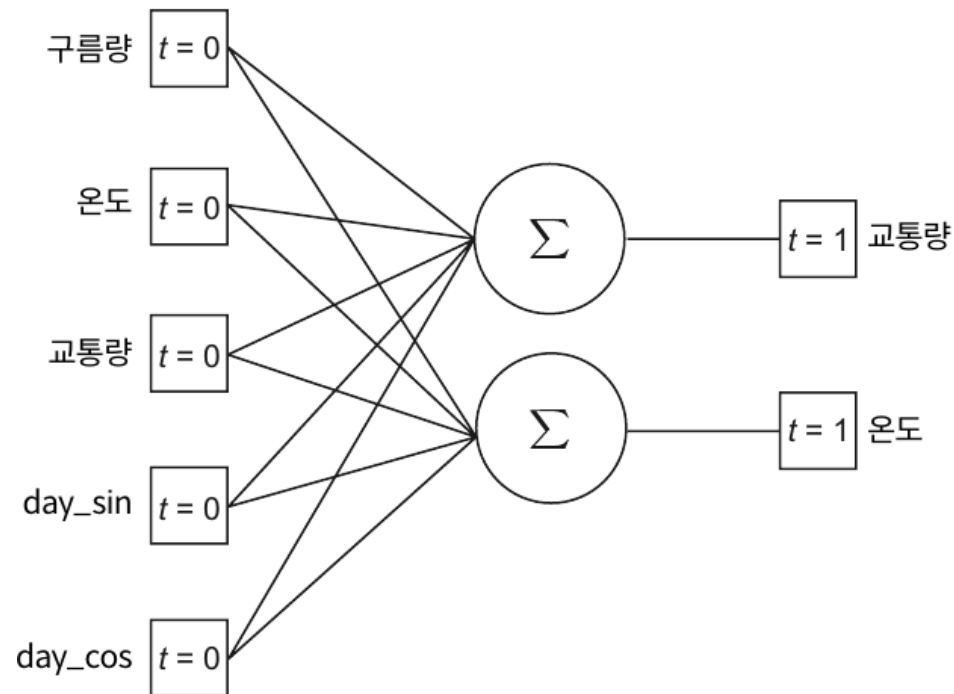
- ✓ 24시간의 데이터를 입력 윈도우로 사용하여 다음 24시간의 데이터 예측



3. 선형모델과 심층신경망 구축

◆ 다중 출력 선형 모델 구현

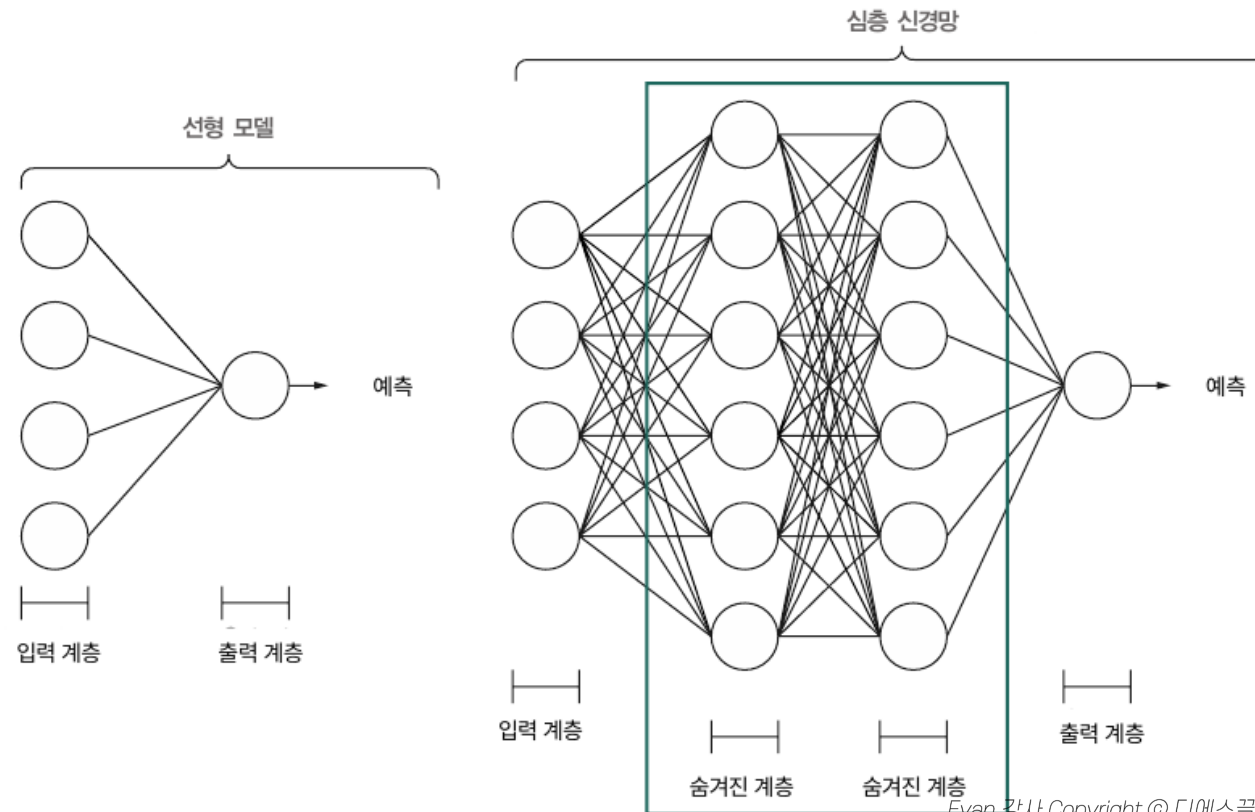
- ✓ 교통량과 온도에 대한 예측 반환
- ✓ 입력은 현재 시간 단계, 예측은 다음 시간 단계에 대한 것



3. 선형모델과 심층신경망 구축

◆ 선형 모델과 심층 신경망 모형 비교

- ✓ 심층 신경망에 숨겨진 계층 포함, 이러한 계층 통해 입력과 예측 간의 비선형 관계 모델링



3. 선형모델과 심층신경망 구축

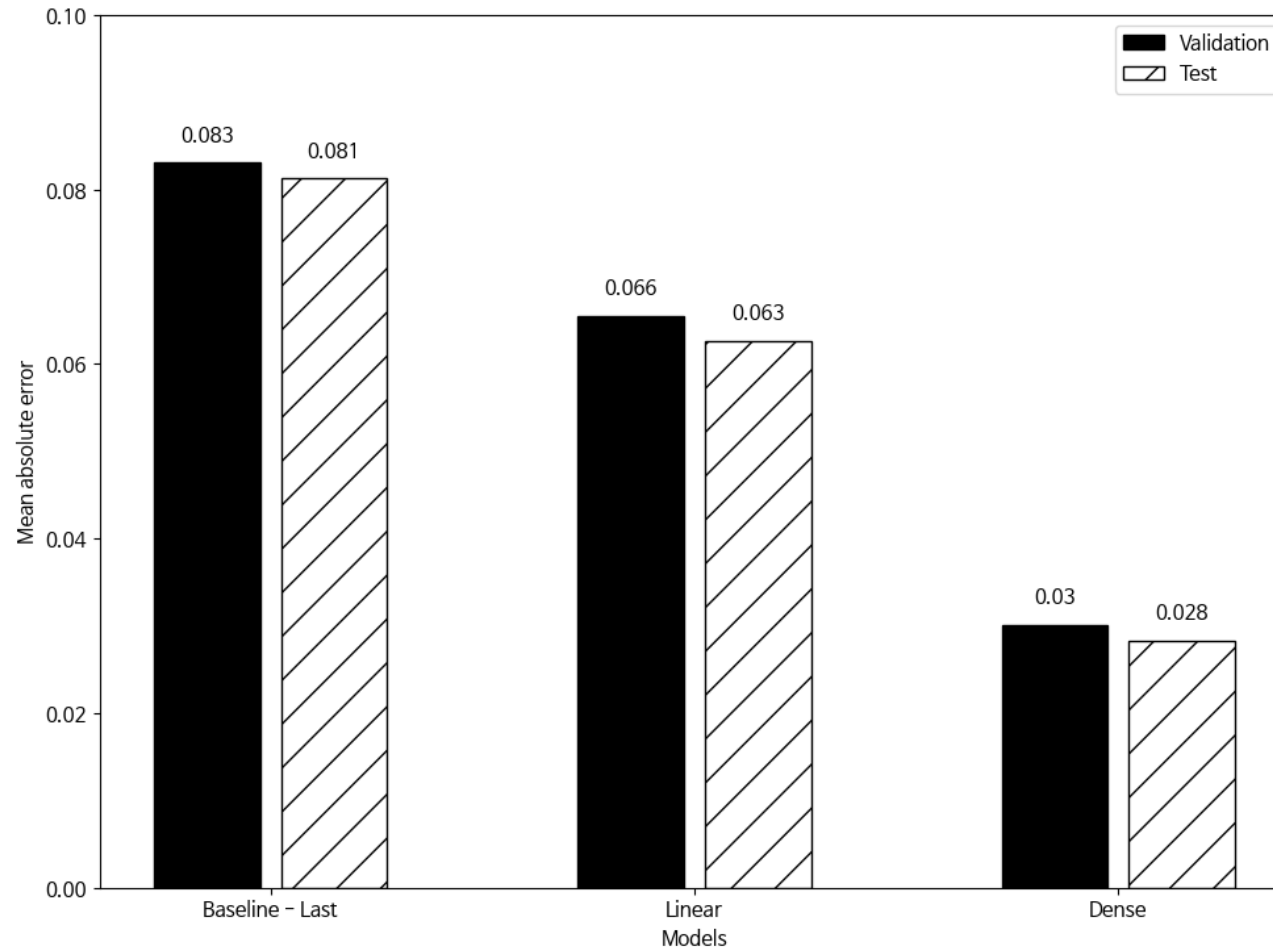
◆ 활성화 함수 Activation Function 설정

- ✓ 숨겨진 계층의 각 뉴런에 활성화 함수 설정
- ✓ 비선형 관계 모델링하려면 비선형 활성화 함수 사용 (ReLU, softmax, tanh)
- ✓ ReLU 함수 공식
 - 양수인 경우에는 그대로 반환, 0 이하인 경우에는 0을 반환

$$f(x) = x^+ = \max(0, x)$$

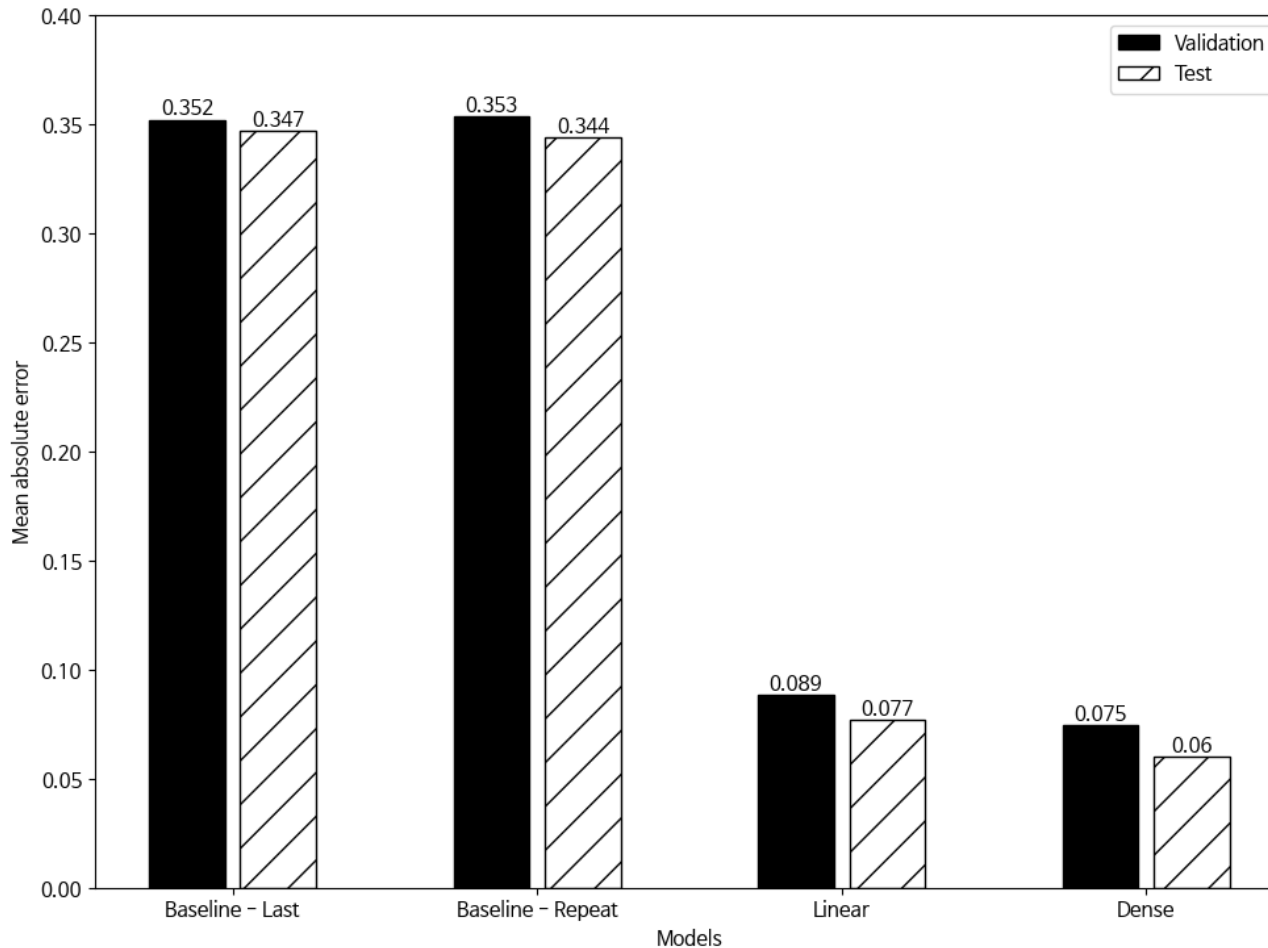
3. 선형모델과 심층신경망 구축

◆ 모델 비교 – 단일 단계 모델



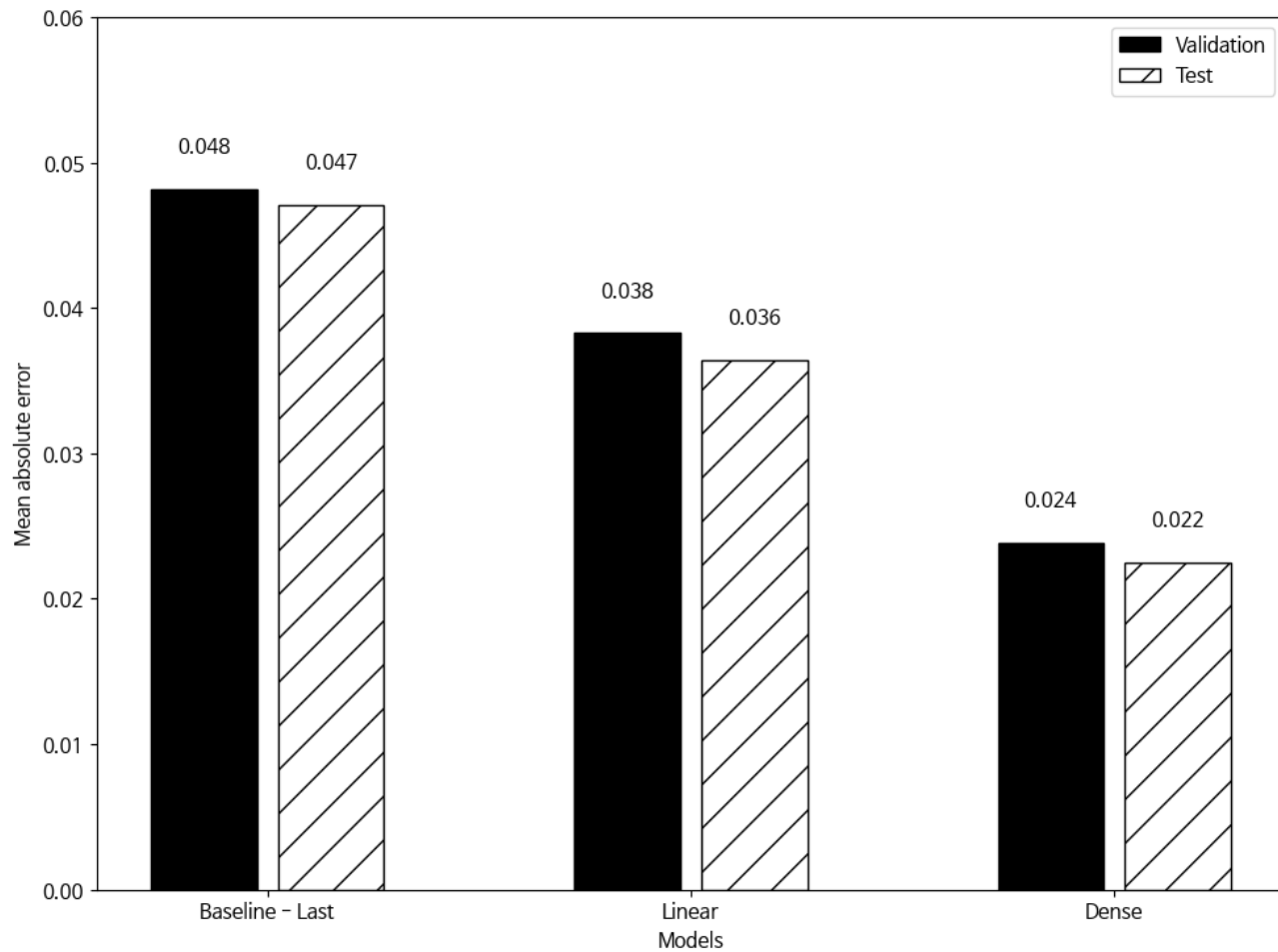
3. 선형모델과 심층신경망 구축

◆ 모델 비교 – 다중 단계 모델



3. 선형모델과 심층신경망 구축

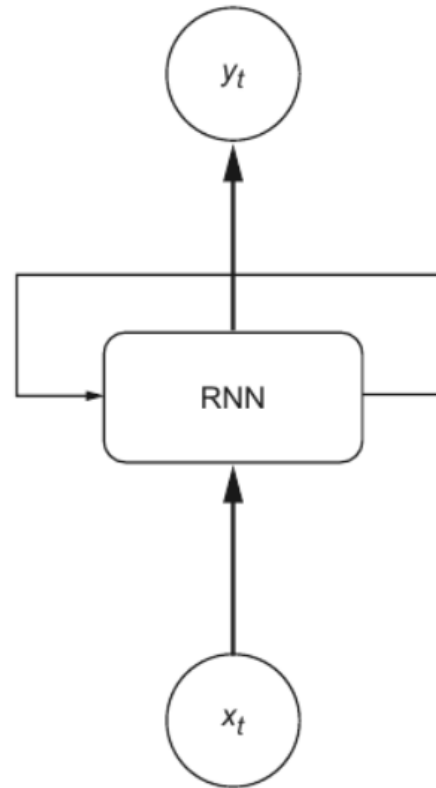
◆ 모델 비교 – 다중 출력 모델



4. LSTM을 활용한 시계열 예측

◆ 순환 신경망(RNN^{recurrent neural network})

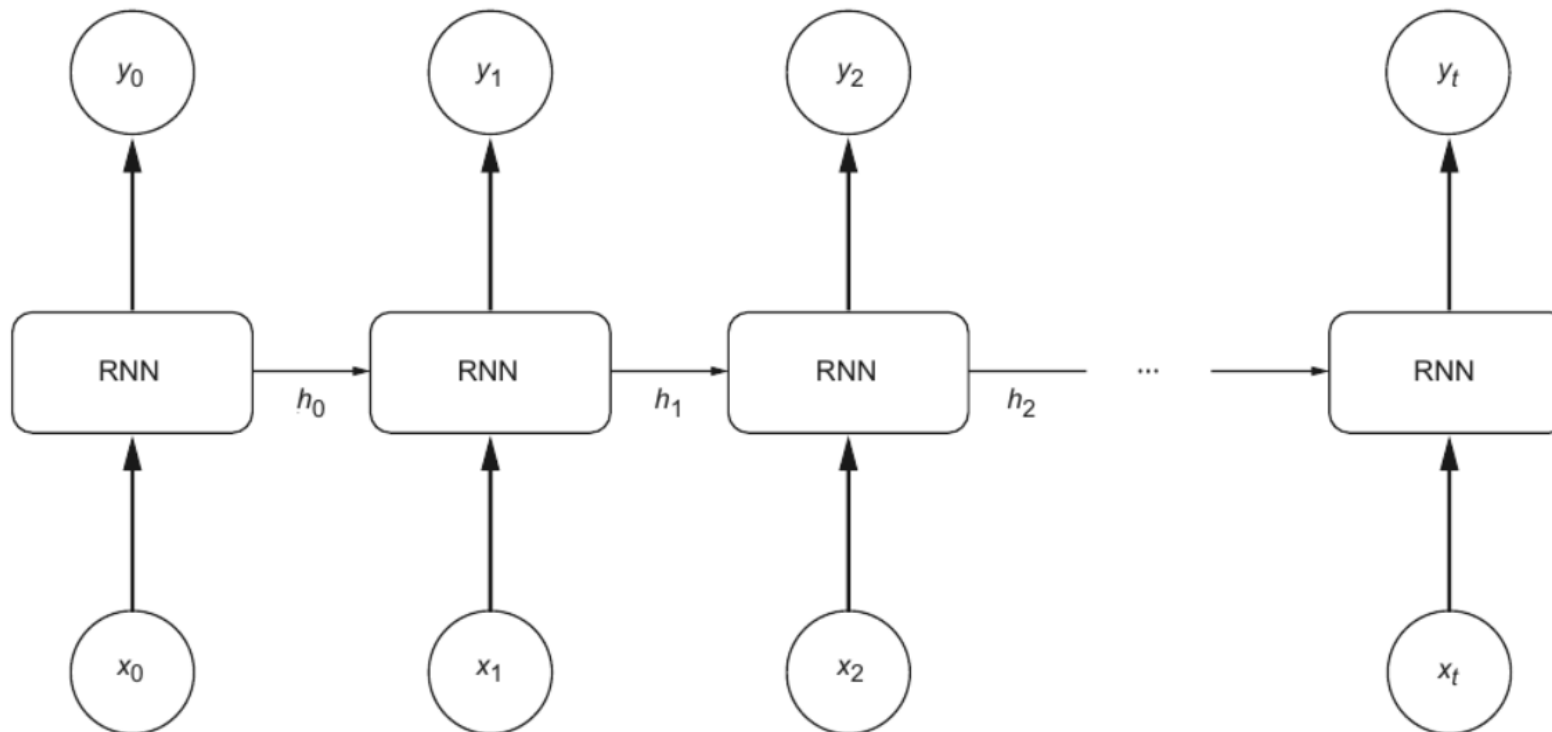
- ✓ x_t 로 표시되는 입력, 네트워크서 루프되어 배열의 다음 입력과 결합되는 숨겨진 상태 h_t 계산



4. LSTM을 활용한 시계열 예측

◆ 순환 신경망(RNN^{recurrent neural network})

- ✓ x_t 로 표시되는 입력, 네트워크서 루프되어 배열의 다음 입력과 결합되는 숨겨진 상태 h_t 계산



4. LSTM을 활용한 시계열 예측

◆ 순환 신경망(RNN^{recurrent neural network})

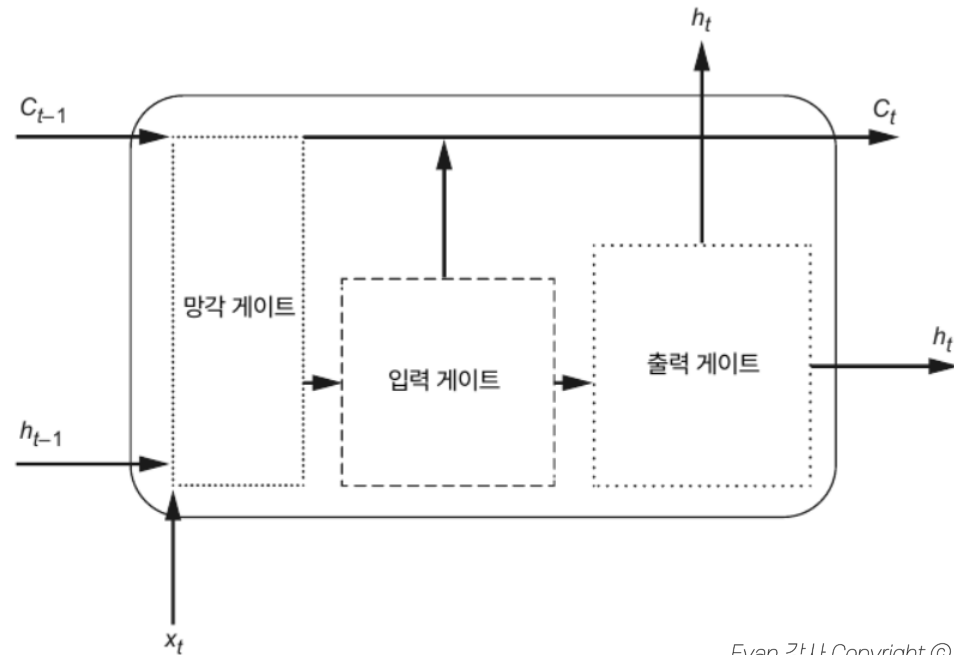
- ✓ x_t 로 표시되는 입력, 네트워크서 루프되어 배열의 다음 입력과 결합되는 숨겨진 상태 h_t 계산
- ✓ 데이터 배열을 처리하는 데 적합
- ✓ 배열의 다음 요소 처리할 때 피드백 되는 숨겨진 상태를 사용하여 과거 정보를 입력으로 활용
- ✓ 단점
 - 단기 기억
 - 배열의 초기 요소에서 얻은 정보가 배열 후반으로 갈수록 더 이상 영향을 미치지 못함
 - 6시 입력 데이터가 7시 출력에는 영향을 줄 수 있지만, 9시, 10시 출력에는 영향 미미해짐

4. LSTM을 활용한 시계열 예측

◆ LSTM Long Short-Term Memory

✓ RNN 아키텍처에 셀 상태 추가

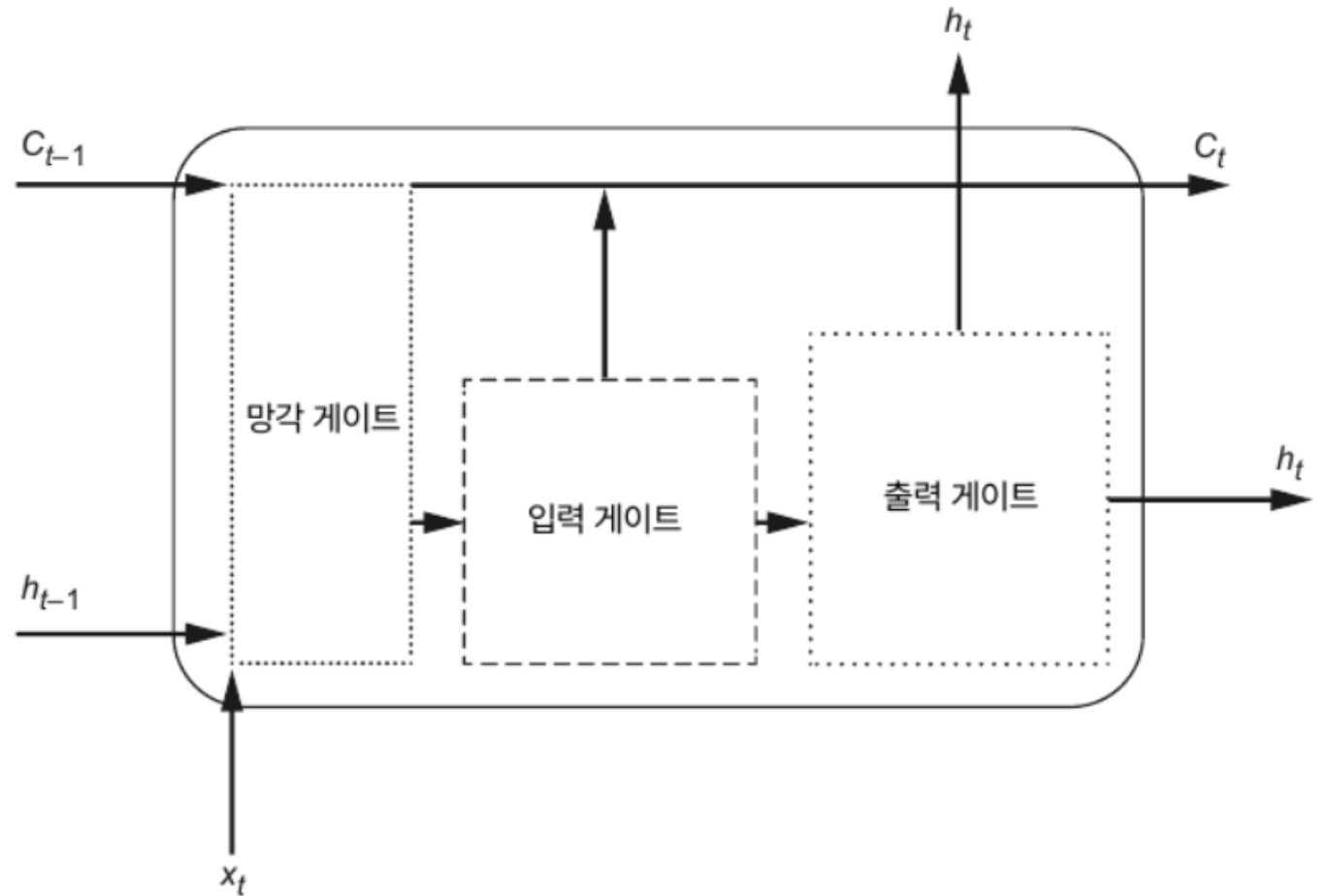
- 과거 정보가 네트워크 학습에 더 이상 영향을 미치지 않는 기울기 소멸 문제 피하기
- 네트워크는 과거 정보를 더 오랫동안 메모리에 보관 가능



4. LSTM을 활용한 시계열 예측

◆ LSTM Long Short-Term Memory

- ✓ LSTM 뉴런의 아키텍처
 - 셀 상태는 C 로 표시
 - 입력 x , 숨겨진 상태 h



4. LSTM을 활용한 시계열 예측

◆ LSTM Long Short-Term Memory

✓ 3개의 게이트

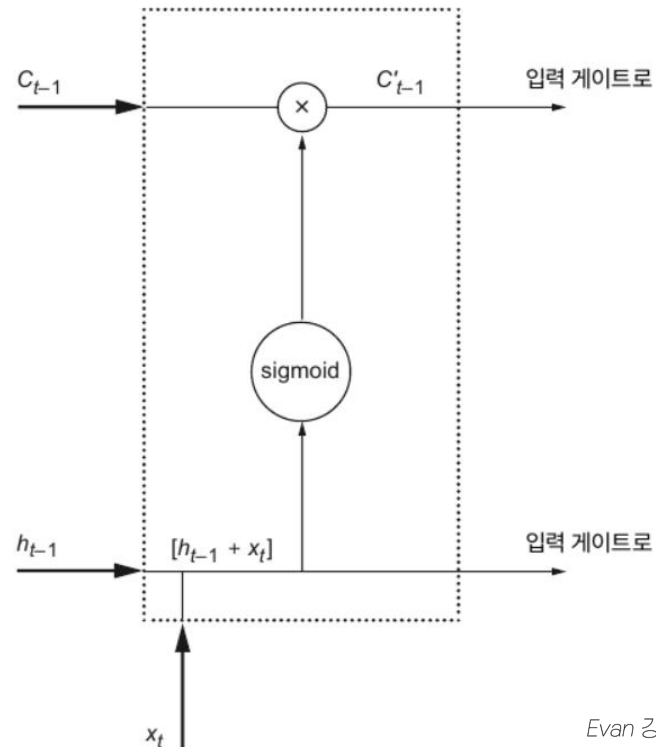
- 망각 게이트 : 과거 단계의 어떤 정보가 여전히 관련성이 있는지 결정
- 입력 게이트 : 현재 단계의 어떤 정보가 관련성이 있는지 결정
- 출력 게이트 : 배열의 다음 요소로 전달할 정보나 결과적으로 출력 계층으로 전달할 정보 결정

4. LSTM을 활용한 시계열 예측

◆ LSTM Long Short-Term Memory

✓ 망각게이트

- 배열의 과것값과 현재값 중 어떤 정보를 네트워크에서 잊어버리거나 유지해야 하는지 결정

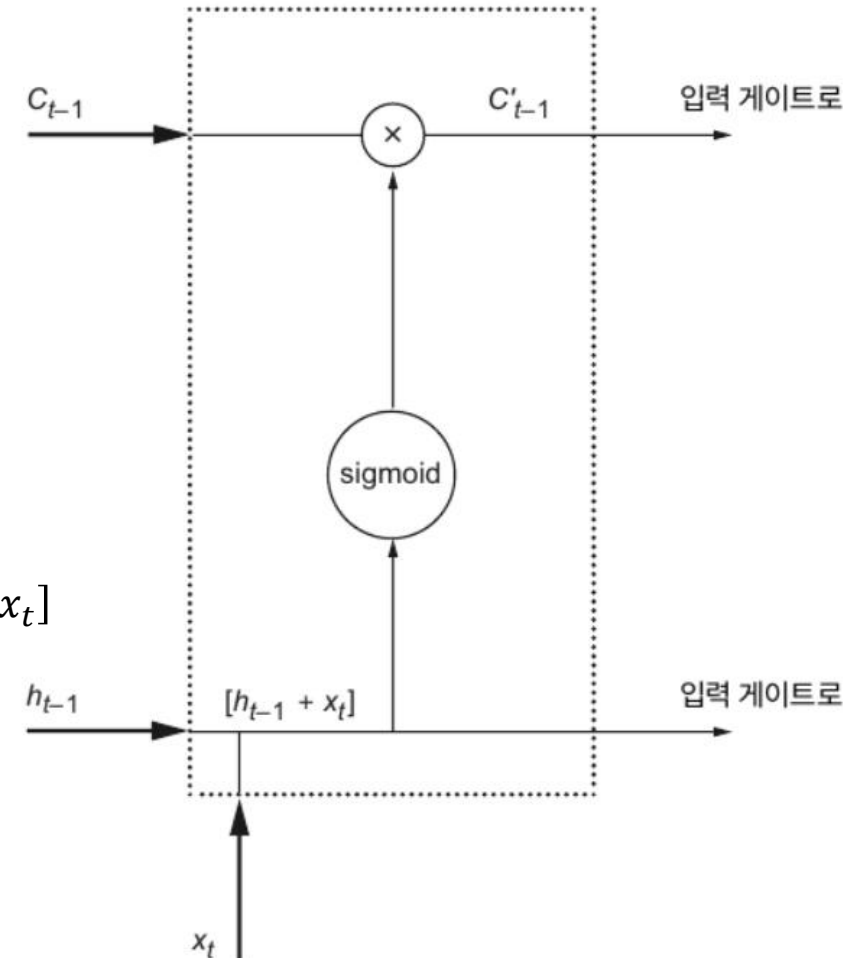


4. LSTM을 활용한 시계열 예측

◆ LSTM Long Short-Term Memory

✓ 망각게이트

- 시그모이드 활성화 함수 통과
 - 출력값 0에 가까움 : 정보를 잊어버려야 함을 뜻함
 - 출력값 1에 가까움 : 정보를 유지해야 함을 뜻함
- 출력 → 입력 게이트에 전달
 - 업데이트 된 셀 상태 C'_{t-1}
 - 과거 숨겨진 상태와 배열의 현재 요소 간 조합의 복사본 $[h_{t-1} + x_t]$

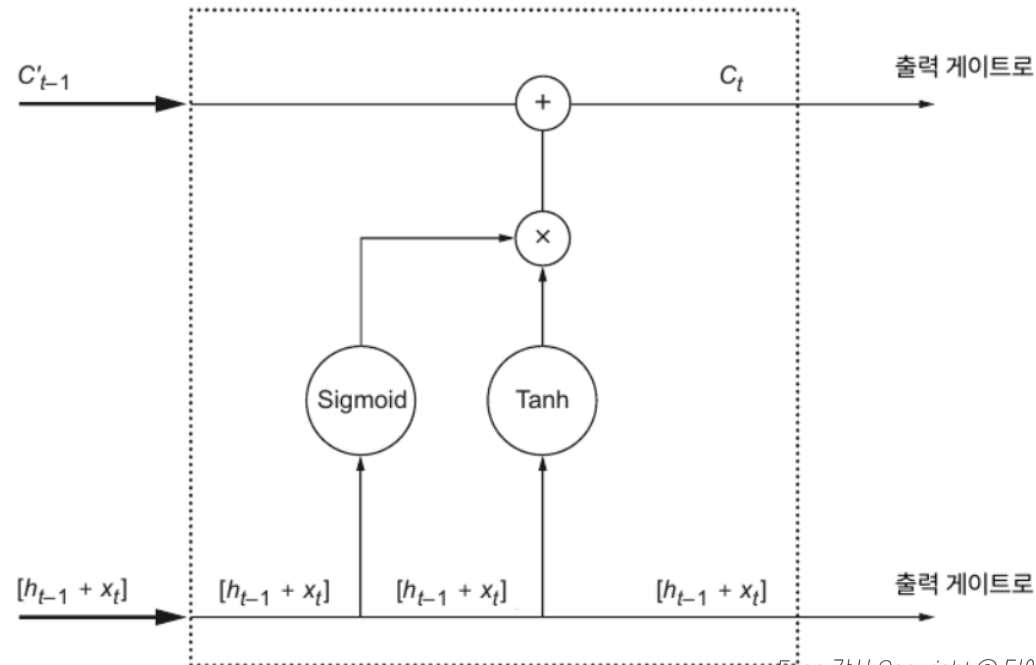


4. LSTM을 활용한 시계열 예측

◆ LSTM Long Short-Term Memory

✓ 입력게이트

- 배열의 현재 요소에서 어떤 정보가 관련성이 있는지 결정하는 단계
- 셀 상태가 다시 업데이트되어 최종 셀 상태가 됨



4. LSTM을 활용한 시계열 예측

◆ LSTM Long Short-Term Memory

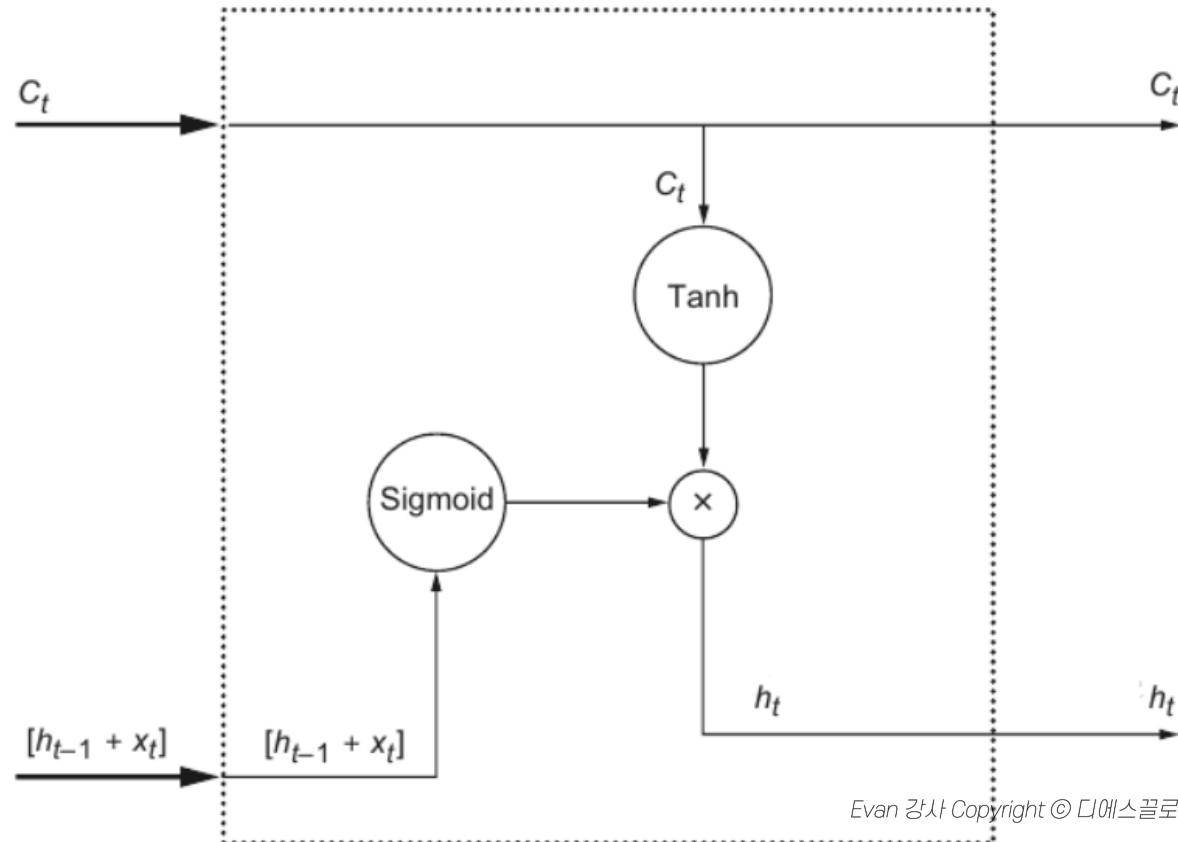
✓ 입력게이트

- 시그모이드 함수의 역할 : 어떤 정보를 유지하거나 삭제할지 결정
- 쌍곡 탄젠트 함수의 역할 : 네트워크의 계산이 효율적으로 유지되도록 하기 위해 네트워크 조절
- 두 연산의 결과를 조합하여 최종 셀 상태를 업데이트 하는데 사용
- 동시에 출력 게이트에 동일한 조합도 전달

4. LSTM을 활용한 시계열 예측

◆ LSTM Long Short-Term Memory

✓ 출력게이트



4. LSTM을 활용한 시계열 예측

◆ LSTM Long Short-Term Memory

✓ 출력게이트

- 시그모이드 함수의 역할 : 어떤 정보를 유지하거나 삭제할지 결정
- 쌍곡 탄젠트 함수의 역할 : 네트워크의 계산이 효율적으로 유지되도록 하기 위해 네트워크 조절
- 과거 정보를 사용하여 배열의 현재 요소를 처리하는 단계
- 그런 다음 새로운 숨겨진 상태 h_t 를 출력하여 다음 LSTM 뉴런이나 출력 계층으로 전달, 셀 상태 출력

4. LSTM을 활용한 시계열 예측

◆ LSTM Long Short-Term Memory

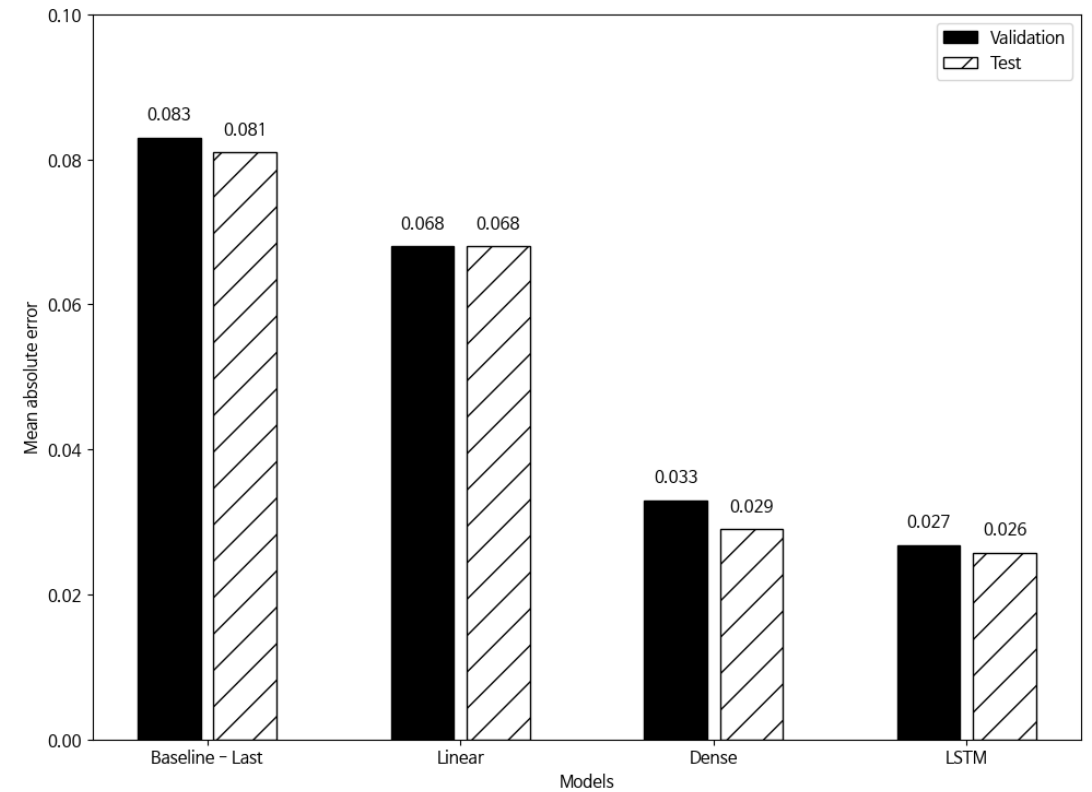
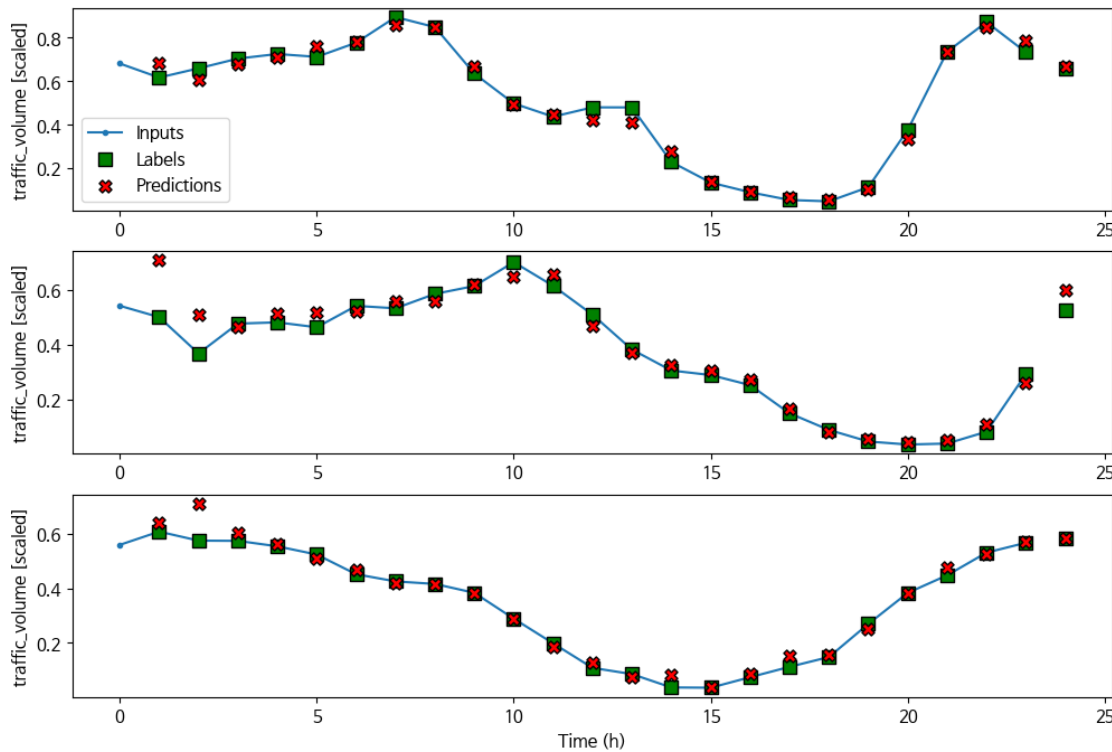
✓ 요약

- 망각게이트 : 과거의 어떤 정보를 보관할지 또는 버릴지 결정
- 입력게이트 : 네트워크 메모리를 업데이트 위해 현재 단계에서 어떤 정보를 유지할지 또는 버릴지 결정
- 출력게이트 : 네트워크 메모리에 저장된 과거 정보 사용하여 배열의 현재 요소 처리

4. LSTM을 활용한 시계열 예측

◆ 단일 단계 LSTM 모델

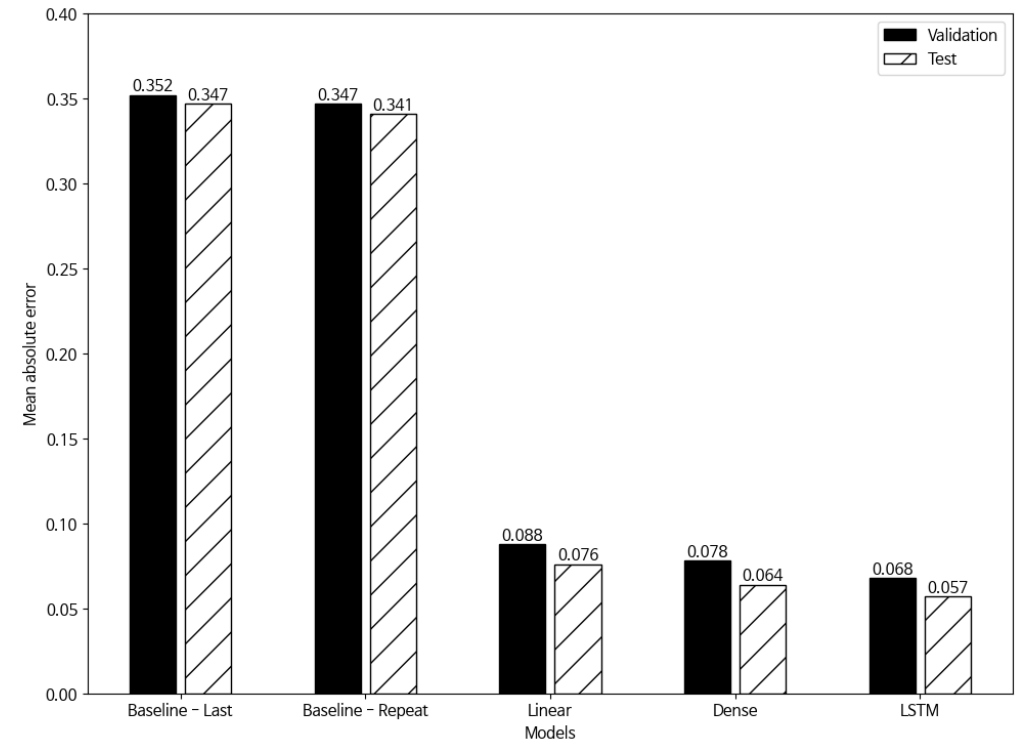
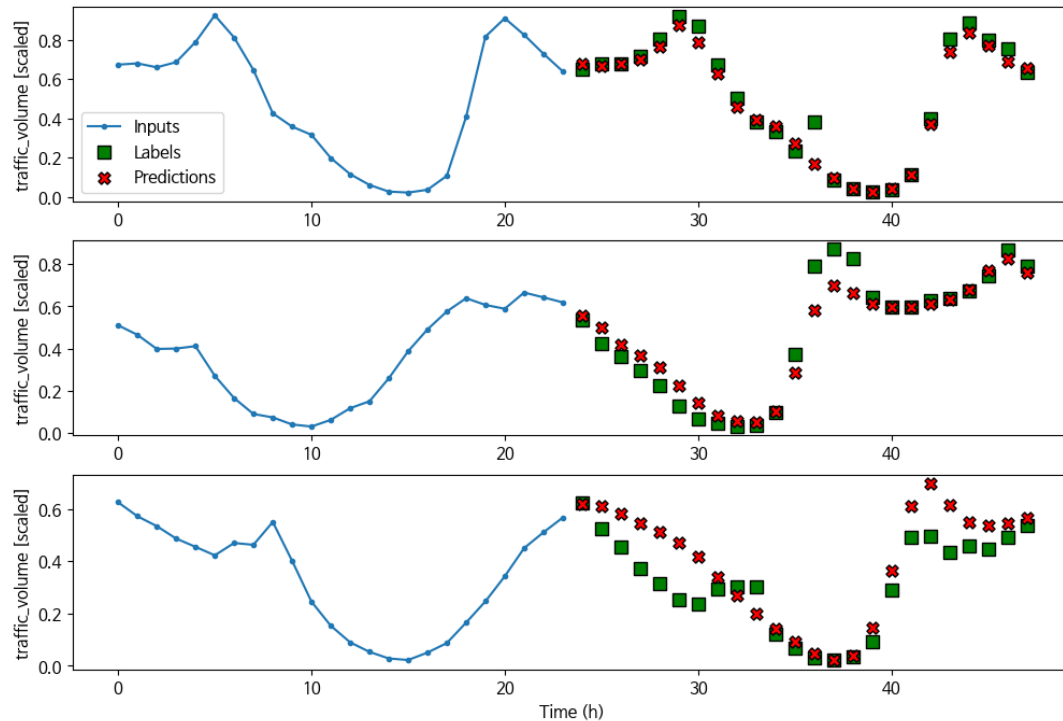
✓ 모델 예측과 평가지표 비교



4. LSTM을 활용한 시계열 예측

◆ 다중 단계 LSTM 모델

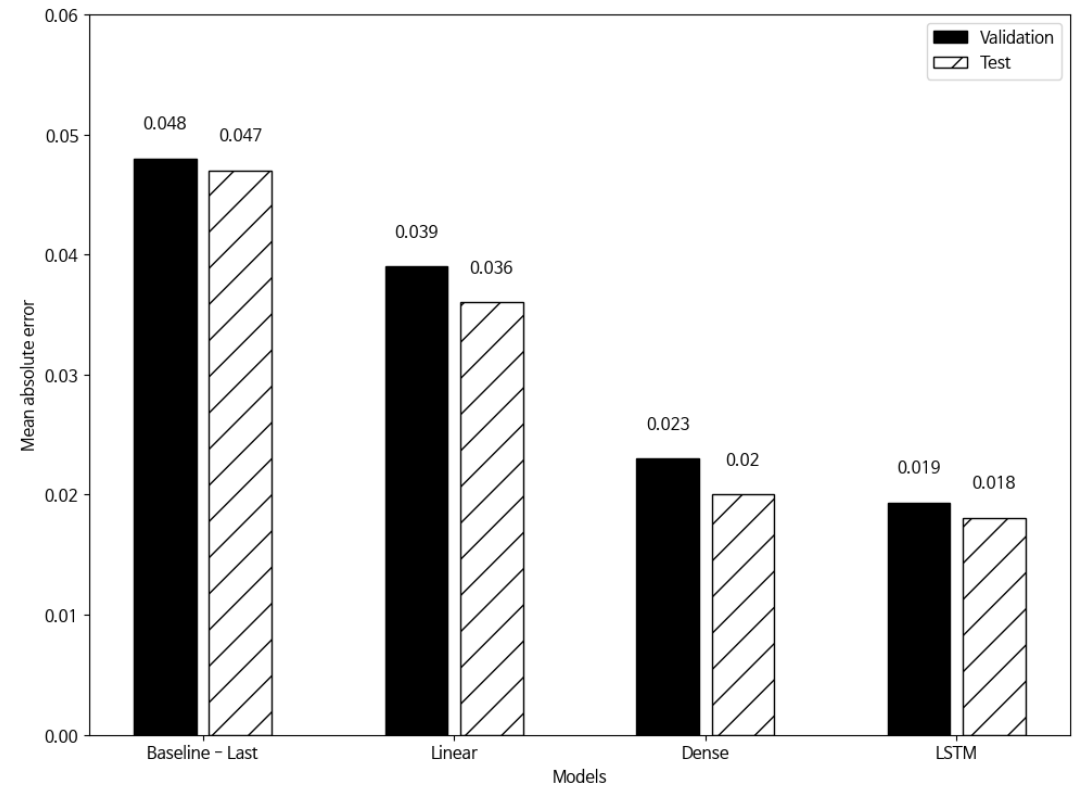
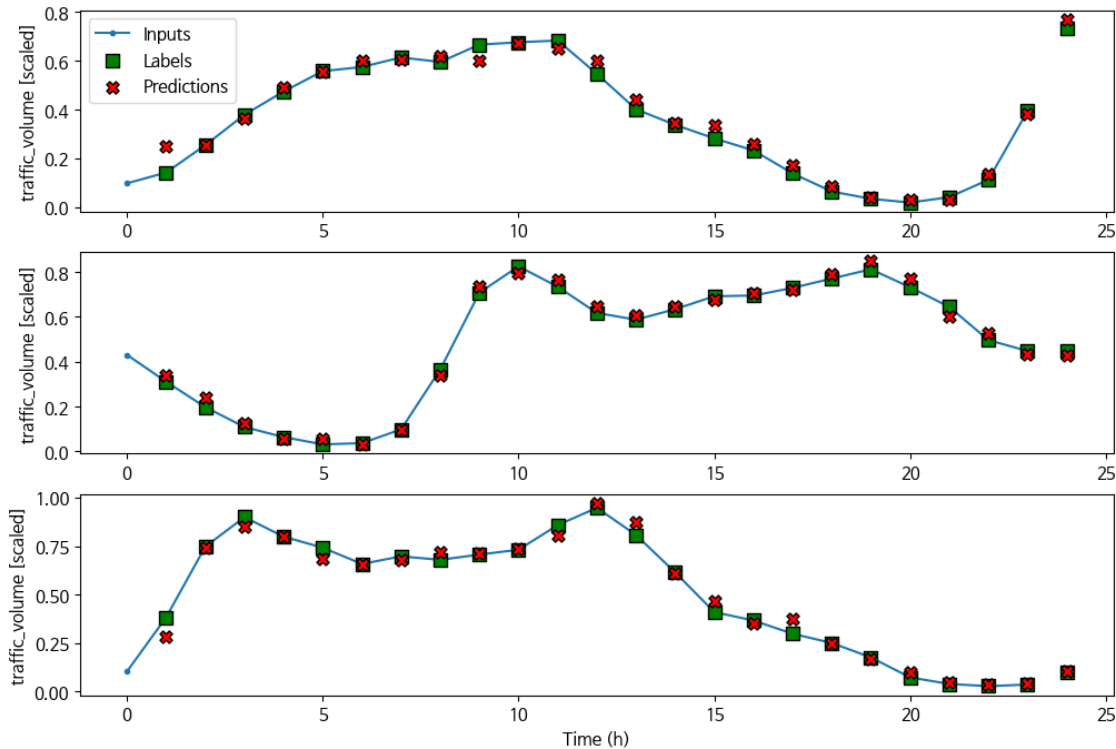
✓ 모델 예측과 평가지표 비교



4. LSTM을 활용한 시계열 예측

◆ 다중 출력 LSTM 모델

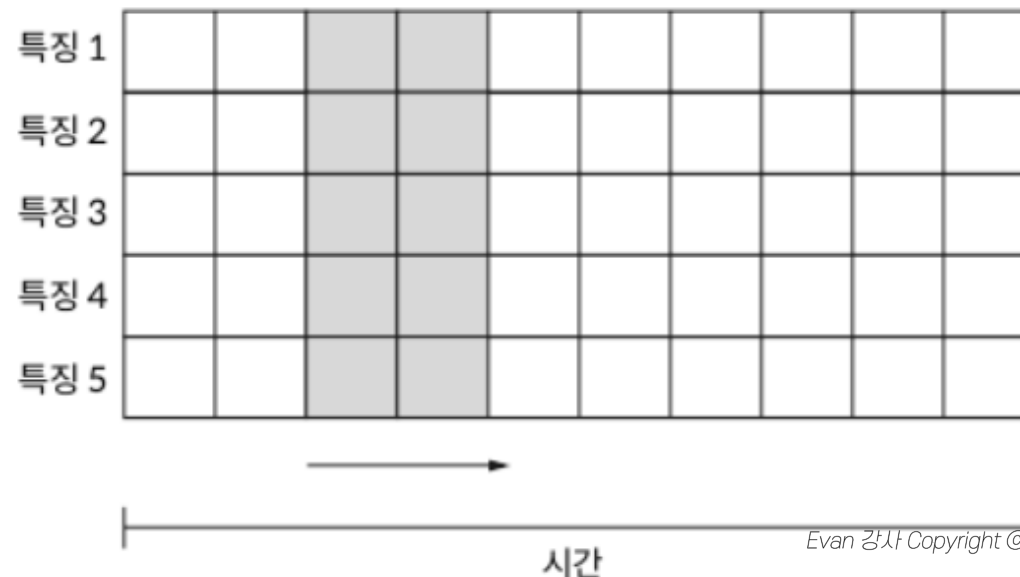
✓ 모델 예측과 평가지표 비교



5. CNN으로 시계열 필터링

◆ 합성곱 신경망(CNN^{Convolutional Neural Network})

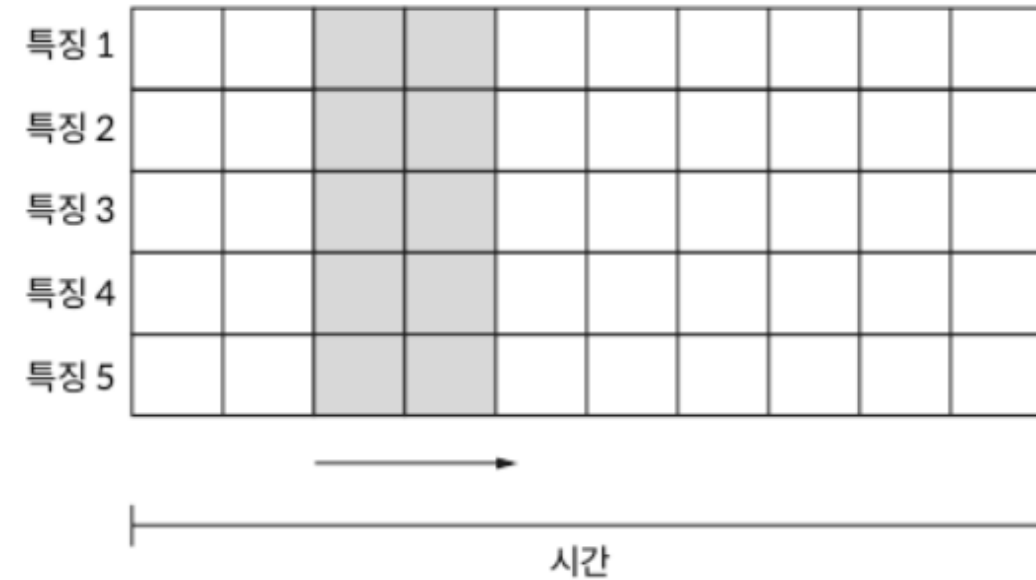
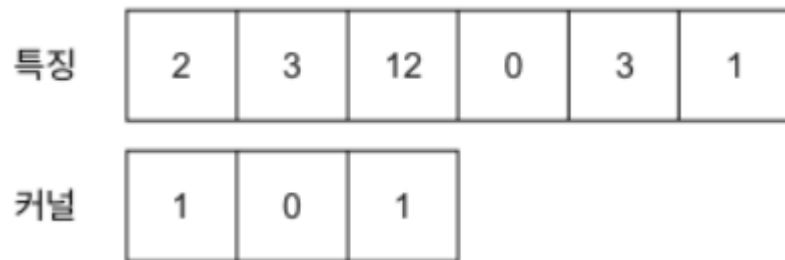
- ✓ 컴퓨터 비전 분야에 적용, 이미지 분류 및 이미지 분할을 위한 많은 알고리즘의 기반
- ✓ 노이즈에 강하며 합성곱(Convolution) 연산을 통해 시계열의 노이즈를 효과적으로 필터링
- ✓ 연산 병렬화, LSTM보다 훈련 속도가 빠름
- ✓ 커널^{kernel}(필터)은 시간 축을 따라 미끄러져 가면서 커널과 특징 사이의 점곱^{Dot Product}을 취함



5. CNN으로 시계열 필터링

◆ 합성곱 신경망(CNN^{Convolutional Neural Network})

- ✓ 한 행의 특징과 하나의 커널에 대한 간단한 예



5. CNN으로 시계열 필터링

◆ 합성곱 신경망(CNNConvolutional Neural Network)

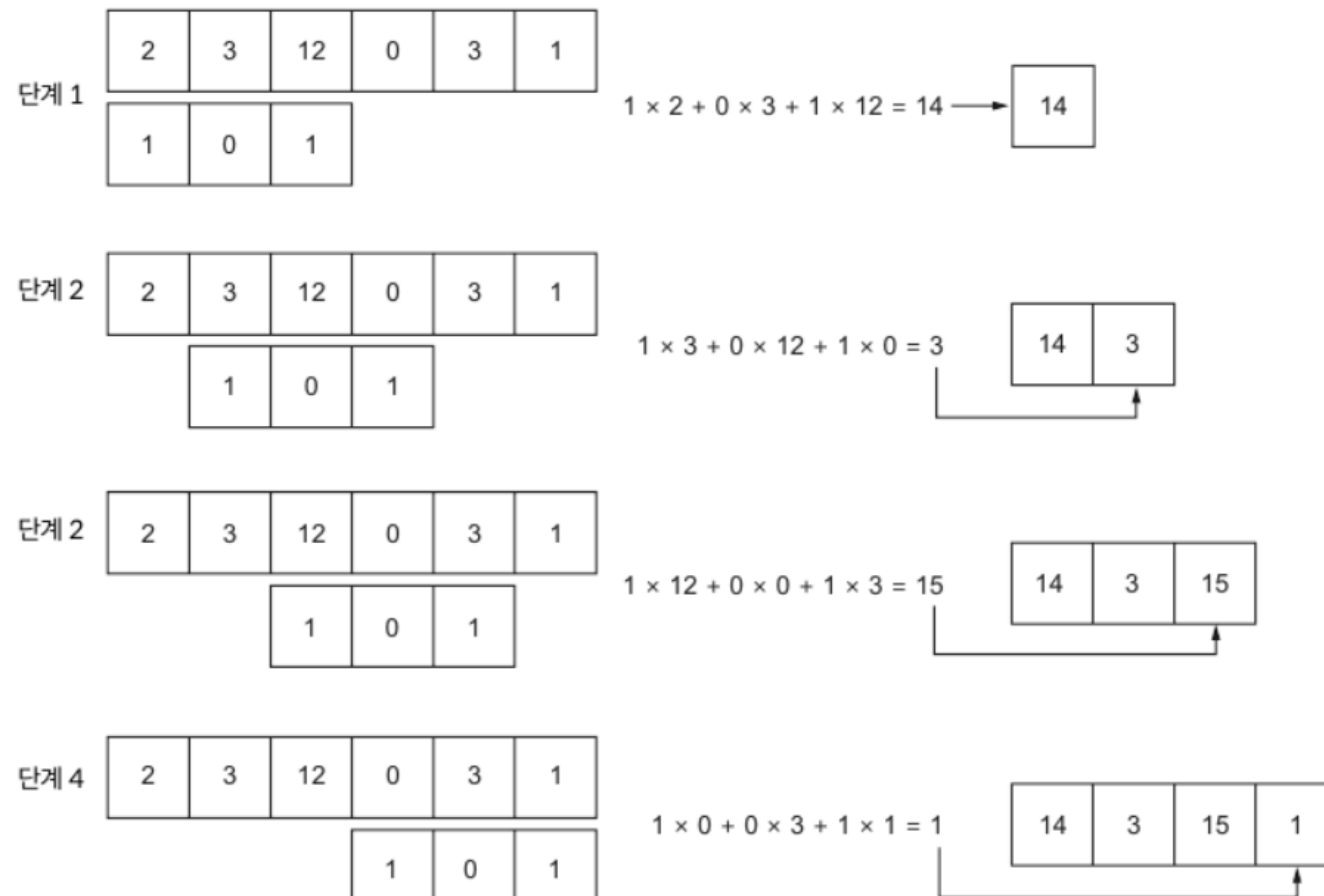
✓ 합성곱 연산

- 커널을 특징 벡터의 시작 부분에 정렬하고 커널과 커널에 정렬된 특징 벡터의 값 사이에서 내적 취함
- 보폭^{stride}, 작업 완료 후, 커널이 한 시간 단계만큼 오른쪽으로 이동
- 커널과 특징 벡터 사이에서 내적을 구하는데, 커널과 정렬된 값만 사용
- 커널은 다시 한 시간 단계 오른쪽으로 이동, 커널이 특징 벡터의 끝에 도달할 때까지 이 과정 반복

5. CNN으로 시계열 필터링

◆ 합성곱 신경망(CNNConvolutional Neural Network)

✓ 합성곱 연산



5. CNN으로 시계열 필터링

◆ 합성곱 신경망(CNNConvolutional Neural Network)

✓ 합성곱 연산

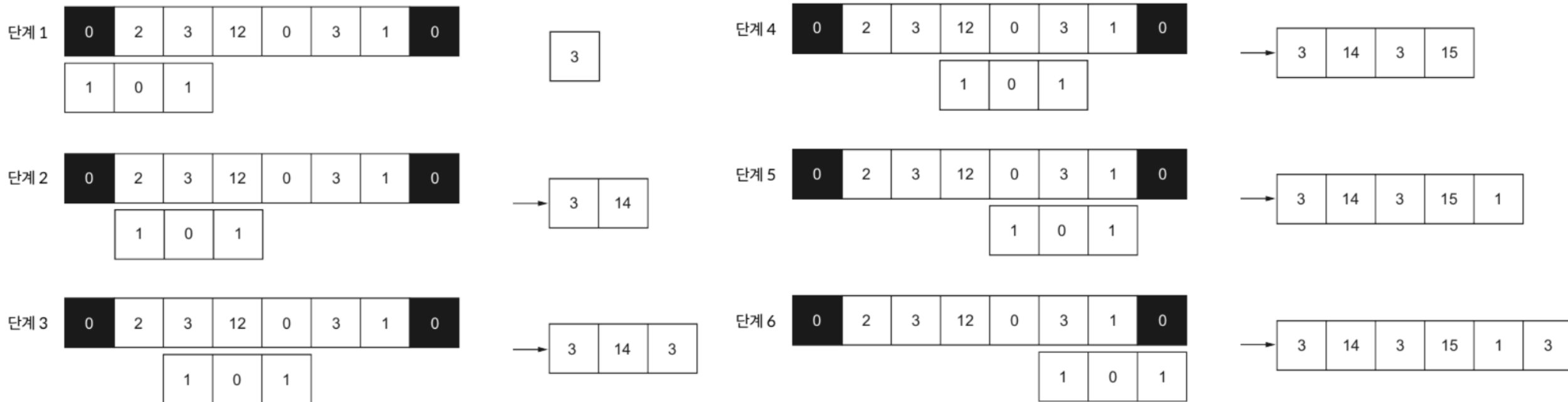
- 커널은 한쪽 방향(오른쪽)으로만 움직이며, 1D 합성곱으로 처리
- 이미지에서는 2D, 3D 합성곱을 자주 볼 수 있음
- 합성곱 계층
 - 특징 집합의 길이를 줄이고, 많은 합성곱 수행 시, 특징 공간이 계속 줄어듦
 - 네트워크의 계층 수 제한 및, 그 과정에서 너무 많은 정보 잃을 수 있기 때문에 문제 발생 가능성 존재
- 패딩 활용 : 특징 벡터의 앞뒤에 값을 추가하여 출력 길이를 입력 길이와 동일하게 유지
 - 패딩값은 일반적으로 0으로 처리

5. CNN으로 시계열 필터링

◆ 합성곱 신경망(CNNConvolutional Neural Network)

✓ 합성곱 연산 예시

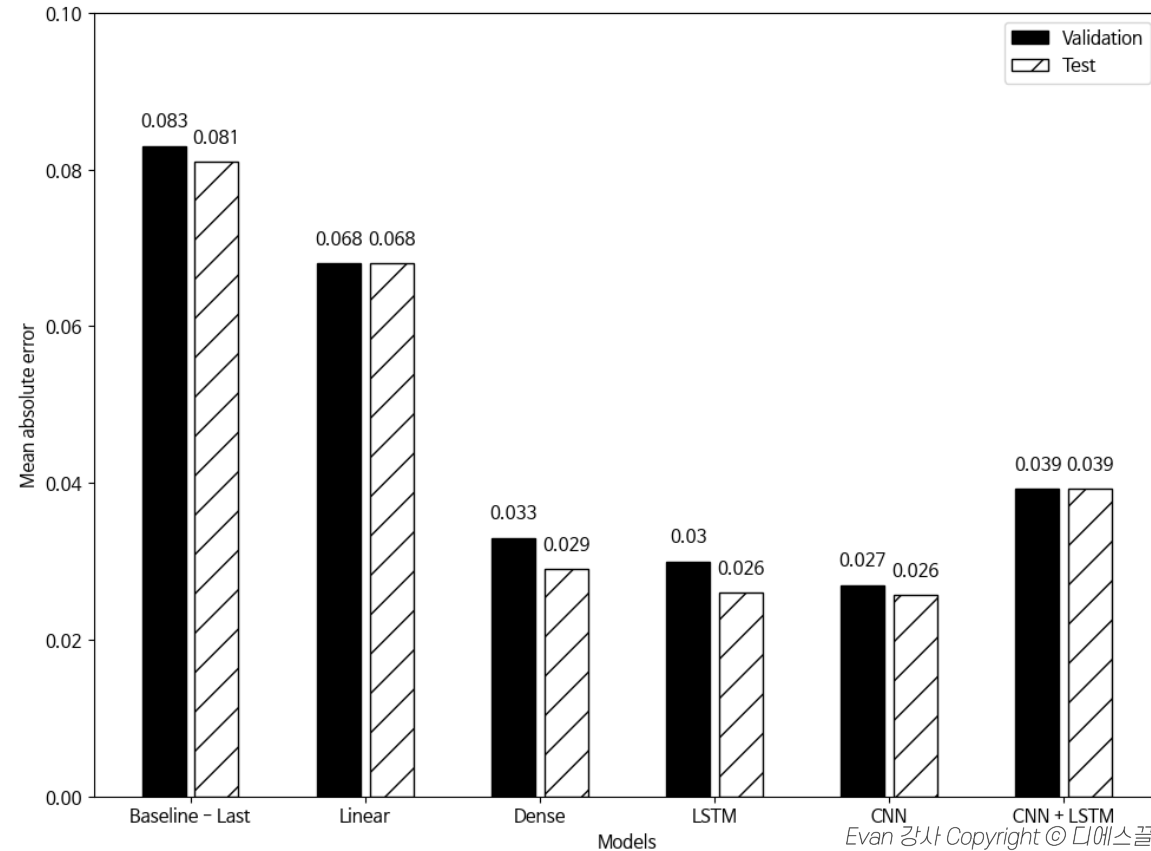
- 입력 벡터에 값이 0인 패딩 추가 → 0과의 곱셈은 무시될 수 있고, 출력값의 길이는 유지 가능



5. CNN으로 시계열 필터링

◆ 합성곱 신경망(CNNConvolutional Neural Network) – 단일 단계 모델

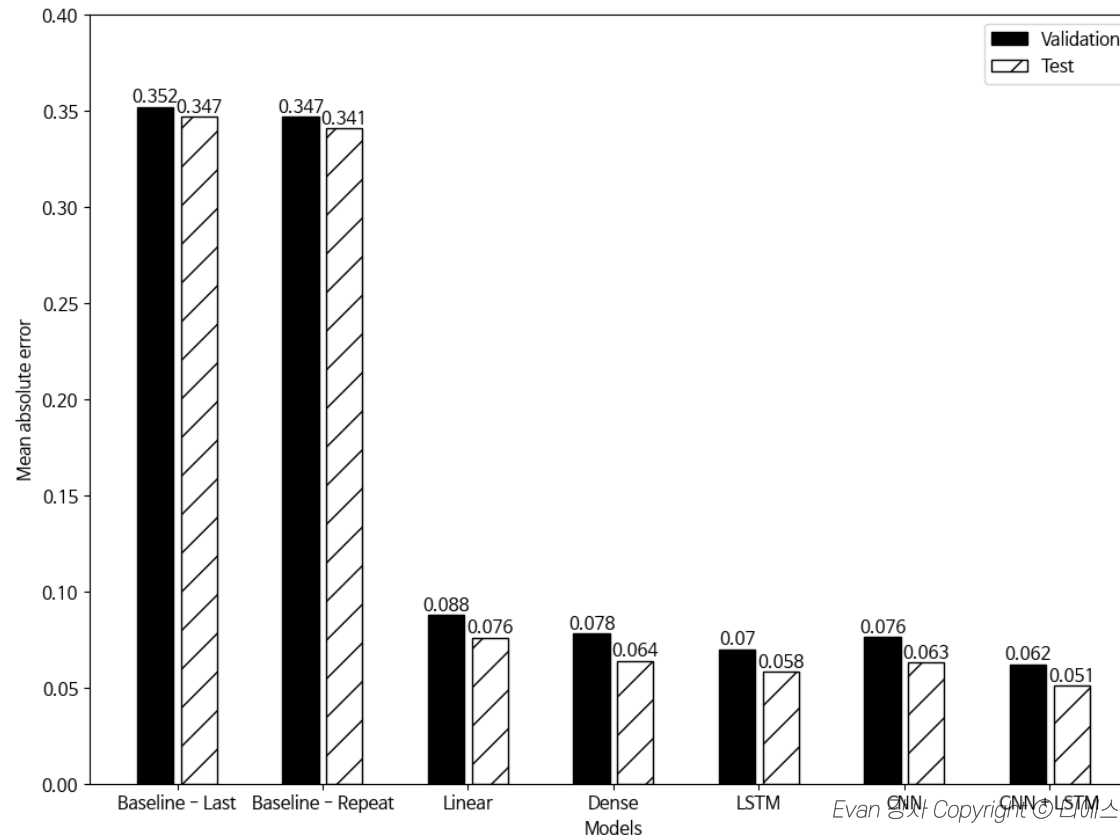
- ✓ 각 모델별 평가지표 확인 : CNN은 LSTM보다 성능은 좋지 못함



5. CNN으로 시계열 필터링

◆ 합성곱 신경망(CNNConvolutional Neural Network) – 다중 단계 모델

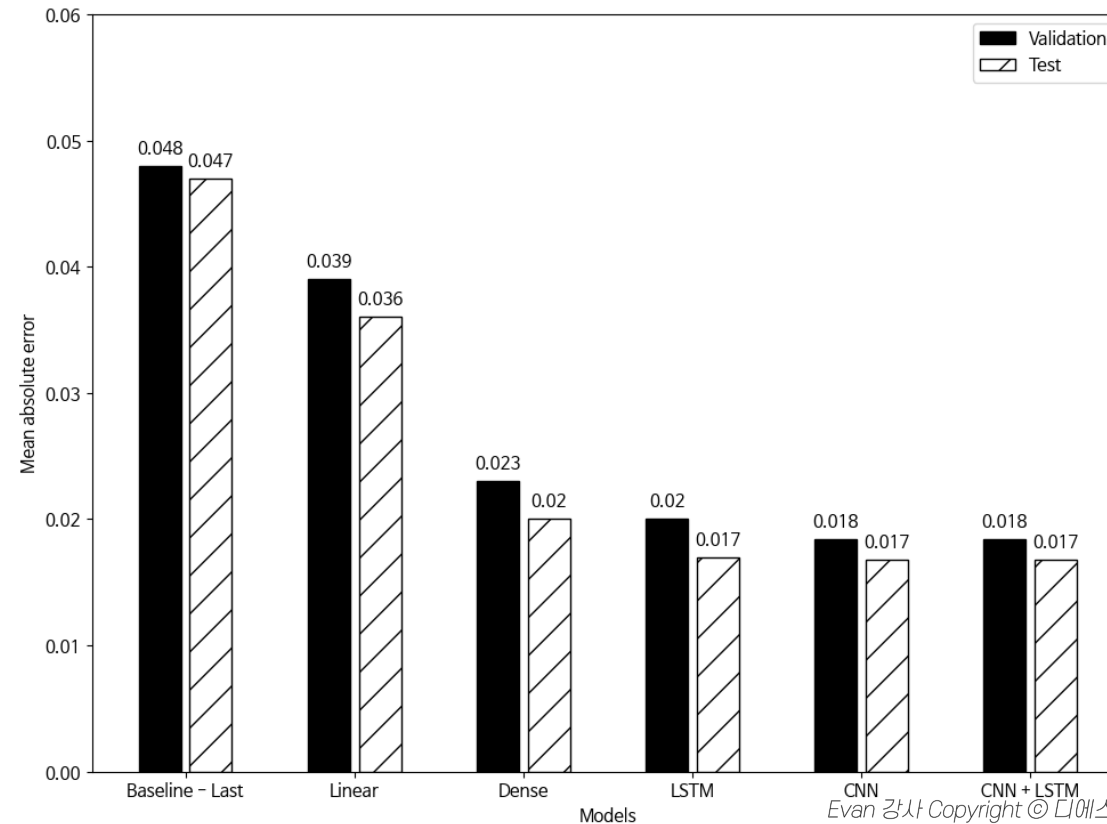
- ✓ 각 모델별 평가지표 확인 : 단일 단계 모델보다는 좋지 못함



5. CNN으로 시계열 필터링

◆ 합성곱 신경망(CNNConvolutional Neural Network) – 다중 출력 모델

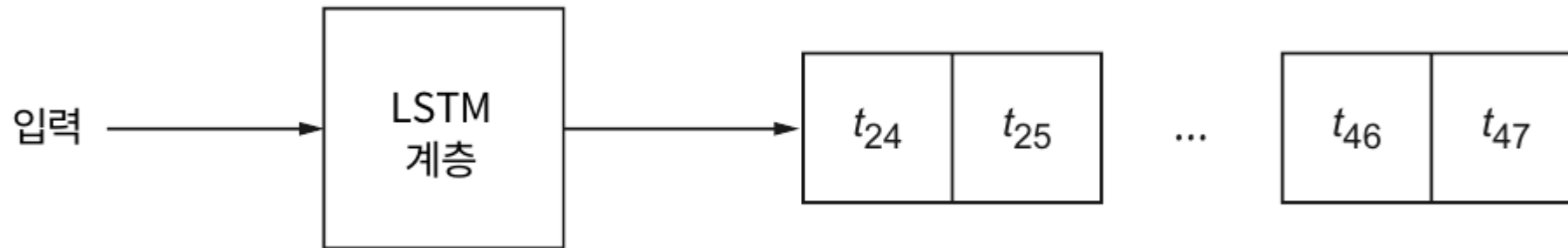
- ✓ 각 모델별 평가지표 확인 : LSTM, CNN, CNN + LSTM 모두 평가 지표 유사



6. 자기회귀 LSTM

◆ ARLSTM Autoregressive LSTM 개요

- ✓ 다중 단계 모델의 일반적인 아키텍처
 - LSTM 계층 대신, CNN 계층, Dense 계층으로 대체 가능

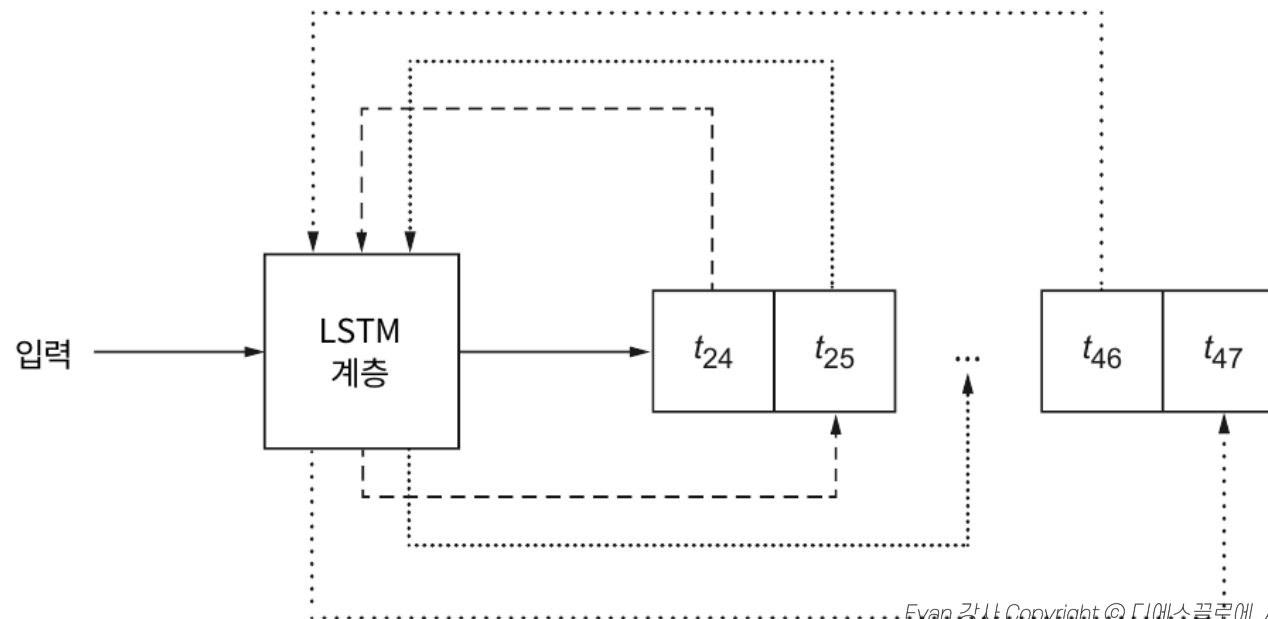


6. 자기회귀 LSTM

◆ ARLSTM Autoregressive LSTM 개요

✓ ARLSTM 모델의 아키텍처

- t_{24} 에서 첫 번째 예측 반환, 다시 모델로 전송되어 t_{25} 에서 예측 생성
- 원하는 출력 길이를 얻을 때까지 반복



6. 자기회귀 LSTM

◆ ARLSTMAutoregressive LSTM 개요

✓ ARLSTM 모델의 아키텍처의 장점

- 배열 길이를 매우 쉽게 다양하게 생성 (기존 모델은 아키텍처를 재 수정해야 함)
- 새로운 모델을 재훈련할 필요 없이, 시간, 일, 월과 같이 다양한 규모의 시계열 예측
- ARLSTM의 영향
 - WaveNet : <https://deepmind.google/research/breakthroughs/wavenet/>
 - DeepAR : <https://arxiv.org/abs/1704.04110>

6. 자기회귀 LSTM

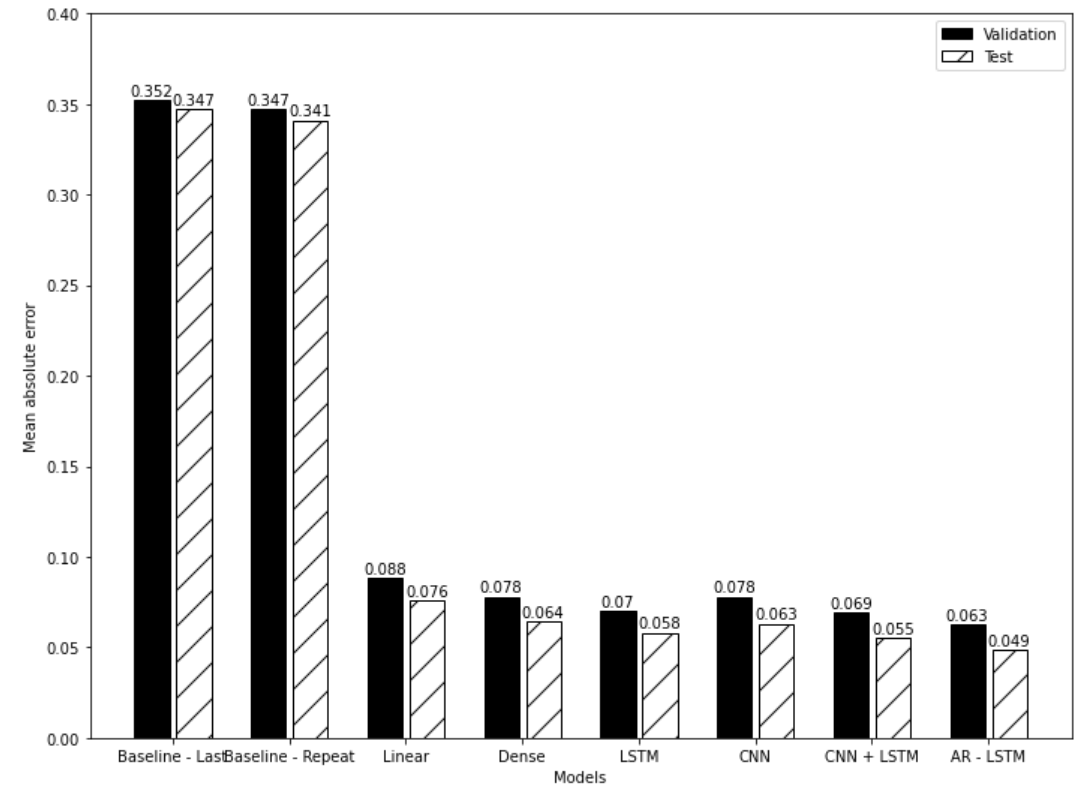
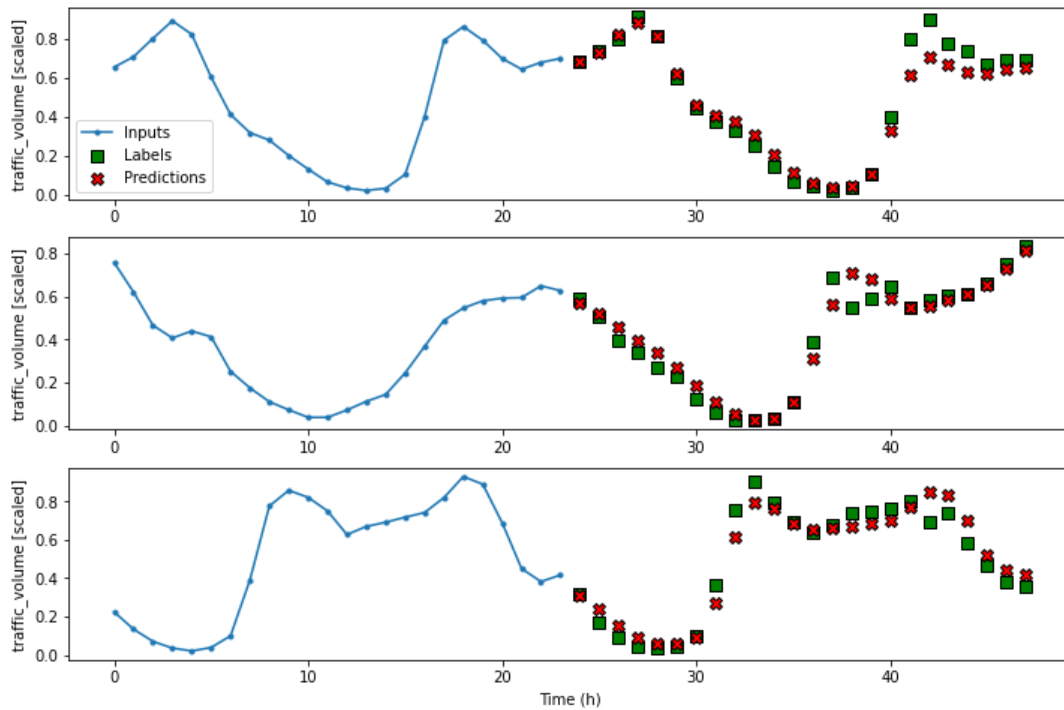
◆ ARLSTM^{Autoregressive LSTM} 개요

- ✓ ARLSTM 모델의 아키텍처의 단점
 - 오차의 누적 확인 필요
 - 예측과 실제값 사이에는 항상 약간의 차이가 존재

6. 자기회귀 LSTM

◆ ARLSTM Autoregressive LSTM 개요

✓ ARLSTM 모델 예측 및 비교



4. 대규모 예측 자동화 (Prophet)

1. 자동화된 예측 라이브러리 소개

◆ 주요 라이브러리

- ✓ pmdarima : <https://alkaline-ml.com/pmdarima/index.html>
- ✓ Prophet : <https://facebook.github.io/prophet/>
- ✓ NeuralProphet : <https://neuralprophet.com/>
- ✓ PyTorch Forecasting : <https://pytorch-forecasting.readthedocs.io/en/stable/>

1. 자동화된 예측 라이브러리 소개

◆ 주요 라이브러리 – pmdarima

- ✓ R의 인기 있는 auto.arima 라이브러리의 Python 구현
- ✓ ARMA, ARIMA, SARIMA
- ✓ 자동화된 p, q, P, Q 선택 및 AIC 최소화, ADF 테스트
- ✓ 통계적 시계열 예측

1. 자동화된 예측 라이브러리 소개

◆ 주요 라이브러리 – Prophet

- ✓ 메타 오픈소스 Meta Open Source 패키지
- ✓ 대규모 비즈니스 예측 위해 구축
- ✓ 비선형 데이터를 분석하는 최근 시장 추세에 적합
- ✓ 여러 계절적 요인의 효과 결합

1. 자동화된 예측 라이브러리 소개

◆ 주요 라이브러리 – NeuralProphet

- ✓ ARIMA와 같은 고전적인 모델과 신경망 조합
- ✓ 백엔드에서 PyTorch 사용
- ✓ Prophet과 유사한 API 사용
- ✓ 주요 논문 : <https://arxiv.org/abs/2111.15397>

1. 자동화된 예측 라이브러리 소개

◆ 주요 라이브러리 – PyTorch Forecasting

- ✓ 시계열 예측을 위한 최첨단 딥러닝 모델
- ✓ PyTorch 외, DeepAR, N-Beats 등과 같은 모델 사용 가능

2. Prophet 활용

◆ 모델 살펴보기

- ✓ 연간, 월간, 주간, 일별 등 여러 계절적 주기의 비선형 추세 고려한 예측 절차 구현
- ✓ 고급 사용자들은 모델 미세 조정 가능
- ✓ 수식은 아래와 같음

$$y(t) = g(t) + s(t) + h(t) + \epsilon_t$$

- ✓ 수식 표현
 - $y(t)$: 추세 구성요소
 - $g(t)$: 계절적 구성요소
 - $h(t)$: 공휴일 효과
 - ϵ_t : 정규분포되어 있는 오차 항

2. Prophet 활용

◆ 모델 살펴보기

✓ 수식 표현

- 추세 구성요소 $y(t)$: 시계열의 비주기적 장기 변화 모델링
- 계절적 구성요소 $g(t)$: 연간, 월간, 주간, 일별 등의 주기적 변화 모델링
- 공휴일 효과 $h(t)$: 불규칙적으로 발생하며 잠재적으로 하루 이상에 걸쳐 발생함
- ϵ_t : 위 세 가지 구성요소로 설명할 수 없는 값의 변화

$$y(t) = g(t) + s(t) + h(t) + \epsilon_t$$

2. Prophet 활용

◆ 모델 살펴보기

✓ 모델의 주요 특징

- ARIMA와의 차이점 : 데이터의 시간 의존성 미 고려
- 기본 프로세스를 찾기보다는 데이터에 곡선을 피팅하는 것에 더 가까움
- 이상값과 누락된 데이터에 강함
- 여러 계절적 기간을 고려하기 위해 푸리에 급수 사용하여 여러 주기적 효과 모델링
- 계절적 구성요소 수식 예시 (P = 계절적 기간, N = 푸리에 급수의 항 개수)

$$s(t) = \sum_{n=1}^N \left(a_n \cos\left(\frac{2\pi nt}{P}\right) + b_n \sin\left(\frac{2\pi nt}{P}\right) \right)$$

2. Prophet 활용

◆ 모델 살펴보기

✓ 계절성 구성 요소

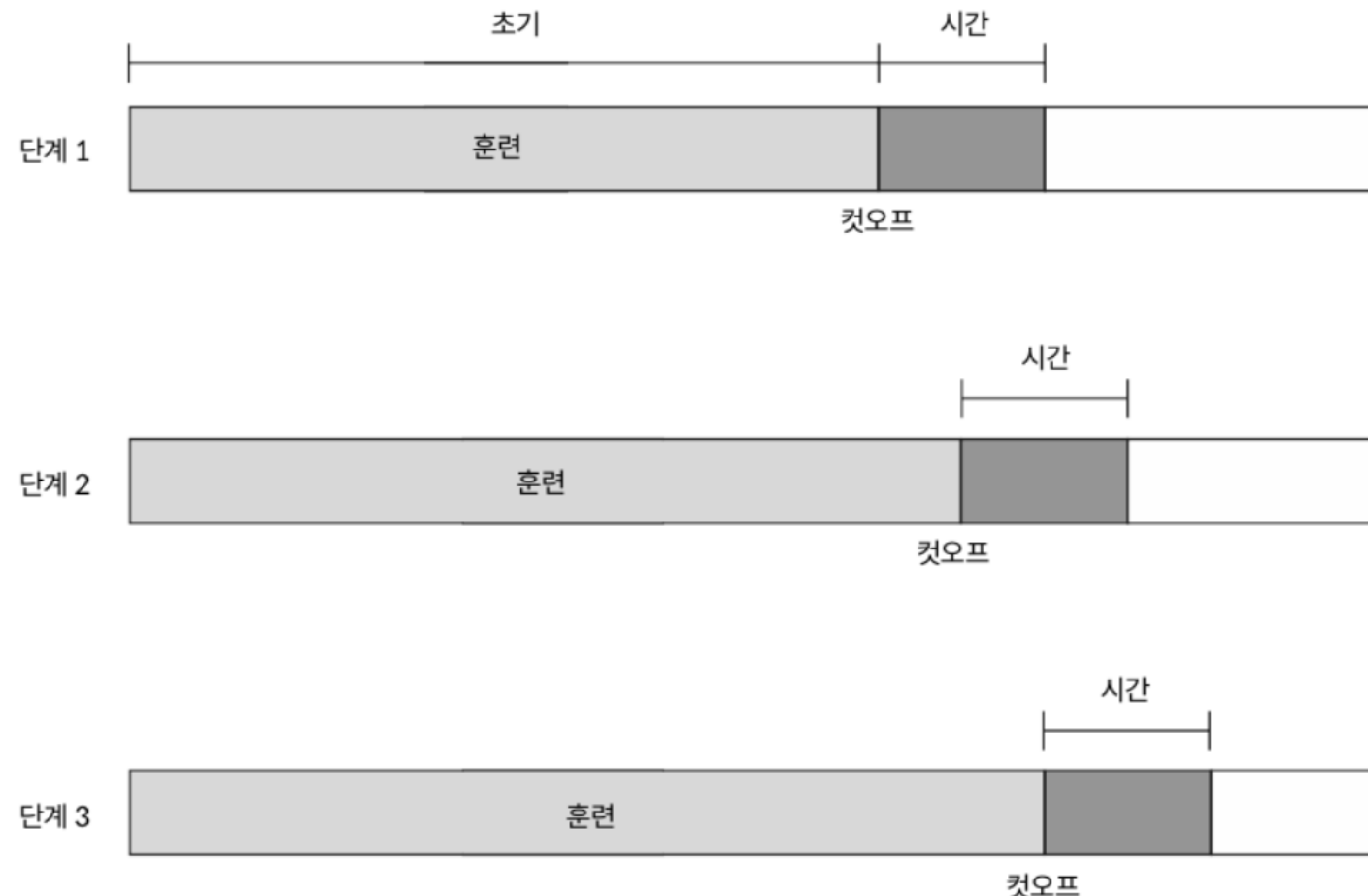
- 연간 계절성이 있는 경우, $P = 365.25$, $N = 10$ (기본값)
- 주간 계절성이 있는 경우, $P = 7$, $N = 3$ (기본값)

$$s(t) = \sum_{n=1}^N \left(a_n \cos\left(\frac{2\pi nt}{P}\right) + b_n \sin\left(\frac{2\pi nt}{P}\right) \right)$$

2. Prophet 활용

◆ 모델 주요 구성 요소

✓ 교차검증



2. Prophet 활용

◆ 모델 주요 구성 요소

✓ 하이퍼파라미터 튜닝

종류	설명	권장 사용 범위
changepoint_prior_scale	- Prophet에서 가장 영향력 있는 파라미터 - 추세의 유연성을 결정하며 변곡점에서의 추세 변화를 조절 - 값이 작으면 과소적합, 값이 크면 과적합 가능성 존재	[0.001, 0.01, 0.1, 0.5]
seasonality_prior_scale	- 계절성의 유연성을 설정 - 값이 크면 작은 변동에 맞춰짐, 값이 작으면 부드러운 계절성 생성	[0.01, 0.1, 1.0, 10.0]
holidays_prior_scale	- 휴일 효과의 유연성을 설정 - seasonality_prior_scale과 동일한 방식으로 작동 - 휴일 효과 강도 조절 가능	[0.01, 0.1, 1.0, 10.0]
seasonality_mode	- 계절성 모드를 additive 또는 multiplicative로 설정 - 기본값 additive, 계절 변동이 커지면 multiplicative 선택 - 시계열 시각화를 통해 판단하거나 튜닝 과정에 포함	additive, multiplicative



THANK YOU.

